

BACKEND

Utilizzando NestJs, JWT

Cosa sono le API e principali tipologie

- **API (Application Programming Interface)** è un insieme di regole che **permette a due applicazioni di comunicare tra loro**.
- il client (browser, app mobile, software) effettua una richiesta;
- l'API riceve la richiesta;
- il server elabora i dati;
- l'API restituisce una risposta.
- **Esempio:**
 - App Mobile → API → Database
 - Quando visualizzi un elenco di clienti o inserisci un nuovo ordine, molto spesso stai utilizzando delle API.

Principali tipologie di API

REST API

Basate sul protocollo HTTP.

Utilizzano metodi come GET, POST, PATCH e DELETE.

Sono le API più diffuse nei software gestionali e nelle applicazioni web.

GraphQL

Permette al client di scegliere esattamente quali dati ricevere.

Riduce il numero di chiamate necessarie.

Utilizzato nei social e in applicazioni con frontend complessi.

WebSocket API

Consentono una comunicazione bidirezionale in tempo reale.

Ideali per chat, notifiche e applicazioni live.

gRPC

Utilizza il protocollo HTTP/2 e dati binari.

Molto veloce ed efficiente.

Diffuso nelle architetture a microservizi.

- gRPC è pensato principalmente per comunicare tra servizi backend ad alte prestazioni

SOAP

Basato su XML.

Ancora utilizzato in alcuni sistemi legacy, banche e pubbliche amministrazioni.

REST API

API – **A**pplication **P**rogramming **I**nterface

RESTful – **R**epresentational **S**tate **T**ransfer

A livello base, un'API è un meccanismo che consente ad un'applicazione o servizio di accedere ad una risorsa all'interno di un'altra applicazione o servizio.

L'applicazione o servizio che accede alle risorse è il client.

L'applicazione o servizio che contiene la risorsa è il server.

Le API rest possono essere sviluppate in diversi linguaggi e supportano diversi formati di dati ma devono seguire i sei vincoli architetturali di progettazione REST.

VINCOLI ARCHITETTURALI REST

1. **INTERFACCIA UNIFORME:** tutte le richieste per la stessa risorsa devono avere lo stesso aspetto, indipendentemente dalla provenienza della richiesta stessa. L'API REST deve quindi garantire che lo stesso dato appartenga ad un solo URI (Uniform Resource Identifier).
2. **DISACCOPPAMENTO CLIENT-SERVER:** le applicazioni client e server devono essere totalmente indipendenti l'una dall'altra. L'unica informazione che il client deve conoscere è l'URI della risorsa, non ha bisogno di sapere come i dati sono memorizzati nel server. Non può interagire con l'applicazione server in nessun altro modo. L'applicazione server, a sua volta, non può modificare l'applicazione client se non passandole i dati richiesti via HTTP.
3. **CONDIZIONE DI STATELESS:** le REST API sono stateless. Ogni richiesta è indipendente e deve includere tutte le informazioni necessarie per elaborarla. Il server non può memorizzare dati relativi alla richiesta del client e non mantiene lo stato delle sessioni tra le richieste.
4. **POSSIBILITÀ DI MEMORIZZAZIONE NELLA CACHE:** le risorse possono essere memorizzate nella cache lato server o client. Le API devono indicare se una risposta è cacheabile o meno.
5. **ARCHITETTURA DI SISTEMA A LIVELLI:** le chiamate e risposte passano attraverso diversi livelli, non sempre le applicazioni si connettono direttamente. Possono esistere diversi intermediari e le API REST devono essere progettate in modo tale che né il client né il server sappiano se comunicano con l'app finale o con un intermediario.

COME FUNZIONANO LE REST API

Passaggi generali di qualsiasi chiamata REST API:

1. Il **client invia una richiesta al server** formattandola in modo che il server la comprenda (ogni API ha una sua documentazione di riferimento)
2. Il **server autentica il client** e conferma che ha il diritto di effettuare la richiesta
3. Il server **riceve la richiesta** e **la elabora** internamente
4. Il **server risponde al client**. La risposta contiene informazioni circa l'esito della richiesta e ogni altra informazione richiesta dal client.

Le API REST comunicano tramite **richieste HTTP** per eseguire le funzioni di database standard come la creazione, lettura, aggiornamento e eliminazione di record (**CRUD**) all'interno di una risorsa.

Ad esempio, un API REST utilizzerà una richiesta GET per recuperare un record. Una richiesta POST crea un nuovo record.

Lo stato di una risorsa in un determinato istante è noto come **rappresentazione della risorsa**. Queste informazioni possono essere inviate al client praticamente in qualsiasi formato. Il più comune è **JSON** perché indipendente dal linguaggio di programmazione e perché leggibile sia dall'uomo che dalle macchine.

CONTENUTO DELLA RICHIESTA API

COMPONENTI FONDAMENTALI

- **ENDPOINT (URL della risorsa):** è l'indirizzo web della risorsa con cui vogliamo interagire.
Esempio: <https://api.example.com/users/123> per richiedere i dettagli dell'utente con ID 123
- **METODO HTTP:** indica l'operazione che vogliamo eseguire sulla risorsa. I principali metodi HTTP nelle REST API sono:
 - **GET:** recupera una risorsa
 - **POST:** crea una risorsa. Inviare la stessa richiesta POST più volte ha l'effetto di creare la stessa risorsa più volte.
 - **PUT:** aggiorna una risorsa esistente (sostituzione completa)
 - **PATCH:** aggiorna parzialmente una risorsa
 - **DELETE:** elimina una risorsa

- **HEADERS (intestazioni HTTP):** forniscono informazioni aggiuntive sulla richiesta (metadati), come il formato dei dati o autenticazione.

ESEMPIO DI HEADER COMUNE

Content-Type: application/json

Authorization: Bearer <token>

User-Agent: MyApp/1.0

- **BODY (corpo della richiesta):** contiene i dati che si vogliono inviare al server (solo per POST, PUT, PATCH).
- **QUERY PARAMETERS** (parametri della query): vengono aggiunti all'URL per filtrare o personalizzare la richiesta
<https://api.example.com/users?limit=10&sort=desc>
- **PATH PARAMETERS** (parametri nel percorso) – opzionale: sono parte dell'URL e servono per identificare una specifica risorsa

<https://api.example.com/users/123>

CONTENUTO DELLA RESPONSE

COMPONENTI FONDAMENTALI

- **STATUS CODE:** risultato della richiesta HTTP

HEADERS (intestazioni della risposta)

Contengono metadati sulla risposta.

Esempio:

Content-Type: application/json

Content-Length: 256

Cache-Control: no-cache

BODY (corpo della risposta)

Contiene i dati restituiti dal server (di solito JSON nelle API moderne).

Esempio:

```
{  
  "id": 123,  
  "name": "Mario Rossi",  
  "email": "mario@example.com"  
}
```


HEADER DELLA REQUEST

- Può contenere informazioni extra per la request, come:

PARAMETRO	FUNZIONE	ESEMPIO
Content-Type	Specifica formato del contenuto del body	application/json
Accept	Tipo di contenuto che il client si aspetta come risposta	application/json, text/html
Authorization	Autorizzazione tramite token	bearer eyjhbgtci0ijiuzi1.....
User-Agent	Informazioni sul client che fa la richiesta	postmanruntime/7.30.0
Cache-Control	Indica se contenuto può essere memorizzato	no-cache, max-age=0
X-Requested-With	Spesso usato per richieste AJAX	xmlhttprequest
Origin	Origine della richiesta (per CORS)	https://frontend.miosito.it
Cookie	Dati di sessione inviati con la richiesta	sessionid=abc123

BODY DELLA REQUEST

Può avere diversi formati a seconda del tipo di contenuto che si vuole inviare o ricevere. Il formato va specificato nell'header così che il server o client sappia come interpretare il body della richiesta o della risposta.

FORMATO	MIME TYPE	USO PRINCIPALE
JSON	Application/json	Dati strutturati, standard moderno
XML	Application/xml	Sistemi legacy, SOAP
URL-encoded	Application/x-www-form-urlencoded	Form HTML classici
Multipart/form-data	Multipart/form-data	Upload file
Plain Text	Text/plain	Testo semplice
HTML	Text/html	Risposte di pagine web
Binari (PDF, img...)	Application/pdf, image/png, ecc	File e media

BODY - FORMATI

JSON (il più comune)

- Usato per scambio dati tra client e server in modo leggibile e strutturato
- Leggero, leggibile, supportato ovunque

```
json
{
  "nome": "Luca",
  "email": "luca@example.com"
}
```

XML (Extensible Markup Language)

- Strutturato, formale ma più verboso del JSON

```
xml
<utente>
  <nome>Luca</nome>
  <email>luca@example.com</email>
</utente>
```

Form URL Encoded

- Usato per form HTML tradizionali, richieste leggere
- Simile ai parametri di una query string

```
ini
nome=Luca&email=luca%40example.com
```

BODY - FORMATI

Multipart/Form-Data

- Usato per Upload di file (es. immagini, documenti).
- diviso in “parti” con intestazioni per ciascuna sezione.

```
--boundary
Content-Disposition: form-data; name="foto"; filename="profilo.jpg"
Content-Type: image/jpeg
```

Plain Text

- Usato per Inviare o ricevere semplice testo, log, messaggi brevi.

```
Questo è un messaggio di testo semplice.
```

HTML

- Usato per ricevere una risposta in formato HTML da server web

```
<html><body><h1>Benvenuto</h1></body></html>
```

FILE BINARI (IMMAGINI, PDF, AUDIO, ECC.)

- Usato per scaricare file, immagini, audio.
- Contiene dati binari non leggibili da uomini.

REQUEST + RESPONSE: ESEMPIO

```
POST /api/utenti HTTP/1.1
Host: api.miosito.it
Authorization: Bearer xyz123
Content-Type: application/json

{
  "nome": "Luca",
  "email": "luca@example.com"
}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 1,
  "nome": "Luca",
  "email": "luca@example.com"
}
```

METODI DI AUTENTICAZIONE REST API

DIFFERENZE TRA AUTENTICAZIONE ED AUTORIZZAZIONE	
AUTENTICAZIONE	AUTORIZZAZIONE
Determina se gli utenti o il client sono chi dicono di essere	Determina cosa un utente o client può o non può fare una volta autenticato
Richiede all'utente di fornire delle credenziali per essere riconosciuto	Verifica se l'accesso alle risorse è consentito attraverso una serie policies e regole basate su ruoli, scopes o permessi
Solitamente avviene prima dell'autorizzazione	Generalmente avviene dopo l'autenticazione
Generalmente trasmette info attraverso un ID token	In genere trasmette info attraverso un Access Token

Le richieste devono essere autenticate prima che il server possa inviare una risposta.

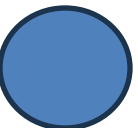
Le API REST presentano quattro **metodi di autenticazione** comuni:

- **AUTENTICAZIONE HTTP**: l'HTTP definisce alcuni schemi di autenticazione che possono essere utilizzati direttamente per implementare API REST. Tra questi:
 - **Autenticazione di base (Basic Authentication)**: il client invia username e password nell'header Authorization di ogni richiesta, codificandoli con base64 (convertiti in un set di 64 caratteri. NB: codificati, non cifrati). Poco sicuro perché i dati viaggiano in chiaro, va usato su HTTPS.
 - **Autenticazione bearer**: il controllo di accesso avviene tramite token bearer. Questo è solitamente una stringa crittografata di caratteri che il server genera in risposta alla richiesta di login. Il client invia il token nell'header della richiesta per accedere alle risorse. I token hanno una durata temporale limitata.

- **CHIAVI API:** sono **valori unici che vengono assegnati dal server ad ogni utente al primo accesso**. Il client usa questa chiave per autenticarsi in ogni richiesta. Le chiavi API sono meno sicure perché possono essere esposte se trasmesse in chiaro o memorizzate nel client. Può essere utile per API pubbliche o interne con accesso limitato. Consentono il monitoraggio del flusso degli utenti.
- **Autenticazione con JWT (JSON Web Token):** meccanismo **stateless** per autenticazione delle API, non prevede la memorizzazione dei dati del client sul server. Quando l'utente accede all'applicazione, il server API crea un JWT che **contiene l'identità dell'utente**. Il JWT può essere firmato digitalmente o crittografato e deve essere incluso in ogni richiesta.
- **OAuth 2.0** (Autenticazione Delegata): **il client ottiene un access token dopo l'autenticazione su un provider di identità** (IdP). Protocollo di autorizzazione utilizzato da Google, Facebook, ecc.
L'utente effettua il login tramite un provider – il provider restituisce un access token – il client usa il token per accedere alle API.
È un sistema sicuro, usato per Single Sign-On (SSO), non richiede gestione di password ma è complesso da implementare e richiede un provider di identità.
- **HMAC (Hash-based Message Authentication Code):** il client genera una firma digitale basata su una chiave segreta. La firma viene verificata dal server per autenticare la richiesta. È un metodo sicuro perché i dati non vengono trasmessi in chiaro ed è difficile da falsificare. Richiede però più risorse computazionali.

In **NestJS** sono tutti implementabili, ma non tutti sono “già pronti” nel core.
Quello più “standard” e già supportato in NestJS è **JWT**.

Meccanismo	In NestJS è già presente?	Come si fa
Chiavi API	No, custom	Guard che controlla header tipo <code>x-api-key</code>
JWT / Bearer Token	Sì, molto supportato	<code>@nestjs/jwt + PassportStrategy('jwt')</code>
OAuth 2.0	Supportato via Passport	Strategie tipo Google, GitHub, Keycloak, Auth0
HMAC	No, custom	Guard che verifica firma con crypto



COSA CONTIENE LA RISPOSTA DEL SERVER – COMPONENTI PRINCIPALI

- **STATUS CODE:** contiene codice di stato che indica l'esito della richiesta. I codici 2xx indicano il buon esito della richiesta, i codici 4xx e 5xx indicano errori, i codici 3xx indicano reindirizzamenti URL.
- **HEADERS (intestazioni HTTP):** forniscono metadati relativi alla risposta stessa. Comunemente includono:
- **BODY:** contiene i dati veri e propri inviati dal server al client. Generalmente in formato JSON o XML (ma può essere in uno dei formati già presentati per la request). Solitamente contiene:
 - Dati della risorsa richiesta
 - Eventuale messaggio di errore come Invalid Parameters o Resource not found
 - **MESSAGGIO DI ERRORE** (se applicabile): se la richiesta non ha avuto successo, il corpo della risposta può contenere info dettagliate sull'errore. Generalmente si usa una struttura JSON o XML che descrive l'errore.

ESEMPIO DI
RISPOSTA 400

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
X-Rate-Limit: 500

{
  "status": "error",
  "message": "Missing required parameter: email"
}
```

VEDI CODICI DI STATO HTTP >>>

HEADERS DELLA RESPONSE

- Content-Type: tipo di contenuto della risposta. Ad es.: application/json, application/xml, text/html, ecc.
- Content-Length: dimensioni del contenuto in bite. Es. 1287
- Date: data e ora della risposta. ES: Wed, 25 Jun 2025 12:30:00 GMT
- Cache-Control: gestisce cache della risposta. ES: no-cache, max-age=3600
- Etag: identificatore univoco della risorsa (per cache validation). ES: "abc123xyz"
- Expires: dati di scadenza della risorsa memorizzata in cache
- Last-Modified: data ultima modifica alla risorsa
- Set-Cookie: invia cookie da memorizzare nel browser
- Access-Control-Allow-Origin: autorizza richieste cross-domain
- Content-Encoding: indica la compressione del contenuto. ES. gzip, br
- Location: indica l'URL della risorsa appena creata nel caso di chiamate POST
- WWW-Authenticate: specifica il tipo di autenticazione richiesta. ES: bearer realm="Access to protected API"
- Retry-After: indica quanto attendere prima di rifare la richiesta. ES: 120 (secondi) o data
- X-RateLimit-Remaining: Quante richieste utente può ancora fare
- Laravel prevede altri parametri aggiuntivi all'header delle response.

```
http

HTTP/1.1 200 OK
Date: Wed, 25 Jun 2025 12:30:00 GMT
Content-Type: application/json
Content-Length: 128
Cache-Control: no-cache
Access-Control-Allow-Origin: *
Set-Cookie: sessionId=xyz123; HttpOnly
```

HTTP Status Code

PIÙ COMUNI

1XX - INFORMAZIONI

- 100** Continue
- 101** Switching Protocols
- 102** Processing
- 103** Early Hints

3XX - REDIRECTION

- 300** Multiple Choices
- 301** Moved Permanently
- 302** Found
- 303** See Other
- 304** Not Modified
- 305** Use Proxy
- 306** Switch Proxy
- 307** Temporary Redirect
- 308** Permanent Redirect

2XX - SUCCESSO

- 200** OK - GET, POST, PUT,
- 201** DELETE Created - POST
- 202** Accepted
- 203** Non-Authoritative
- 204** No Content
- 205** Reset Content
- 206** Partial Content

5XX - SERVER ERRORS

- 500** Internal Server Error
- 501** Not Implemented
- 502** Bad Gateway
- 503** Service Unavailable
- 504** Gateway Timeout
- 505** HTTP Version Not Supported
- 507** Insufficient Storage
- 511** Network Authentication Required

4XX - CLIENT ERRORS

- 400** Bad Request
- 401** Unauthorized
- 402** Payment Required
- 403** Forbidden
- 404** Not Found
- 405** Method Not Allowed
- 407** Proxy Authentication Required
- 408** Request Timeout
- 409** Conflict
- 410** Gone
- 412** Precondition Failed
- 416** Request Range Not Satisfiable
- 417** Expectation Failed
- 422** Unprocessable Entity
- 423** Locked
- 429** Too Many Requests
- 451** Unavailable For Legal Reasons

Cos'è CRUD

Acronimo che descrive **le quattro operazioni fondamentali** che un'applicazione può fare sui dati:
Rappresenta il **fondamento della gestione dei dati** in qualsiasi applicazione.

OPERAZIONE	SIGNIFICATO	ESEMPIO SQL / ORM
Create	Creare un nuovo record	<code>INSERT INTO users ... o</code> <code>prisma.user.create({ data })</code>
Read	Leggere / recuperare dati	<code>SELECT * FROM users o</code> <code>prisma.user.findMany()</code>
Update	Modificare dati esistenti	<code>UPDATE users SET ... WHERE id=1 o</code> <code>prisma.user.update({ where: { id }, data })</code>
Delete	Eliminare dati	<code>DELETE FROM users WHERE id=1 o</code> <code>prisma.user.delete({ where: { id } })</code>

Cos'è REST

REST sta per **Representational State Transfer** ed è uno **stile architetturale per progettare API web**.

Principi principali:

1. **Client-Server** → Separazione tra frontend (client) e backend (server)
2. **Stateless** → Ogni richiesta contiene tutte le informazioni necessarie; il **server non mantiene lo stato della sessione**
3. **Cacheable** → Le risposte possono essere memorizzate per ottimizzare le performance
4. **Uniform Interface** → Interfaccia uniforme e prevedibile (**tipicamente HTTP verbs + risorse**)
5. **Layered System** → Possibile usare proxy, gateway, load balancer
6. **Code on Demand (opzionale)** → Il server può inviare codice eseguibile al client (JS, script)

CRUD → operazioni sui dati

REST → stile architetturale per esporre dati come API

Resource design → definizione chiara e coerente delle risorse e dei loro endpoint

HTTP verbs principali in REST

Nota: REST si basa su **risorse**, non su azioni. Ogni endpoint rappresenta una risorsa (utente, prodotto, ordine...).

Operazione CRUD	HTTP Verb	Endpoint REST
Create	POST	/users
Read (all)	GET	/users
Read (by id)	GET	/users/:id
Update	PUT / PATCH	/users/:id
Delete	DELETE	/users/:id

RESOURCE DESIGN (PROGETTAZIONE DELLE RISORSE)

Resource design significa **definire le risorse della API e come il client interagisce con esse.**

In REST tutto è una risorsa. Non si pensa a cosa fa il server, ma a cosa contiene il server.

RISORSE E NAMING

Una risorsa è un oggetto o un concetto del dominio.

- **USARE NOMI AL PLURALE:** rappresentano collezioni (es. /users, /products)
- **SOSTANTIVI, NON VERBI:** evitare endpoint come /getUser o /deleteuser
- **RAPPRESENTAZIONE:** la risorsa non è il database, ma una sua rappresentazione (solitamente JSON). Si possono omettere dati sensibili o aggiungere dati calcolati non presenti su DB.

URI (IDENTIFICATORI UNIVOCI) e GERARCHIA: ogni risorsa DEVE avere un indirizzo unico

RELAZIONI NIDIFICATE: per esprimere appartenenza, usare
/:parent/:id/:child

ESEMPIO: /users/45/orders

SINGLETON (singolo elemento) vs. **COLLECTION**

- /photos (Collezione): rappresenta intero set di foto
- /photos/12 (Singleton): rappresenta una specifica foto
- **ECCEZIONE: A VOLTE ESISTONO RISORSE "SINGLETON" naturali**, come /profile (profilo dell'utente attualmente loggato), dove l'ID è implicito nel token di autenticazione.

GESTIONE DELLA COMPLESSITÀ: FILTRI E PAGINAZIONE

Quando le collezioni diventano grandi, non si può restituire tutto.

Il Resource Design prevede l'uso delle **Query String**:

- **Filtro:** GET /cars?color=red
- **Paginazione:** GET /cars?page=2&limit=50
- **Ricerca:** GET /cars?q=ferrari

AZIONI NON CRUD STANDARD (CONTROLLER)

Cosa succede quando un'azione non è un semplice CRUD? Ad esempio, "inviare una mail" o "archiviare un documento".

- **Approccio "Risorsa Fittizia":** Invece di pensare all'azione come ad un comando, pensarla come la **creazione di una nuova risorsa che rappresenta quell'azione**.
 - **CONCETTO:** archiviare un documento diventa "**Creare un'archiviazione**"
 - **ENDPOINT:** Crea una **sottorisorsa fittizia**, es: `POST /documents/12/archive` OPPURE `POST /documents/12/archived-status`
 - **PERCHÉ USARLO:** metodo più fedele a filosofia REST. Si tratta azione come un evento o cambio di stato che può essere registrato.
 - **ESEMPIO MAIL:** `POST /users/45/notifications` (invece di `sendEmail`). Inviando un oggetto notifica alla collezione, si scatena l'invio della mail.

In sintesi: Cercare sempre di trasformare il "verbo" in un "sostantivo".
Se devi *approvare* una fattura, non chiamare `POST /invoices/1/approve`, ma prova a pensare a `POST /invoices/1/approvals`. Se proprio non ci riesci, usa il controller con POST come ultima spiaggia.

- **Approccio "Cambio di stato" (Field Update)**

Se l'azione **modifica** semplicemente **una proprietà della risorsa**, usare una **PATCH** o una **PUT**.

- **CONCETTO:** Archiviare un documento significa cambiare il suo campo status da "attivo" a "archiviato"
- **ENDPOINT:** `PATCH /documents/12` con body `{"status": "archived"}`
- **PERCHÉ USARLO:** pulito se l'azione è una semplice modifica di attributi
- **LIMITE:** non va bene se azione scatena logiche complesse (es. invio di notifiche a terzi, integrazioni esterne)

- **Approccio "controller" (verbo nell'URL)**

Approccio meno "puro" ma a volte il più utile per azioni che non si mappano bene su un oggetto.

- **CONCETTO:** si tratta l'azione come una **funzione specifica**
- **ENDPOINT:** `POST /documents/12/archive` o `POST /emails/send`
- **REGOLA:** **usare sempre POST**. Poiché non si sta manipolando direttamente una risorsa tramite CRUD, la POST indica che si sta "attivando" qualcosa che ha effetti sul server
- **ESEMPIO CLASSICO:** `POST /users/me/password-reset`

IDEMPOTENZA E SICUREZZA

- **Sicurezza (Safe):** Un metodo HTTP è **safe** se non modifica lo stato del server, è semanticamente di sola lettura, non produce effetti collaterali osservabili (**GET, HEAD**).
- **Idempotenza:** Un metodo è **idempotente** quando puoi chiamarlo **più volte con la stessa richiesta** e il risultato sul server è sempre lo stesso..
- **Perché è importante:** se una chiamata fallisce per un timeout di rete, il client può riprovare in sicurezza. Se il metodo è idempotente, non verranno creati duplicati.
- **ESEMPIO: PUT vs POST**
Se si invia 3 volte la stessa POST /orders, si creeranno tre ordini diversi. Se si invia 3 volte PUT /users/1 con gli stessi dati, l'utente 1 verrà aggiornato sempre allo stesso stato finale.
- **ESEMPIO DELETE:** la prima DELETE /photos/5 elimina la foto. Le chiamate successive troveranno la risorsa già eliminata, ma lo stato finale del server (la foto non esiste) rimane lo stesso.

Metodo	Azione	Sicuro?	Idempotente?	Successo
GET	Legge una risorsa	✔ Sì	✔ Sì	200 OK
POST	Crea una risorsa	✘ No	✘ No	201 Created
PUT	Sostituisce/Aggiorna	✘ No	✔ Sì	200 OK
PATCH	Aggiorna parzialmente	✘ No	✘ No*	200 OK
DELETE	Elimina la risorsa	✘ No	✔ Sì	204 No Content

STATUS CODE COERENTI

- 200 OK → richiesta completata
- 201 Created → risorsa creata
- 404 Not Found → risorsa non trovata
- 409 Conflict → conflitto (es. unique constraint)
- 500 Internal Server Error → errore server

HATEOAS (opzionale avanzato)

La risposta può contenere link verso risorse correlate

```
{
  "id": 123,
  "links": [
    { "rel": "self", "href": "/users/123" },
    { "rel": "orders", "href": "/users/123/orders" }
  ]
}
```



HTTP METHODS (Verbi HTTP)

Definiscono l'intenzione semantica dell'operazione su una risorsa.

Nel paradigma REST:

- Risorse identificate da URL
- Metodi HTTP **descrivono azione su risorsa**
- **Corpo della richiesta contiene lo stato** (quando necessario)

CRUD	HTTP	Semantica
Create	POST	Crea una nuova risorsa
Read	GET	Recupera una o più risorse
Update	PUT / PATCH	Aggiorna una risorsa
Delete	DELETE	Elimina una risorsa

GET – List & Detail

1. **LIST ENDPOINT**: recupera una lista di risorse

GET /users

- Safe (non modifica stato)
- Idempotente
- Supporta **query params** per **pagination**, **filtering** e **sorting**

```
GET /users?page=2&limit=10&role=admin
```

2. **DETAIL ENDPOINT**: recupera una singola risorsa

GET /users/{id}

RISPOSTE: 200 → Trovato

404 → non trovato

```
json
{
  "data": [...],
  "meta": {
    "page": 2,
    "total": 120
  }
}
```


POST - Create

Crea una nuova risorsa nella collezione

POST /users

BODY:

```
json
{
  "email": "test@example.com",
  "name": "Mario"
}
```

- NON idempotente
- Server genera identificatore
- Restituisce tipicamente:
 - 201 created
 - Header Location
 - Risorsa creata nel body

PUT vs PATCH

1. **PUT – Full Replacement**: sostituisce completamente la risorsa

PUT /users/42

- Idempotente
- Tutti i campi devono essere inviati
Se un campo manca,
viene sovrascritto (potenzialmente null)

```
{
  "email": "new@example.com",
  "name": "Mario",
  "isActive": true
}
```

USO CORRETTO: aggiornamento completo dello stato delle risorse o quando si lavora con DTO completi

2. **PATCH – Partial Update**: aggiornamento parziale.
Idempotente (se implementato correttamente)

Invia solo campi da modificare

PATCH /users/{id}

Più usato rispetto a PUT per update business-driven

```
{
  "email": "new@example.com"
}
```

DELETE – Soft vs Hard

Quando un client chiama DELETE /resources/:id, il server può gestire la rimozione in due modi diversi a seconda della criticità dei dati.

CARATTERISTICA

Azione DB

Recupero

Audit trail

Query

Performance

Quando usarlo

HARD DELETE

```
DELETE FROM table WHERE id = x
```

Impossibile (Perso fisicamente)

Perso

Standard

DB più snello

Log temporanei, dati di sessione scaduti, richieste GDPR "Diritto all'oblio"

SOFT DELETE

```
UPDATE table SET deleted_at = NOW()
```

Possibile (Restore del flag/null)

Mantenuto (Storico presente)

Richiede **WHERE deleted_at IS NULL**

DB cresce (richiede indici specifici)

Utenti, ordini, fatture, commenti (qualsiasi dato con valore storico o legale)

	Node.js	Express	Nest.js
COS'È	NON è un linguaggio di programmazione e NON è un framework . È l'ambiente di esecuzione (runtime) che permette a JavaScript di girare fuori dal browser, direttamente sul server	Framework standard per Node.js. "Minimalista" perché non impone una struttura rigida.	Framework progressivo costruito sopra Express (o Fastify). Introduce un'architettura rigorosa , simile ad Angular.
COSA FA	Fornisce le API per interagire con il filesystem, la rete e i database	Semplifica la gestione delle rotte, delle richieste HTTP e dei middleware . Trasforma le API grezze di Node.js in codice leggibile e veloce da scrivere	Obbliga lo sviluppatore ad usare pattern specifici come i Module, i Controller ed i Service . Supporta nativamente TypeScript e la Dependency Injection
PUNTO CHIAVE	Senza Node.js, Express e NestJS non potrebbero esistere. È la base.	Lascia totale libertà nell'organizzazione dei file. Rischioso per grandi progetti e team.	Pensato per applicazioni scalabili ed enterprise.

COME INTERAGISCONO TRA DI LORO – relazione gerarchica e incrementale

1. Node.js interpreta il codice JS/TS sul server
2. Express gira sopra Node.js per gestire il traffico web in modo più semplice
3. **NestJS utilizza un adapter HTTP. Di default Express, ma può utilizzare anche Fastify**

NestJs vs Express vs Fastify

Express è ottimo per partire velocemente. NestJS è preferibile quando vogliamo costruire applicazioni più strutturate, mantenibili e adatte a crescere nel tempo

Express

Pro

Enorme comunità
Tantissimi esempi e middleware
Molto conosciuto

Contro

Prestazioni inferiori a Fastify
Architettura lasciata allo sviluppatore

Fastify

Pro

Più veloce
Minore consumo di memoria
Validazione JSON Schema integrata
Ottimo per API ad alto traffico

Contro

Ecosistema più piccolo
Alcuni middleware Express non sono compatibili

NestJs + Express

1. Architettura chiara
Controller, Service, Module, DTO, Guard, Pipe, Middleware
2. Dependency Injection
I servizi vengono iniettati automaticamente
3. Codice più ordinato
Ogni cosa ha il suo posto
4. Più adatto a team
Tutti seguono la stessa struttura
5. TypeScript nativo
Tipi, classi, decoratori, interfacce
6. Funzionalità già integrate
Guard, Pipe, Interceptor, ValidationPipe, Module
7. Scalabilità
È più facile mantenere applicazioni grandi

ANATOMIA DI UN'APP NESTJS

main.ts → punto di **ingresso** app

Bootstrap applicazione: viene creato il server Express/Koa e avviate tutte configurazioni globali (pipes, interceptors, filtri errori, ecc.)

app.module.ts → modulo radice

Dice a Nest quali feature caricare e quali moduli importare. In NestJS tutto è un modulo: il framework usa decorator `@Module()` per definire cosa contiene e cosa esporta.

COSA FA:

1. IMPORTA ALTRI MODULI
2. REGISTRA CONTROLLER DI ALTO LIVELLO
3. REGISTRA PROVIDER GLOBALI

<feature>.module.ts → Modulo di feature

Ogni feature (es. "users") ha il suo modulo. Il modulo è una unità di organizzazione che raggruppa controller, servizi e altri provider.

```
async function bootstrap() {  
  const app = await NestFactory.create(AppModule);  
  await app.listen(3000);  
}  
bootstrap();
```

```
@Module({  
  imports: [UsersModule],  
  controllers: [AppController],  
  providers: [AppService],  
})  
export class AppModule {}
```

```
@Module({  
  controllers: [UsersController],  
  providers: [UsersService],  
})  
export class UsersModule {}
```

CONTROLLER - users.controller.ts

RESPONSABILITÀ PRINCIPALE:

- Esporre endpoint REST
- Ricevere richieste HTTP
- Mappe HTTP Methods a funzioni (@Get(), @Post(), @Put(), @Delete)
- Delegare logiche ai servizi
- Inviare le risposte

NON DEVE CONTENERE LOGICA BUSINESS: quella va nei servizi

Service - users.service.ts

RESPONSABILITÀ PRINCIPALE:

- Logica di business
- Interazioni con il database o ORM
- Astrazione delle regole di applicazione

Spesso i servizi:

- Vengono iniettati nei controller tramite Dependency Injection
- Contengono metodi per CRUD, validazione di logica business, transazioni, ecc.

```
@Controller('users')
export class UsersController {
  constructor(private readonly userService: UserService) {}

  @Get()
  findAll() {
    return this.userService.findAll();
  }
}
```

```
@Injectable()
export class UserService {
  findAll() {
    return []; // query al DB tramite Prisma qui
  }
}
```

DTO — Data Transfer Objects

DTO non sono obbligatori, ma sono raccomandati per:

- Definire forma dei dati accettati dalle API
- Validazione con class-validator
- Sicurezza (whitelist dei campi)
- Separare il contratto API dal modello dati

Servono soprattutto nel Controller Layer (Input validation)

```
export class CreateUserDto {  
  @IsEmail()  
  email: string;  
  
  @IsString()  
  name: string;  
}
```

Pipes

Vengono eseguiti **prima dei controller**, validano gli input

Utilizzano:

- validazione input (ValidationPipe)
- trasformazioni (ParseIntPipe)

```
@Get('/:id')  
findOne(@Param('id', ParseIntPipe) id: number) {}
```

Interceptors

Interceptors si frappongono **prima o dopo le chiamate controller**

Utilizzo:

- logging
- trasformazione response
- gestione timeouts
- wrappers di output (ad esempio aggiungere meta paginazione)

Exception Filters

- Gestiscono gli errori in modo uniforme:
- trasformano errori in response HTTP standard
- centralizzano comportamenti di errore

```
@Catch(PrismaClientKnownRequestError)  
export class PrismaFilter implements ExceptionFilter { ... }
```


Providers

Provider è un concetto ampio che include:

- Service
- Repository
- Helpers
- Utility classes
- Nest li gestisce tramite **Dependency Injection**.

🧠 Helpers/Guards/Middleware

- **Guards:** gestione autorizzazione/autenticazione
- **Middleware:** log, CORS, request parsing
- **Pipes:** validazione/transforms
- **Interceptors:** trasformazioni e cross-cutting

Si applicano a request prima o dopo i controller.

■ Tipica struttura file NestJS

```
cpp
src/
  main.ts           # bootstrap server
  app.module.ts     # root module
  users/
    users.module.ts # feature module
    users.controller.ts # routing & HTTP
    users.service.ts # business logic
  dto/
    create-user.dto.ts # validation schema
    update-user.dto.ts
  entities/ (optional)
    user.entity.ts
  repository/ (optional)
    users.repository.ts
  common/          # filters, guards, pipes, interceptors condivisi
```

📌 Ruolo a colpo d'occhio

File	Scopo
main.ts	Avvia l'app
app.module.ts	Punto di composizione dei moduli
*.module.ts	Raggruppa controller+services per feature
*.controller.ts	Espone endpoints
*.service.ts	Logica e orchestrazione
*.dto.ts	Contratti e validazione
Repository	Astrazione DB

import TypeScript vs imports di NestJS

controller con guard con express e TypeScript / JavaScript

Serve per usare una classe definita in un altro file

- TypeScript collega i file
- Tu crei l'oggetto con new
- Tu richiami i metodi
- Nessuna dependency injection

api-key.guard.ts

```
export class ApiKeyGuard {  
  canActivate(apiKey: string | undefined): boolean {  
    return apiKey === '123456';  
  }  
}
```

users.controller.ts

```
app.get('/users', (req: Request, res: Response) => {
```

```
  const guard = new ApiKeyGuard();  
  const apiKey = req.headers['x-api-key'];
```

```
  if (!guard.canActivate(apiKey as string)) {  
    return res.status(401).json({ message: 'API key non valida' });  
  }
```

```
  res.json([  
    { id: 1, name: 'Mario Rossi' },  
    { id: 2, name: 'Luca Bianchi' },  
  ]);  
});
```

Controller con guard e Providers di NestJS

- TypeScript importa la classe dal file
- Nest registra il provider
- Nest crea automaticamente l'istanza
- Nest richiama canActivate()
- Nessun new manuale

```
// api-key.guard.ts  
@Injectable()  
export class ApiKeyGuard implements CanActivate  
{  
  canActivate(): boolean {  
    return true;  
  }  
}
```

users.controller.ts

```
import { Controller, Get, UseGuards } from '@nestjs/common';  
import { ApiKeyGuard } from './api-key.guard';  
  
@Controller('users')
```

```
export class UsersController {  
  @Get()  
  @UseGuards(ApiKeyGuard)  
  findAll() {  
    return [];  
  }  
}
```

users.module.ts

```
import { Module } from '@nestjs/common';  
import { UsersController } from './users.controller';  
import { ApiKeyGuard } from './api-key.guard';
```

```
@Module({  
  controllers: [UsersController],  
  providers: [ApiKeyGuard],  
})  
export class UsersModule {}
```

creare un nuovo progetto NestJS

`nest new nomeprogetto`

Viene creata una cartella nomeprogetto

Viene generata la struttura base del progetto

Ti viene chiesto quale package manager usare:

npm

yarn

pnpm

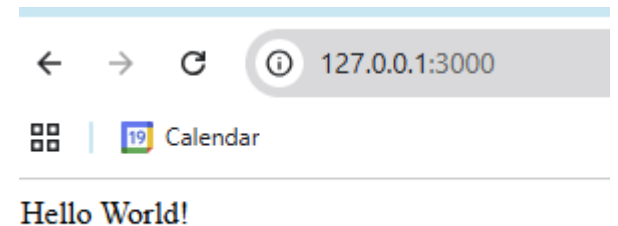
Vengono installate automaticamente le dipendenze

per eseguire: `npm run start`

oppure `npm run start:dev`

nomeprogetto/

```
|
| └─ src/
|   └─ app.controller.ts
|   └─ app.service.ts
|   └─ app.module.ts
|   └─ main.ts
|
| └─ test/
| └─ package.json
| └─ tsconfig.json
└─ nest-cli.json
```



lint e format

npm run lint --fix
(corregge gli errori)

```
PS C:\Software\nest.js\test2-mysql> npm run lint

> test2-mysql@0.0.1 lint
> eslint "{src,apps,libs,test}/**/*.ts" --fix

C:\Software\nest.js\test2-mysql\src\main.ts
  8:1  warning  Promises must be awaited, end with a call to .catch() or be explicitly marked as ignored with the `void` operator  @typescript-eslint/no-floating-promises

✖ 1 problem (0 errors, 1 warning)
```

npm run format

function test(){console.log("ciao")}

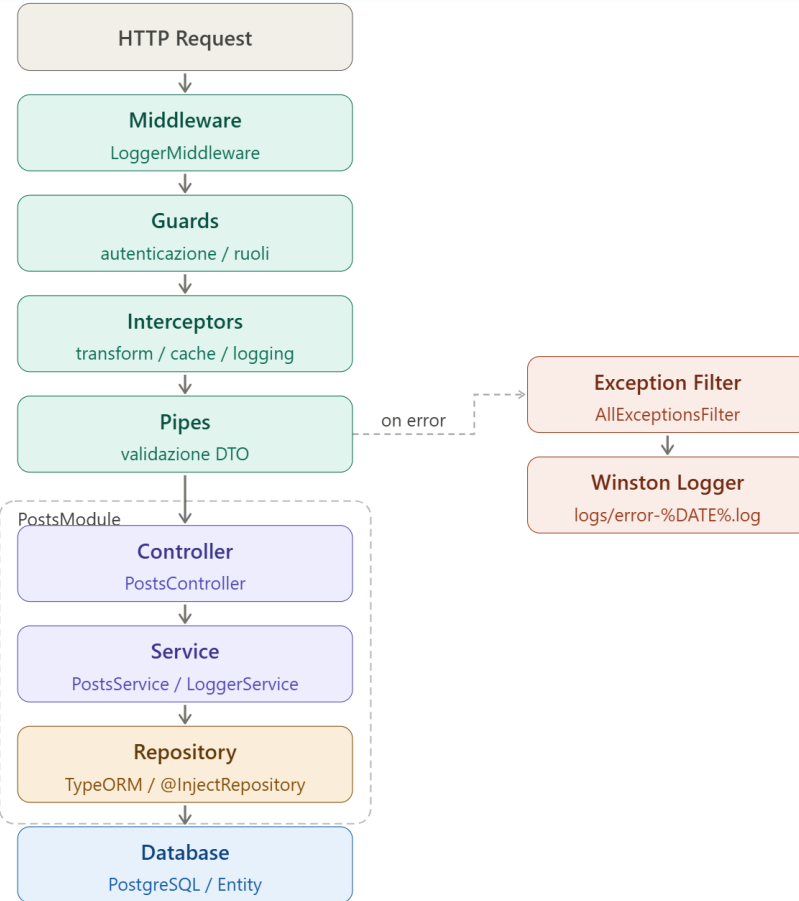
function test() {
 console.log('ciao');
}

```
mysql > src > TS app.controller.ts > test
import { AppService } from './app.service';

@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}

function test() {
  console.log('ciao');
}
```



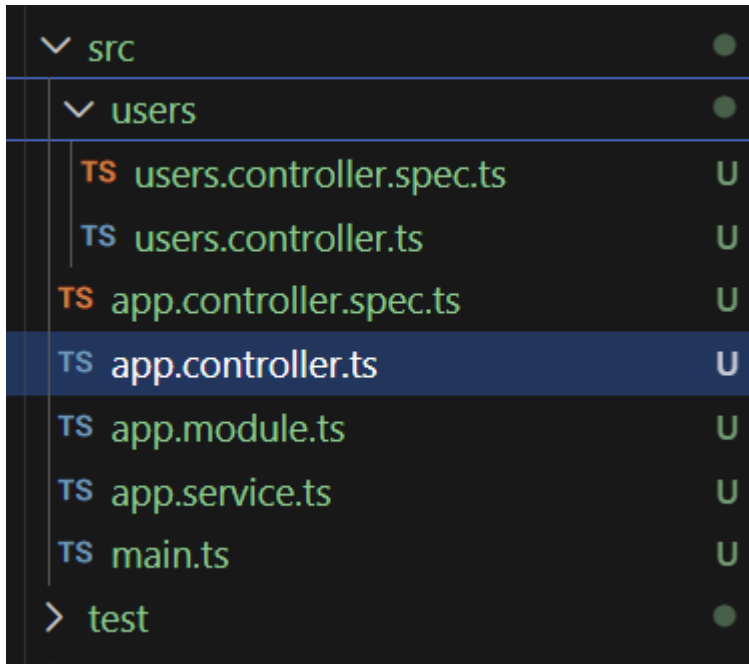
■ Pipeline ■ App Layer ■ Data Layer ■ Error Handling

† clicca per approf

controllers - GET

Un **Controller** è il componente che:

- 👉 Riceve le richieste HTTP
- 👉 Decide cosa fare
- 👉 Restituisce una risposta al client



nest generate controller users

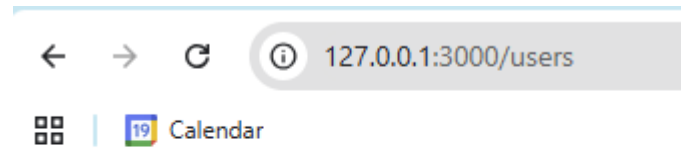
oppure

nest g co users

gestisce le rotte /users

```
import { Controller, Get } from '@nestjs/common';  
import { get } from 'http';
```

```
@Controller('users')  
export class UsersController {  
  @Get()  
  getUsers() {  
    return 'This action returns all users';  
  }  
}
```



This action returns all users

@Controller() e @Get()

Il decoratore **@Controller()** definisce il prefisso comune delle rotte gestite dal controller. In questo esempio tutte le rotte inizieranno con /users.

Il decoratore **@Get()** indica che il metodo deve rispondere alle richieste HTTP di tipo **GET**. Poiché non viene specificato alcun percorso aggiuntivo, il metodo sarà associato direttamente alla rotta base del controller.

```
import { Controller, Get } from '@nestjs/common';
```

```
@Controller('users')
export class UsersController {
```

```
  @Get()
  findAll() {
    return 'Lista utenti';
  }
```

```
}
```

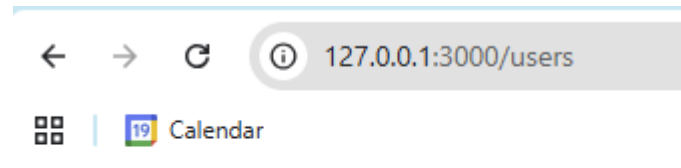
Rotta generata

GET /users

URL completa in locale

<http://localhost:3000/users>

Questo tipo di metodo viene normalmente usato per recuperare l'elenco delle risorse, ad esempio tutti gli utenti presenti nel database. In un'applicazione reale potrebbe restituire un array JSON



This action returns all users

Reponse

In **NestJS**, se restituisci un oggetto da un metodo di un controller:

```
@Get()
getHello(): object {
  return { message: 'Benvenuto in NestJS!' };
}
```

NestJS lo converte automaticamente in **JSON** in quanto usa l'adapter HTTP standard (Express o Fastify).

La risposta effettiva sarà:

```
HTTP/1.1 200 OK
Content-Type: application/json;
charset=utf-8
```

Se restituisco una stringa?

```
@Get()
getHello(): string {
  return 'Benvenuto in NestJS!';
}
```

Risposta:

```
Content-Type: text/html;
charset=utf-8
Benvenuto in NestJS!
```

Quindi:

Object → JSON

Array → JSON

String → testo

Buffer/Stream → dati binari

Se voglio forzare il Content-Type o la Res:

```
@Get()
getHello(@Res() res: Response) {
  res.status(200).json({
    message: 'Benvenuto in NestJS!'
  });
}
```

```
@Get()
@Header('Content-Type',
'application/json')
getHello() {
  return { message: 'Benvenuto in NestJS!' };
}
```

oppure ..

```
@Header('Content-Type', 'text/plain')
```


@Post()

@Post() indica che un metodo risponde a una richiesta HTTP di tipo **POST**.

Di solito si usa per **creare una nuova risorsa**.

```
import { Controller, Post } from '@nestjs/common';
```

```
@Controller('users')
```

```
export class UsersController {
```

```
  @Post()
```

```
  create() {
```

```
    return 'Utente creato';
```

```
  }
```

```
}
```

La rotta completa sarà:

POST /users

Esempio chiamata:

http://localhost:3000/users

Risposta:

Utente creato

Uso tipico:

Creare un nuovo utente

Creare un nuovo prodotto

Salvare un nuovo ordine

← → ↻ ⓘ 127.0.0.1:3000/users

🗄️ | 📅 19 Calendar

This action returns all users

@Post() con @Body()

@Body() permette di leggere i dati inviati nel corpo della richiesta.

```
import { Controller, Post, Body } from
  '@nestjs/common';
```

@Controller('users')

```
export class UsersController {
```

```
  @Post()
```

```
  create(@Body() body: any) {
```

```
    return {
```

```
      message: 'Utente creato',
```

```
      data: body
```

```
    };
```

```
  }
```

```
}
```

Body della richiesta:

```
{
  "name": "Mario Rossi",
  "email": "mario@test.it"
}
```

Risposta:

```
{
  "message": "Utente creato",
  "data": {
    "name": "Mario Rossi",
    "email": "mario@test.it"
  }
}
```

@Post() gestisce la richiesta

@Body() legge i dati inviati dal client

@Get(':id') con @Param() – Rotte parametriche

@Param() legge un valore presente nella URL.

```
import { Controller, Get, Param } from  
  '@nestjs/common';
```

```
@Controller('users')  
export class UsersController {
```

```
  @Get(':id')  
  findOne(@Param('id') id: string) {  
    return `Utente con id ${id}`;  
  }  
  
}
```

Rotta:

GET /users/5

Risposta:

Utente con id 5

In pratica:

:id crea un parametro dinamico

@Param('id') legge quel parametro

@Query()

@Query() legge i parametri presenti dopo il simbolo ?.

```
import { Controller, Get, Query } from  
  '@nestjs/common';
```

```
@Controller('users')  
export class UsersController {
```

```
  @Get('search')  
  search(@Query('name') name: string) {  
    return `Cerco utente: ${name}`;  
  }
```

```
}
```

Rotta:

GET /users/search?name=Mario

Risposta:

Cerco utente: Mario

Uso tipico:

Filtri

Ricerca

Paginazione

Ordinamento

@Put()

@Put() gestisce richieste HTTP di tipo **PUT**.

Si usa normalmente per **aggiornare completamente** una risorsa esistente.

```
import { Controller, Put, Param, Body } from  
'@nestjs/common';
```

```
@Controller('users')  
export class UsersController {
```

```
  @Put(':id')  
  update(  
    @Param('id') id: string,  
    @Body() body: any,  
  ) {  
    return {  
      id,  
      data: body,  
    };  
  }  
}
```

Rotta:

PUT /users/5

Body:

```
{  
  "name": "Mario Rossi",  
  "email": "mario@test.it"  
}
```

Uso tipico:

Aggiornare un utente

Aggiornare un prodotto

Aggiornare una categoria

@Patch()

@Patch() viene utilizzato per aggiornamenti **parziali**.

```
import { Controller, Patch, Param, Body } from
  '@nestjs/common';
```

```
@Controller('users')
export class UsersController {
```

```
  @Patch('/:id')
  updateEmail(
    @Param('id') id: string,
    @Body() body: any,
  ) {
    return {
      id,
      email: body.email,
    };
  }
}
```

Rotta:

PATCH /users/5

Body:

```
{
  "email": "nuova@mail.com"
}
```

Differenza:

PUT = aggiorna tutto

PATCH = aggiorna solo alcuni campi

@Delete()

@Delete() gestisce richieste HTTP di eliminazione.

```
import { Controller, Delete, Param } from  
  '@nestjs/common';
```

```
@Controller('users')  
export class UsersController {
```

```
  @Delete(':id')  
  remove(@Param('id') id: string) {  
    return `Utente ${id} eliminato`;  
  }  
  
}
```

Rotta:

DELETE /users/5

Risposta:

Utente 5 eliminato

@Headers()

Gli **header HTTP** sono informazioni aggiuntive nella risposta.

Esempi: cache, autenticazione, tipo di contenuto

@Headers() legge gli header HTTP inviati dal client.

```
import { Controller, Get, Headers } from  
  '@nestjs/common';
```

```
@Controller('users')  
export class UsersController {
```

```
  @Get('info')  
  info(  
    @Headers('user-agent') userAgent: string,  
  ) {  
    return userAgent;  
  }  
}
```

Richiesta:

GET /users/info

User-Agent: Chrome

Risposta:

Chrome

Uso tipico:

Leggere token custom

Leggere informazioni del browser

Leggere header API

@Req()

@Req() permette di accedere all'intera richiesta HTTP.

```
import { Controller, Get, Req } from
  '@nestjs/common';
import { Request } from 'express';
```

```
@Controller('users')
export class UsersController {
```

```
  @Get()
  findAll(@Req() req: Request) {
    return req.url;
  }
```

```
}
```

Permette di accedere a:

```
req.body
req.params
req.query
req.headers
req.ip
```

⚠ In NestJS è preferibile usare @Body(), @Param() e @Query() quando possibile.

@Res()

@Res() permette di gestire direttamente la risposta HTTP.

```
import { Controller, Get, Res } from
  '@nestjs/common';
import { Response } from 'express';
```

```
@Controller('users')
export class UsersController {
```

```
  @Get()
  findAll(@Res() res: Response) {
    res.status(200).json({
      message: 'OK',
    });
  }
}
```

Permette di controllare:

Status code

Header

Cookie

JSON restituito

@Redirect()

Effettua un redirect verso un'altra URL.

```
import { Controller, Get, Redirect } from  
  '@nestjs/common';
```

```
@Controller('users')  
export class UsersController {
```

```
  @Get('site')  
  @Redirect('https://nestjs.com', 302)  
  site() {}
```

```
}
```

Rotta:

GET /users/site

Effetto:

Redirect verso <https://nestjs.com>

(di solito 301 o 302)

@HttpCode()

Permette di modificare il codice HTTP restituito (normalmente impostati default).

```
import {
  Controller,
  Post,
  HttpStatusCode,
} from '@nestjs/common';

@Controller('users')
export class UsersController {

  @Post()
  @HttpCode(201)
  create() {
    return {
      message: 'Creato',
    };
  }
}
```

Risposta:
HTTP/1.1 201 Created

Metodo HTTP	Operazione	Codice	Significato	Quando usarlo
POST	Crea una nuova risorsa	200	OK (richiesta riuscita)	Se la risorsa è stata creata e restituita
		201	Created (risorsa creata)	Quando la risorsa è stata creata con successo
		400	Bad Request	Se i dati inviati sono non validi
		500	Errore server	Se c'è stato un errore lato server
GET	Legge una o più risorse	200	OK (richiesta riuscita)	Quando la risorsa è stata trovata e restituita
		204	No Content (ok ma senza risposta)	Quando la richiesta è ok ma non c'è contenuto da restituire
		400	Bad Request	Se la richiesta non è valida
		404	Not Found	Se la risorsa non esiste
		500	Errore server	Se c'è stato un errore lato server
PUT	Sostituisce completamente una risorsa	200	OK (richiesta riuscita)	Quando la risorsa è stata aggiornata e restituita
		204	No Content (ok ma senza risposta)	Quando la risorsa è aggiornata ma non si restituisce nulla
		400	Bad Request	Se i dati inviati sono non validi
		404	Not Found	Se la risorsa non esiste
		500	Errore server	Se c'è stato un errore lato server
PATCH	Aggiorna parzialmente una risorsa	200	OK (richiesta riuscita)	Quando la risorsa è stata aggiornata e restituita
		204	No Content (ok ma senza risposta)	Quando la risorsa è aggiornata ma non si restituisce nulla
		400	Bad Request	Se i dati inviati sono non validi
		404	Not Found	Se la risorsa non esiste
		500	Errore server	Se c'è stato un errore lato server
DELETE	Elimina una risorsa	200	OK (richiesta riuscita)	Quando la risorsa è stata eliminata e si restituisce qualcosa
		204	No Content (ok ma senza risposta)	Quando la risorsa è eliminata e non si restituisce nulla
		400	Bad Request	Se la richiesta non è valida
		404	Not Found	Se la risorsa non esiste
		500	Errore server	Se c'è stato un errore lato server
ALTRI CODICI UTILI				
401	Unauthorized		Autenticazione mancante o non valida	
403	Forbidden		Non hai i permessi per accedere a questa risorsa	
409	Conflict		Conflitto con lo stato attuale della risorsa (es. duplicato)	
422	Unprocessable Entity		Dati corretti ma logicamente non validi	
503	Service Unavailable		Servizio temporaneamente non disponibile	

@HostParam()

Legge parametri presenti nel sottodominio.

```
import {
  Controller,
  Get,
  HostParam,
} from '@nestjs/common';

@Controller({
  host: ':account.example.com',
})
export class UsersController {

  @Get()
  index(
    @HostParam('account') account: string,
  ) {
    return account;
  }
}
```

URL:

mario.example.com

Risposta:

mario

Decoratori principali

@Body() body JSON

```
@Post()  
  create(@Body() dto: CreateUserDto) {}
```

@Param() parametri URL

```
@Get(':id')  
  findOne(@Param('id') id: string) {}
```

@Query() query string

```
@Get()  
  find(@Query('search') search: string) {}
```

@Headers() header

```
@Get()  
  find(@Headers('authorization') auth: string) {}
```

@Req() @Res() request completa

```
@Get()  
  find(@Req() req, @Res() res) {}
```

Decorator Scopo

@Module() organizza moduli

```
– @Module({  
  imports: [], controllers: [], providers: [], exports: [], })  
  export class UsersModule {}
```

@Controller() crea controller

```
– @Controller('users')  
  export class UsersController {}
```

@Get() @Post() @Put() @Delete() definisce route HTTP

```
– @Get()  
  findAll() {}
```

@Injectable() abilita dependency injection

```
– @Injectable()  
  export class UsersService {}
```

@UseGuards() protegge route

```
– @UseGuards(ApiKeyGuard)  
  @Get()  
  findAll() {}
```

Providers

Un **provider** è semplicemente una **classe** che può essere “**iniettata**” in un’altra classe.

👉 In pratica:

è un oggetto che contiene **logica riutilizzabile**

NestJS lo gestisce automaticamente

può essere condiviso tra più componenti

chiariamo:

Provider

qualsiasi cosa che Nest può iniettare

👉 **Service**

una classe con logica

il tipo più comune di provider

Service

- @Injectable()
export class UsersService {}

Guard

- @Injectable()
export class AuthGuard implements CanActivate {}

Pipe

- @Injectable()
export class ValidationPipe implements PipeTransform {}

Interceptor

- @Injectable()
export class LoggingInterceptor implements NestInterceptor {}

Exception Filter

- @Injectable()
export class HttpExceptionHandler implements ExceptionFilter {}

Strategy (Passport/JWT)

- @Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {}

Il Service

In **NestJS**, un **service** è una classe dove metti la **logica dell'applicazione**.

Perché usare un Service?

Generato con:

```
nest g service users
```

Viene creato:

```
src/users/users.service.ts
```

```
import { Injectable } from '@nestjs/common';
```

```
@Injectable()
```

```
export class UsersService {
```

```
  findAll() {  
    return [  
      { id: 1, name: 'Mario' },  
      { id: 2, name: 'Luigi' },  
    ];  
  }  
}
```

Il decorator `@Injectable()`

Un Service deve essere decorato con:

`@Injectable()`

```
import { Injectable } from '@nestjs/common';
```

```
@Injectable()
```

```
export class UsersService {}
```

Questo dice a NestJS:

- Questa classe può essere iniettata in altri componenti.

Iniettare un Service nel Controller

Controller:

```
import { Controller, Get } from '@nestjs/common';  
import { UsersService } from '../users.service';
```

```
@Controller('users')
```

```
export class UsersController {
```

```
  constructor(  
    private readonly usersService: UsersService,  
  ) {}
```

```
  @Get()
```

```
  findAll() {
```

```
    return this.usersService.findAll();
```

```
  }
```

```
}
```

Qui NestJS crea automaticamente l'istanza del service.

Non serve fare new UsersService()

Dependency injection

La Dependency Injection è un pattern in cui un oggetto riceve automaticamente le proprie dipendenze invece di crearle manualmente.

È un modo per **non creare manualmente gli oggetti**, ma farli fornire dal framework

è un singleton

➡ **NON viene creato ogni volta**

👉 è una ISTANZA condivisa

Senza Dependency Injection

Il controller crea direttamente il service.

```
class UserService {  
  getUsers() {  
    return ['Mario', 'Luca'];  
  }  
}  
  
class UsersController {  
  private userService = new UserService();  
  
  findAll() {  
    return this.userService.getUsers();  
  }  
}
```

Con Dependency Injection (NestJS)

Nest crea e inietta automaticamente il service.

```
@Injectable()  
export class UserService {  
  getUsers() {  
    return ['Mario', 'Luca'];  
  }  
}  
  
@Controller('users')  
export class UsersController {  
  
  constructor(  
    private readonly userService: UserService,  
  ) {}  
  
  @Get()  
  findAll() {  
    return this.userService.getUsers();  
  }  
}
```

vantaggi:

- codice più pulito
- componenti riutilizzabili
- testing più semplice
- basso accoppiamento
- gestione automatica dipendenze

Dependency Injection NestJs vs Ts/Js

- In JavaScript classico:
 - `const service = new UsersService();`
- In NestJS:
 - `constructor(
 private readonly usersService: UsersService,
) {}`
- NestJS:
 - 1. crea il service
 - 2. lo memorizza
 - 3. lo inietta dove serve

Service e Controller per CRUD con search

user.service.ts

```
@Injectable()
export class UsersService {

  findAll() {}

  findOne(id: number) {}

  create(data: CreateUserDto) {}

  update(
    id: number,
    data: UpdateUserDto,
  ) {}

  remove(id: number) {}

}

search(q: string) {
}
```

user.controller.ts

```
@Controller('users')
export class UsersController {
  constructor(
    private readonly usersService: UsersService,
  ) {}

  @Get()
  findAll() {
    return this.usersService.findAll();
  }

  @Get('search')
  search(@Query('q') q: string) {
    return this.usersService.search(q);
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.usersService.findOne(+id);
  }
}
```

```
@Post()
create(@Body() data: CreateUserDto) {
  return this.usersService.create(data);
}

@Patch(':id')
update(
  @Param('id') id: string,
  @Body() data: UpdateUserDto,
) {
  return this.usersService.update(+id, data);
}

@Delete(':id')
remove(@Param('id') id: string) {
  return this.usersService.remove(+id);
}
}
```

Esempio di Controller + Service

- nest g controller categories
- nest g service categories

- **service**

- import { Injectable } from '@nestjs/common';
-
- @Injectable()
- export class CategoriesService {
- private categories: {
- id: string;
- name: string;
- }[] = [];
- getAllCategories() {
- return this.categories;
- }
- createCategory(id: string, name: string) {
- const newCategory = { id, name };
- this.categories.push(newCategory);
- }
- }

- **controller**

- import { Controller, Query, Get, Post, Body } from '@nestjs/common';
- import { CategoriesService } from './categories.service';
-
- @Controller('categories')
- export class CategoriesController {
- constructor(private readonly categoriesService: CategoriesService) {}
- @Get()
- getAllCategories() {
- return this.categoriesService.getAllCategories();
- }
- @Post()
- createCategory(@Body('id') id: string, @Body('name') name: string) {
- this.categoriesService.createCategory(id, name);
- }
- }
-

Service - Singleton

i service sono singleton e **possono essere importati in più moduli ad esempio creando locations**

nest g resource locations

locations.module.ts deve esportare il locations.service

customers.module deve importare il locations.module

quindi

Per usare un service di un altro modulo:

1. Il modulo ORIGINALE deve esportare il service

2. Il modulo che lo usa deve importare quel modulo

```
{
  "message": "This action returns all customers",
  "locations": [
    {
      "id": 1,
      "name": "Warehouse A",
      "address": "123 Main St"
    },
    {
      "id": 2,
      "name": "Warehouse B",
      "address": "456 Elm St"
    }
  ]
}
```

dentro locations.module

```
@Module({
  controllers: [LocationsController],
  providers: [LocationsService],
  exports: [LocationsService],
})
export class LocationsModule {}
```

dentro customers.module

```
import { LocationsModule } from 'src/locations/locations.module';
```

...

```
@Module({
  imports: [LocationsModule],
  controllers: [CustomersController],
  providers: [CustomersService],
})
```

...

Module

I **Module** sono il meccanismo con cui NestJS organizza l'applicazione.

Ogni funzionalità viene raggruppata in un modulo.

Un **module** è un contenitore organizzativo

Cosa contiene un module?

1. controllers

controllers: [CategoriesController]

2. providers

providers: [CategoriesService]

3. imports (opzionale)

imports: [OtherModule]

4. exports (opzionale)

exports: [CategoriesService]

`nest g module customers`

lo importa in app.module

i successivi

`nest g controller customers`

`nest g service customers`

li inserirà direttamente nel module

oppure `nest g resource customers`

crea module, controller, service, provider, dto customers/

```
├── dto/
│   ├── create-customer.dto.ts
│   └── update-customer.dto.ts
├── entities/
│   └── customer.entity.ts
├── customers.controller.ts
├── customers.service.ts
└── customers.module.ts
```

Perché esistono i Module?

Senza moduli avremmo tutto dentro un unico file.

- Controller
- Service
- Repository
- Entity
- DTO

Dopo pochi mesi il progetto diventerebbe ingestibile.

NestJS utilizza i Module per:

- Organizzare il codice
- Separare le responsabilità
- Favorire il riutilizzo
- Gestire le dipendenze

Il decorator @Module()

Tutti i moduli utilizzano il decorator:
@Module()

Esempio:

```
import { Module } from '@nestjs/common';
```

```
@Module({  
  controllers: [],  
  providers: [],  
  imports: [],  
  exports: [],  
})  
export class UsersModule {}
```

Le sezioni di @Module()

```
@Module({  
  imports: [],  
  controllers: [],  
  providers: [],  
  exports: [],  
})
```

Scopo

- imports Importa altri moduli
- controllers Elenco dei controller
- providers Elenco dei service/provider
- exports Rende disponibili **provider** ad altri moduli

Controllers nei Module

- I controller del modulo vengono dichiarati qui:
- `@Module({
 controllers: [UsersController],
})`
- `export class UsersModule {}`
- NestJS sa che:
- UsersController
 appartiene a UsersModule

Registrazione del Service nel Module

Per essere utilizzato, il service deve essere registrato nel modulo.

```
@Module({  
  controllers: [UsersController],  
  providers: [UserService],  
})  
export class UsersModule {}
```

La sezione:

- providers: [UserService]

dice a NestJS:

- Questo service può essere iniettato.

In questo modo NestJS può eseguire la Dependency Injection (es nel Controller).

```
constructor(  
  private userService: UserService  
)
```

AppModule

- È il modulo principale dell'applicazione.
- ```
import { Module } from '@nestjs/common';
import { UsersModule } from './users/users.module';

@Module({
 imports: [UsersModule],
})

export class AppModule {}
```
- NestJS parte sempre da:
  - AppModule che rappresenta la root dell'applicazione.

# Condivisione di un Service con exports

Di default un Service è visibile solo all'interno del proprio modulo.

```
@Module({
 providers: [UsersService],
})
export class UsersModule {}
```

Il service NON è accessibile da altri moduli.

Per rendere disponibile un service ad altri moduli:

```
@Module({
 providers: [UsersService],
 exports: [UsersService],
})
export class UsersModule {}
```

Esempio

```
UsersModule:
@Module({
 providers: [UsersService],
 exports: [UsersService],
})
export class UsersModule {}
```

```
AuthModule:
@Module({
 imports: [UsersModule],
})
export class AuthModule {}
```

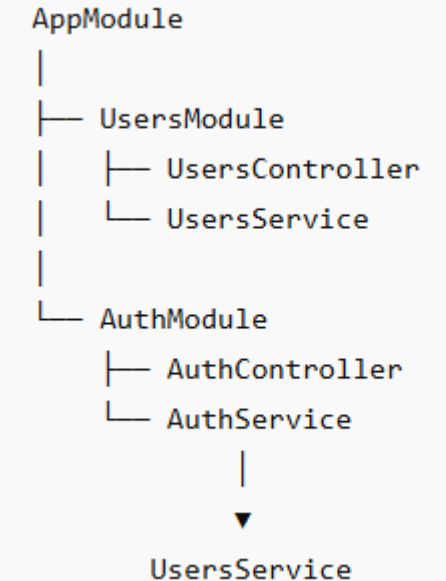
```
AuthService:
constructor(
 private readonly usersService: UsersService,
) {}
```

```
AuthController
@Controller('auth')
export class AuthController {

 constructor(
 private readonly authService: AuthService,
) {}
}
```

Funziona perché:

UsersModule esporta UsersService  
AuthModule importa UsersModule



# Cos'è una Origin

Una Origin è composta da:

Protocollo

Host

Porta

Esempi:

| URL                    | Origin                   |
|------------------------|--------------------------|
| http://localhost:3000  | localhost:3000           |
| http://localhost:4200  | localhost:4200           |
| https://localhost:3000 | https + localhost + 3000 |
| https://api.miosito.it | api.miosito.it           |

# CORS

**CORS** significa:

Cross-Origin Resource Sharing

È un meccanismo di sicurezza implementato dai browser.

Serve a controllare quali siti web possono effettuare richieste verso la nostra API.

Il problema

Supponiamo di avere:

Frontend:

`http://localhost:4200`

Backend NestJS:

`http://localhost:3000`

Dal browser Angular eseguiamo:

```
fetch('http://localhost:3000/users')
```

Il browser vede che:  $4200 \neq 3000$   
e considera la richiesta: Cross-Origin

Se cambia anche un solo elemento da :

Protocollo

Host

Porta

si parla di:

Cross-Origin Request

# Errore tipico

Se CORS non è configurato:

Access to fetch at

'http://localhost:3000/users'

from origin

'http://localhost:4200'

has been blocked by CORS policy

Errore molto comune quando:

Angular + NestJS

React + NestJS

Vue + NestJS

girano su porte diverse



# CORS

## Abilitare CORS in NestJS

### main.ts

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
```

```
async function bootstrap() {
```

```
 const app = await NestFactory.create(AppModule);
```

```
 app.enableCors();
```

```
 await app.listen(3000);
}
```

```
bootstrap();
```

Adesso il backend accetterà richieste provenienti da altri domini.

CORS personalizzato - limitare gli accessi.

```
app.enableCors({
 origin: 'http://localhost:4200',
});
```

Solo Angular potrà chiamare le API.

```
app.enableCors({
 origin: [
 'http://localhost:4200',
 'http://localhost:5173',
],
});
```

Permette:

- Angular
- React/Vite

### Tutti i domini

```
app.enableCors({
 origin: '*',
});
```

Equivale a:

Access-Control-Allow-Origin: \*

- ⚠ Utile in sviluppo.
- ⚠ Sconsigliato in produzione.

ATTENZIONE: ABILITARE ORIGIN \* SIGNIFICA NON CONTROLLARE PIU I CORS IN QUANTO TUTTI SONO ABILITATI

# CORS: Metodi HTTP consentiti

Consente solo questi metodi.

```
app.enableCors({
 origin: 'http://localhost:4200',
 methods: [
 'GET',
 'POST',
 'PUT',
 'PATCH',
 'DELETE',
],
});
```

Header consentiti

```
– app.enableCors({
 origin: 'http://localhost:4200',
 allowedHeaders: [
 'Content-Type',
 'Authorization',
],
});
```

Molto utile quando utilizziamo JWT.

Esempio:

```
– Authorization: Bearer eyJ...
```

# Esempio NestJs e React

## Backend NestJS

### main.ts

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
 const app = await NestFactory.create(AppModule);

 app.enableCors({
 origin: 'http://localhost:5173',
 methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
 allowedHeaders: ['Content-Type', 'Authorization'],
 });

 await app.listen(3000);
}

bootstrap();
```

#### Regola da ricordare:

React gira su 5173  
NestJS gira su 3000

↓

`serve app.enableCors()`

## Frontend React

```
const [users, setUsers] = useState<User[]>([]);
```

```
useEffect(() => {
 fetch('http://localhost:3000/users')
 .then((response) => response.json())
 .then((data) => setUsers(data));
}, []);
```

```
return (
 <div>
 <h1>Lista utenti</h1>

 {users.map((user) => (
 <li key={user.id}>
 {user.name}

))}

 </div>
);
```

```
export default App;
```

# Dto

**DTO** significa: *Data Transfer Object*

Serve a:

definire **forma dei dati**

dare **tipi (TypeScript)**

aiutare il codice a essere più sicuro

```
nest g class users/dto/user.dto
```

```
nest g class users/dto/create-user.dto
```

```
nest g class users/dto/update-user.dto
```

UserDto (response) Get

```
export class UserDto {
 id: number;
 name: string;
 email: string;
}
```

CreateUserDto (per POST)

```
nest g class users/dto/create-user.dto
```

```
users/
└─ dto/
 └─ create-user.dto.ts
```

```
export class CreateUserDto {
```

```
 name: string;
```

```
 email: string;
```

```
}
```

UpdateUserDto (per modificare)

```
export class UpdateUserDto {
 name?: string;
 email?: string;
}
```

? = facoltativo

# Dto

**DTO** significa: *Data Transfer Object*

Serve a:

definire **forma dei dati**

dare **tipi (TypeScript)**

aiutare il codice a essere più sicuro

In NestJS viene usata soprattutto per controllare i dati ricevuti nel `@Body()`.

`src/users/dto/create-user.dto.ts`

```
export class CreateUserDto {
 name: string;
 email: string;
 password: string;
}
```

`src/users/user.controller.ts`

```
@Post()
create(@Body() dto: CreateUserDto) {
 return dto;
}
```

	Decorator	Description		Decorator	Description
COMMON VALIDATION DECORATOR	@IsDefined(value: any)	Checks if value is defined (!== undefined, !== null).	STRING VALIDATION	@IsString()	Checks if the string is a string.
	@IsOptional()	Ignores all validators if value is empty.		@Length(min: number, max?: number)	Checks if the string's length falls in range.
	@Equals(comparison: any)	Checks if value equals comparison.		@MinLength(min: number)	Checks if the string's length is not less than given number.
	@NotEquals(comparison: any)	Checks if value not equals comparison.		@MaxLength(max: number)	Checks if the string's length is not more than given number.
	@IsEmpty()	Checks if given value is empty.		@Matches(pattern: RegExp)	Checks if string matches the pattern.
	@IsNotEmpty()	Checks if given value is not empty.		@IsEmail()	Checks if the string is an email.
	@IsIn(values: any[])	Checks if value is in a array of allowed values.		@IsURL()	Checks if the string is an URL.
	@IsNotIn(values: any[])	Checks if value is not in a array of disallowed values.		@IsFQDN()	Checks if the string is a fully qualified domain name.
NUMBER VALIDATION	@IsNumber()	Checks if the value is a number.	ARRAY VALIDATION	@isArray()	Checks if the value is an array
	@IsInt()	Checks if the value is an integer.		@ArrayContains(values: any[])	Checks if array contains all values from the given array
	@Min(min: number)	Checks if the value is greater than or equal to given number.		@ArrayNotContains(values: any[])	Checks if array does not contain any of the given values
	@Max(max: number)	Checks if the value is less than or equal to given number.		@ArrayNotEmpty()	Checks if given array is not empty
	@IsPositive()	Checks if the value is a positive number.		@ArrayMinSize(min: number)	Checks if array's length is greater than or equal to specified number
	@IsNegative()	Checks if the value is a negative number.		@ArrayMaxSize(max: number)	Checks if array's length is less or equal to specified number
DATE VALIDATION	@IsDate()	Checks if the value is a Date.		@ArrayUnique()	Checks if all array's values are unique
	@MinDate(date: Date)	Checks if the date is greater than given date.			
	@MaxDate(date: Date)	Checks if the date is less than given date.			

# Esempio dto nel controller

## create-user.dto.ts

```
export class CreateUserDto {
 name: string;
 email: string;
 password: string;
}
```

## user.dto.ts

```
export class UserDto {
 id: number;
 name: string;
 email: string;
}
```

## update-user.dto.ts

```
export class UpdateUserDto {
 name?: string;
 email?: string;
 password?: string;
}
```

```
src/
├── users/
│ ├── dto/
│ │ ├── create-user.dto.ts
│ │ ├── update-user.dto.ts
│ │ └── user.dto.ts
│ ├── user.controller.ts
│ ├── user.service.ts
│ └── user.module.ts
```

## user.controller.ts

```
import { Get, Post, Body, Controller, Put, Param } from '@nestjs/common';
import { CreateUserDto } from '../dto/create-user.dto/create-user.dto';
import { UserDto } from '../dto/user.dto/user.dto';
import { UpdateUserDto } from '../dto/update-user.dto/update-user.dto';
```

```
@Controller('users')
export class UsersController {
 private users: UserDto[] = [];
 private idCounter = 1;
```

```
@Get()
getAllUsers(): UserDto[] {
 return this.users;
}
```

```
@Post()
createUser(@Body() user: CreateUserDto): UserDto {
 const newUser: UserDto = {
 id: this.idCounter++,
 name: user.name,
 email: user.email,
 };

 this.users.push(newUser);
 return newUser;
}

@Put(':id')
updateUser(
 @Param('id') id: string,
 @Body() user: UpdateUserDto,
): UserDto | string {
 const userIndex = this.users.findIndex(u => u.id === Number(id));

 if (userIndex === -1) {
 return 'Utente non trovato';
 }

 this.users[userIndex] = {
 ...this.users[userIndex],
 ...user,
 };

 return this.users[userIndex];
}
```

# Pipe e le validazioni

Le **Pipe** in NestJS servono per:

Validare dati

Trasformare dati

Bloccare dati non validi

Vengono eseguite **prima** che il controller riceva i parametri

```
npm install class-validator class-transformer
```

VALIDAZIONE INT

```
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: string) {
 return this.locationsService.findOne(+id);
}
```

{{base\_url}}/locations/aa

SENZA VALIDAZIONE: RITORNO: 200

CON VALIDAZIONE: RITORNO

```
{
 "message": "Validation failed (numeric string is expected)",
 "error": "Bad Request",
 "statusCode": 400
}
```

il **+id** (operatore unario plus) = converte in numero

Non viene eseguito il metodo se il tipo è errato

ValidationPipe

- ParseIntPipe
- ParseFloatPipe
- ParseBoolPipe
- ParseArrayPipe
- ParseUUIDPipe
- ParseEnumPipe
- DefaultValuePipe
- ParseFilePipe
- ParseDatePipe

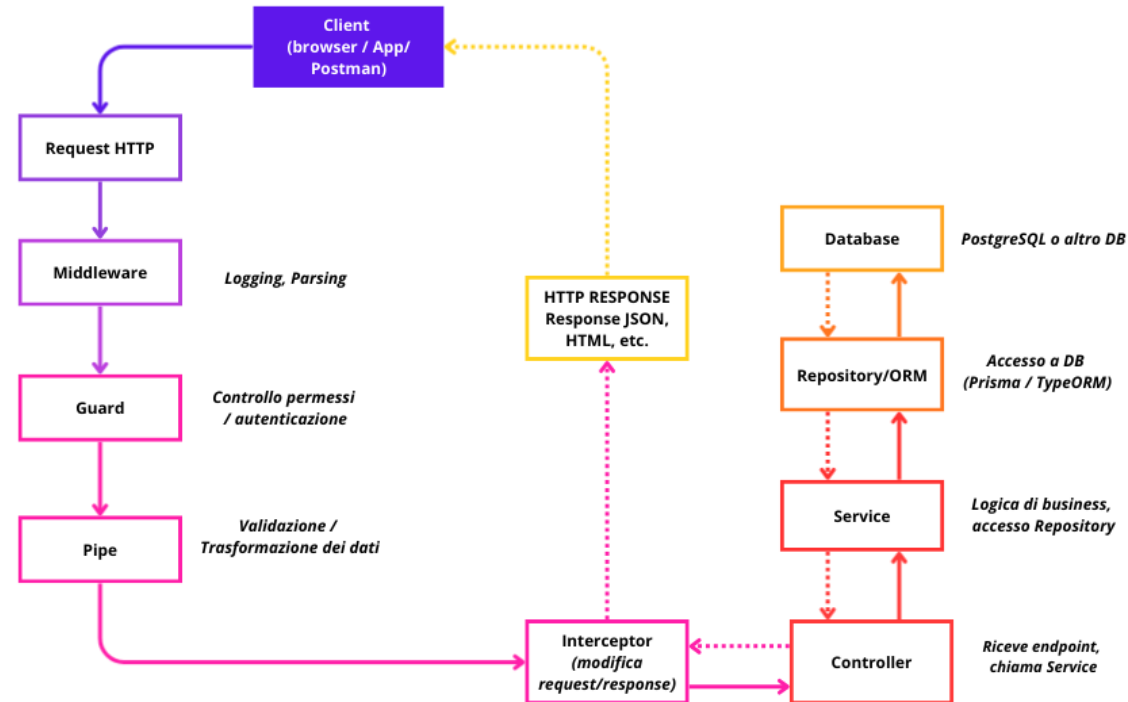
(@Param('id', ParseIntPipe) id: string)



# Schema Concettuale

1. **Request HTTP:** L'utente invia un POST a /posts con il titolo ed il contenuto del post
2. **Middleware:** il primo filtro "grezzo". Ad esempio, estrae indirizzo IP del client e logga il tempo di arrivo della richiesta
3. **Guard:** **verifica che l'utente** abbia il permesso di accedere. Se fallisce si riceve un 401
4. **Interceptor - parte 1 (Pre-Controller):** può, ad esempio, avviare un timer per calcolare quanto tempo impiegherà l'applicazione a generare la risposta.
5. **ValidationPipe (Controllo Qualità):** prende il corpo della richiesta, lo valida contro il DTO e lo trasforma. Se i dati non sono validi, risponde con un 400 Bad Request.
6. **Controller** riceve i dati puliti, chiama i Service appropriati e decide cosa restituire. È il controller che invia effettivamente la risposta al client.
7. **Service (cuore operativo):** contiene ed esegue la logica applicativa e accesso ai dati (DB, repository, API esterne).
8. **Database** esegue la query (PostgreSQL) e restituisce dati al Service.
9. **service** restituisce dati al Controller → response al client
10. **Interceptor - parte 2 (Post-Controller):** Interceptor prende i dati appena restituiti dal Service e li trasforma. ES. Dato del Service {id: 123, title: 'ciao'}; Dato trasformato da Interceptor {status: "success", data: {id:123, title: "Ciao"}, processedAt: "2026-02-06"}
11. **Response** (Invio al Client) inviata al client come JSON.

SCHEMA CONCETTUALE DEL FLUSSO HTTP IN NestJS + Express



# Perchè usare una Pipe

Supponiamo di avere:

GET /users/**abc**

e il controller si aspetta un numero.

Senza Pipe:

```
@Get('/:id')
findOne(@Param('id') id: number) {}
```

In realtà:

id = "abc"

oppure

id = "123"

arrivano sempre come stringhe.

**Le Pipe risolvono questo problema.**

**ParseIntPipe - Trasforma una stringa in intero.**

```
import {
 Controller,
 Get,
 Param,
 ParseIntPipe,
} from '@nestjs/common';

@Controller('users')
export class UsersController {

 @Get('/:id')
 findOne(
 @Param('id', ParseIntPipe)
 id: number,
) {
 return id;
 }
}
```

Richiesta:

GET /users/10

Risultato:

id = 10

# Pipe - Errore automatico

Pipe

```
@Param('id', ParseIntPipe)
```

Richiesta:

GET /users/abc

NestJS risponde automaticamente:

```
{
 "statusCode": 400,
 "message": "Validation failed (numeric string is
expected)",
 "error": "Bad Request"
}
```

Senza scrivere codice aggiuntivo.

I parametri URL arrivano sempre come stringhe. ParseIntPipe li trasforma in numeri e blocca i valori non validi.

/users/123

Senza Pipe funziona comunque, ma id arriva come **stringa**.

```
- @Get(':id')
 findOne(@Param('id') id: number) {
 return id;
 }
```

Richiesta: GET /users/1

Dentro il metodo:

```
- id = "1" // stringa, non number
```

Anche se hai scritto: id: number

TypeScript non converte i dati a runtime.

Senza ParseIntPipe:

"user/1" → id = "1"

Con ParseIntPipe:

"user/1" → id = 1

"user/ab" → 400 Bad Request

# Pipe predefinite di NestJS

- NestJS include diverse Pipe già pronte all'uso.

## Pipe

ValidationPipe

ParseIntPipe

ParseFloatPipe

ParseBoolPipe

ParseUUIDPipe

ParseEnumPipe

ParseArrayPipe

DefaultValuePipe

ParseFilePipe

## Funzione

Valida DTO con class-validator

String → Integer

String → Float

String → Boolean

Verifica UUID valido

Verifica valori di un Enum

String → Array

Imposta un valore predefinito

Valida file upload

# ParseBoolPipe

- Convert stringhe in boolean.
- @Get()  
findAll(  
    @Query('active', ParseBoolPipe)  
    active: boolean,  
){  
    return active;  
}
- Richiesta:
- GET /users?active=true
- Risultato:
- active = true

# ParseUUIDPipe

- Verifica che il parametro sia un UUID valido.
- @Get(':id')  
findOne(  
 @Param('id', ParseUUIDPipe)  
 id: string,  
) {}
- Valido:
- 550e8400-e29b-41d4-a716-446655440000
- Non valido:
- 123
- Risposta:
- 400 Bad Request

# ValidationPipe

È la Pipe più utilizzata.

Serve a validare i DTO.

DTO:

```
export class CreateUserDto {
 name: string;
 email: string;
}
```

Controller:

```
@Post()
create(
 @Body() dto: CreateUserDto,
) {}
```

Da sola non basta.

Serve la ValidationPipe.

**main.ts**

```
import { ValidationPipe } from '@nestjs/common';
app.useGlobalPipes(
 new ValidationPipe(),
);
```

Ora tutti i DTO verranno validati automaticamente.

DTO con class-validator

```
import {
 IsEmail,
 IsNotEmpty,
} from 'class-validator';
```

```
export class CreateUserDto {
```

```
 @IsNotEmpty()
 name: string;
```

```
 @IsEmail()
 email: string;
```

```
}
```

# Pipes - Dto + ValidationPipe

```
npm install class-validator class-transformer
```

validare TUTTI i campi di una API:

```
import { IsString, IsOptional } from 'class-validator';
```

```
export class CreateLocationDto {
```

```
 @IsString()
```

```
 name: string;
```

```
 @IsString()
```

```
 @IsOptional()
```

```
 address?: string;
```

```
}
```

come detto: se non si passa parametro name:

```
{
 "message": [
 "name must be a string"
],
 "error": "Bad Request",
 "statusCode": 400
}
```

**IMPORTANTE:** devono essere attivate le ValidationPipes in `main.ts` (altrimenti i decorator NON vengono presi in considerazione)

**main.ts**

```
import { NestFactory } from '@nestjs/core';
```

```
import { AppModule } from './app.module';
```

```
import { ValidationPipe } from '@nestjs/common';
```

```
async function bootstrap() {
```

```
 const app = await NestFactory.create(AppModule);
```

```
 app.useGlobalPipes(new ValidationPipe());
```

```
 await app.listen(process.env.PORT ?? 3000);
```

```
}
```

```
bootstrap();
```



# Middleware

Un **middleware** è una funzione che viene eseguita **prima del controller (route handler)**.

Intercetta la request HTTP e può:

- leggere/modificare request e response
- validare dati o headers
- controllare token/autenticazione
- eseguire logging
- bloccare la request
- passare il controllo al controller

## nest g middleware logger

```
src/
├── logger/
│ ├── logger.middleware.ts
│ └── logger.middleware.spec.ts
```

## il middleware

```
import { Injectable, NestMiddleware } from '@nestjs/common';
```

```
@Injectable()
export class LoggerMiddleware implements NestMiddleware {
 use(req: any, res: any, next: () => void) {
 console.log('Request from logger middleware: ', req.method, req.url);
 next();
 }
}
```

## In AppModule:

```
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { LoggerMiddleware } from './logger/logger.middleware';
@Module({})
export class AppModule implements NestModule {
 configure(consumer: MiddlewareConsumer) {
 consumer.apply(LoggerMiddleware).forRoutes('*');
 }
}
```

# Middleware Routes

Possiamo applicare middleware diversi su rotte diverse per gestire autorizzazione, sicurezza e logica separata.

```
.forRoutes('admin/*');
```

```
export class AppModule implements NestModule {
```

```
 configure(consumer: MiddlewareConsumer) {
```

```
 // ✅ SOLO CORS su /categories
```

```
 consumer
 .apply(CorsMiddleware)
 .forRoutes('categories');
```

```
 // ✅ ADMIN + CORS su /admin/categories
```

```
 consumer
 .apply(CorsMiddleware, AdminMiddleware)
 .forRoutes('admin/categories');
 }
```

```
}
```

oppure

```
.forRoutes(ProductController);
```

# Middleware Excluding routes

è possibile anche escludere specifiche route dal middleware

```
consumer
 .apply(LoggerMiddleware)
 .exclude(
 { path: 'cats', method: RequestMethod.GET },
 { path: 'cats', method: RequestMethod.POST },
)
 .forRoutes(CatsController);
```

# Esempio AuthMiddleware

Per attivare un middleware su una risorsa:

1. Creare il middleware
2. Implementare la logica  
(es. validazione headers, logging, token, ecc.)
3. Registrare il middleware nel Module
4. Associare il middleware alle rotte con forRoutes()

## users.module

```
import { MiddlewareConsumer, Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { AuthMiddleware } from 'src/auth/auth.middleware';
```

```
@Module({
 controllers: [UsersController]
})
export class UsersModule {
 configure(consumer: MiddlewareConsumer) {
 consumer.apply(AuthMiddleware).forRoutes(UsersController);
 }
}
```

## auth.middleware

```
import { Injectable, NestMiddleware } from '@nestjs/common';
```

```
@Injectable()
export class AuthMiddleware implements NestMiddleware {
 use(req: any, res: any, next: () => void) {
 console.log('AuthMiddleware: Checking authentication...');
 console.log('Request Headers:', req.headers);
 console.log('Request Method:', req.method);
 if (req.headers['email'] !== 'admin@email.it'
 || req.headers['password'] !== 'admin'
) {
 console.log('AuthMiddleware: Unauthorized access attempt');
 return res.status(401).json({ message: 'Unauthorized' });
 }
 next();
 }
}
```

# test con curl del middleware

richiesta errata

```
curl.exe -X GET "http://localhost:3000/users" `
-H "token: 123"
401
```

richiesta corretta

```
curl.exe -X GET "http://localhost:3000/users" `
-H "email: admin@email.it" `
-H "password: admin"
```

# Guard

## Cosa sono i Guard?

I **Guard** sono componenti che decidono se una richiesta può accedere a una rotta o a un controller.

## Funzione principale

Intercettano la richiesta **prima** del controller

Verificano condizioni di accesso

Restituiscono:

true → accesso consentito

false → accesso negato

## Quando utilizzare un Guard?

Verifica autenticazione (utente loggato)

Controllo ruoli (Admin, User, Manager)

Controllo permessi

Verifica token JWT

Protezione di API riservate

Controllo throttling

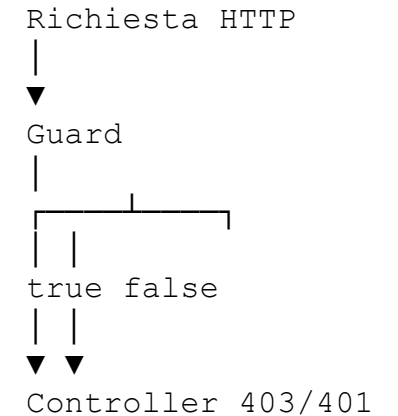
## Interfaccia CanActivate

Ogni Guard implementa:

CanActivate

e il metodo:

```
canActivate(
 context: ExecutionContext,
): boolean {
 return true;
}
```



## Vantaggi

- ✓ Separazione della logica di sicurezza
- ✓ Codice riutilizzabile
- ✓ Maggiore manutenibilità
- ✓ Protezione centralizzata delle rotte

## Concetto chiave

Un Guard è il "buttafuori" dell'applicazione: decide chi può entrare e chi deve rimanere fuori.

# Guard Auth

- Un guard Auth:
- `nest g guard auth`
- 1. Legge i ruoli richiesti dalla rotta con `@Roles()`
  2. Recupera l'utente da `request.user` in base all'header nella request
  3. Confronta i ruoli richiesti con `user.roles`
  4. Se almeno un ruolo coincide → accesso consentito
  5. Altrimenti → 403 Forbidden
- esempio
- <https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/guard-app>

# Guard: Esempio 2

**obiettivo: permettere accesso a chi dispone di un header x-api-key: 12345**

```
nest g guard auth/api-key
```

```
import { CanActivate, ExecutionContext, Injectable } from
 '@nestjs/common';
import { Observable } from 'rxjs';
```

```
@Injectable()
export class ApiKeyGuard implements CanActivate {
 canActivate(
 context: ExecutionContext,
): boolean | Promise<boolean> | Observable<boolean> {
 const request = context.switchToHttp().getRequest();
 const apiKey = request.headers['x-api-key'];
 return apiKey === '12345';
 }
}
```

```
post.controller.ts
nest g controller PostController
```

```
import { Controller, Get, UseGuards } from '@nestjs/common';
import { ApiKeyGuard } from 'src/auth/api-key/api-key.guard';
```

```
@Controller('posts')
export class PostControllerController {
 @Get()
 getHello(): string {
 return 'Hello World!';
 }
}
```

```
@UseGuards(ApiKeyGuard)
@Get('protected')
getProtected(): string {
 return 'This is a protected route!';
}
}
```

**il Guard bloccherà l'accesso se non fornito api-key**



# Guard con Throttling

**Obiettivo:** limitare il numero di richieste per evitare abuso delle API.

```
npm i --save @nestjs/throttler
```

## OPZIONALE:

per modificare il limit nel post-controller

```
@Controller('post-controller')
export class PostControllerController {
```

.....

.....

```
@UseGuards(ThrottlerGuard)
@Throttle({ default: { limit: 3, ttl: 60000 } })
@Get('protected')
getProtected(): string {
 return 'This is a protected route!';
}
```

tempo in millisecondi

In `app.module.ts`:

```
...
import { ThrottlerGuard, ThrottlerModule } from
 '@nestjs/throttler';
```

```
@Module({
 imports: [
 ThrottlerModule.forRoot({
 throttlers: [
 {
 ttl: 60000,
 limit: 10,
 },
],
 })
],
 providers: [
 {
 provide: APP_GUARD,
 useClass: ThrottlerGuard,
 },
],
})
export class AppModule {}
```

# Guard e Ruoli senza JWT

## Il problema: distinguere gli utenti

Obiettivo: Alcune rotte devono essere accessibili solo ad alcuni ruoli

/admin → solo admin  
/profile → admin e user  
/public → tutti

idea didattica:

1. Creo una rotta posts
2. Simulo un utente loggato
3. Definisco i ruoli disponibili
4. Creo @Roles()
5. Creo RolesGuard
6. Proteggo le rotte

test con admin:

GET http://localhost:3000/posts/admin  
x-role-key: admin-key

oppure test con user

GET http://localhost:3000/posts/user  
x-role-key: user-key

*<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/guard-roles>*

# Guard: Esempio3 - Roles enum + guard fake

In questo esempio introduciamo un sistema di autorizzazione basato sui **ruoli degli utenti**. Per prima cosa definiamo un **enum Roles** che rappresenta i diversi livelli di accesso dell'applicazione, ad esempio ADMIN, MANAGER e USER.

Successivamente realizziamo un **Guard simulato (Fake Guard)** che non verifica realmente l'identità dell'utente tramite database o JWT, ma utilizza dati fittizi per dimostrare il funzionamento del controllo degli accessi. Il Guard intercetta la richiesta, legge il ruolo associato all'utente simulato e verifica se possiede i permessi necessari per accedere alla risorsa richiesta.

```
nest new guard-roles
```

Enum per i ruoli

```
src/auth/role.enum.ts
```

```
export enum Role {
```

```
 Admin = 'admin',
```

```
 User = 'user',
```

```
}
```

```
nest g guard auth/fake-auth
```

```
src/auth/fake-auth.ts
```

```
import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';
```

```
import { Observable } from 'rxjs';
```

```
import { Role } from '../role.enum';
```

```
@Injectable()
```

```
export class FakeAuthGuard implements CanActivate {
```

```
 canActivate(
```

```
 context: ExecutionContext,
```

```
): boolean | Promise<boolean> | Observable<boolean> {
```

```
 const request = context.switchToHttp().getRequest();
```

```
 const roleKey = request.headers['x-role-key'];
```

```
 let roles = [Role.User];
```

```
 if (roleKey === 'admin-key') {
```

```
 roles = [Role.Admin];
```

```
 }
```

```
 if (roleKey === 'user-key') {
```

```
 roles = [Role.User];
```

```
 }
```

```
 request.user = {
```

```
 id: 1,
```

```
 username: 'testuser',
```

```
 roles,
```

```
 };
```

```
 return true;
```

```
 }
```

```
}
```

```
se NON arriva un utente lo assegna di prova
```

# Guard: Esempio 3 - Roles decorator + RolesGuard

nest g decorator auth/roles

```
import { SetMetadata } from
 '@nestjs/common';
import { Role } from '../role.enum';
export const ROLES_KEY = 'roles';
export const Roles = (...args: Role[])
 => SetMetadata(ROLES_KEY,
 args);
```

@Roles() attacca alla rotta l'elenco dei ruoli autorizzati.

creiamo il guard  
RolesGuard controlla se il ruolo dell'utente è tra quelli richiesti dalla rotta.

nest g guard auth/roles

```
import { CanActivate, ExecutionContext, Injectable } from
 '@nestjs/common';
import { Observable } from 'rxjs';
import { Role } from '../role.enum';
import { Reflector } from '@nestjs/core';
import { ROLES_KEY } from '../roles.decorator';
```

```
@Injectable()
export class RolesGuard implements CanActivate {
 constructor(private reflector: Reflector) {}
 canActivate(
 context: ExecutionContext,
): boolean | Promise<boolean> | Observable<boolean> {
 const handler = context.getHandler();
 const controller = context.getClass();
 const roles = this.reflector.getAllAndOverride<Role[]>(
 ROLES_KEY,
 [handler, controller],
);
 if (!roles) {
 return true;
 }
 const request = context.switchToHttp().getRequest();
 const user = request.user;
 console.log('User roles:', user.roles);
 return roles.some(role => user.roles.includes(role));
 }
}
```

notare: in questo modo si legge il meta data iniettato da @Roles(Role.Admin)

```
const handler =
context.getHandler();
const controller =
context.getClass();
const roles =
this.reflector.getAll
AndOverride<Role[]>(
 ROLES_KEY,
 [handler,
controller],
);
```

# Guard: Esempio 3 - Controller

nest g controller posts  
src/posts/posts.controller.ts

```
import { Controller, Get, UseGuards } from '@nestjs/common';
import { FakeAuthGuard } from '../auth/fake-auth/fake-auth.guard';
import { Role } from '../auth/role.enum';
import { Roles } from '../auth/roles/roles.decorator';
import { RolesGuard } from '../auth/roles/roles.guard';
```

```
@Controller('posts')
@UseGuards(FakeAuthGuard, RolesGuard)
```

```
export class PostsController {
 @Get()
 getPosts() {
 return 'Lista post pubblica';
 }
}
```

```
@Roles(Role.Admin)
```

```
@Get('admin')
```

```
getAdminPosts() {
 return 'Post visibili solo agli admin';
}
```

```
@Roles(Role.User)
```

```
@Get('user')
```

```
getUserPosts() {
 return 'Post visibili solo agli user';
}
}
```

```
@UseGuards(FakeAuthGuard, RolesGuard)
```

Significa:

1. FakeAuthGuard crea request.user
2. RolesGuard legge request.user.role
3. Il controller viene eseguito solo se il ruolo è corretto

Schema:



# Variabili d'ambiente (.env)

## GESTIONE CONFIGURAZIONI SENSIBILI

Le variabili d'ambiente servono a **separare le configurazioni sensibili dal codice**. **Port, connessione db, chiavi JWT o API key** non devono **MAI ESSERE SCRITTE NEL CODICE**, ma caricati dal file `.env`.

Con Node.js, la libreria `dotenv` permette di leggere queste variabili all'avvio del server.

**Mai committare .env in repository pubblici.**

Il codice `dotenv` deve essere eseguito prima di usare qualsiasi variabile d'ambiente, ad esempio prima di connettersi al database o avviare il server

COMANDO: `npm i dotenv`

`.env` files

`APP_NAME=Guard Roles App`

`APP_ENV=development`

`DB_HOST=localhost`

`DB_PORT=3306`

`DB_USER=root`

`DB_PASSWORD=secret`

`DB_NAME=guard_roles_db`

[https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/dbconnection\\_con\\_env](https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/dbconnection_con_env)

# Utilizzo del file .env con la libreria dotenv

## Perché usare un file .env?

Un file .env permette di separare le configurazioni dal codice sorgente.

È utile per memorizzare:

- Credenziali database
- Chiavi API
- Porte del server
- URL di servizi esterni
- Configurazioni specifiche per ambiente

```
npm install dotenv
```

file .env

PORT=3000

DB\_HOST=localhost

DB\_USER=root

DB\_PASSWORD=password123

JWT\_SECRET=mySecretKey

dotenv.config() deve essere eseguito **una sola volta all'avvio**  
**esempio in main.ts** Poi in qualsiasi file:  
process.env.NOME\_VARIABILE

## main.ts

```
import * as dotenv from 'dotenv';
dotenv.config();
```

```
async function bootstrap() {
 const app = await NestFactory.create(AppModule);
 await app.listen(process.env.PORT || 3000);
}
bootstrap();
```

## Utilizzo delle variabili

```
const port = process.env.PORT;
const dbHost = process.env.DB_HOST;
```

```
console.log(port);
console.log(dbHost);
```

# Esempio1: .env con libreria dotenv e info

src/main.ts

```
import * as dotenv from 'dotenv';
dotenv.config();
```

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
async function bootstrap() {
 const app = await NestFactory.create(AppModule);
 const port = process.env.PORT || 3000;
 await app.listen(port);
 console.log(`Server running on port ${port}`);
}
bootstrap();
```

nest g service database

src/database/database.service.ts

```
import { Injectable } from '@nestjs/common';
```

```
@Injectable()
```

```
export class DatabaseService {
```

```
 returnConnectionInfo() {
```

```
 return {
```

```
 host: process.env.DB_HOST,
```

```
 port: Number(process.env.DB_PORT),
```

```
 user: process.env.DB_USER,
```

```
 database: process.env.DB_NAME,
```

```
 }
```

```
 }
```

```
}
```

.env

↓

process.env

↓

DatabaseService

↓

test connessione MySQL

↓

DatabaseController



# .env e Config Module

[https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/dbconnection\\_configmodule](https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/dbconnection_configmodule)

I file .env si possono leggere con dotenv; in NestJS di solito si usa ConfigModule, che offre un approccio più integrato e ordinato.

Può anche essere cachato: con ConfigModule puoi scrivere:

```
ConfigModule.forRoot({
 isGlobal: true,
 cache: true,
})
```

Il vantaggio di cache: true è che NestJS **memorizza le variabili già lette** e non rilegge ogni volta da process.env

## COMANDO:

```
npm i @nestjs/config
npm i @nestjs/config mysql2
```

## .env

```
APP_NAME=dbconnection_con_env
APP_PORT=3000
APP_ENV=development
```

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=root
DB_NAME=hamburgeria
```

## app.module.ts

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { DatabaseController } from './database/database.controller';
import { DatabaseService } from './database/database.service';
```

```
@Module({
 imports: [
 ConfigModule.forRoot({
 isGlobal: true,
 }),
],
 controllers: [DatabaseController],
 providers: [DatabaseService],
})
export class AppModule {}
```

rendiamo ConfigService disponibile in tutta l'app, senza dover importare ConfigModule in ogni modulo.

isGlobal rende ConfigModule disponibile ovunque.

## .env: esempio 1a - info

**in app.controller**

```
@Controller()
export class AppController {
 constructor(private readonly appService:
 AppService) {}
```

```
 @Get('info')
```

```
 getInfo() {
```

```
 return {
```

```
 appName: process.env.APP_NAME,
```

```
 port: process.env.PORT
```

```
 };
```

```
 }
```

```
}
```

**in app.service.ts con il ConfigService**

```
getInfo(): string {
```

```
 return this.configService.get('APP_NAME') + ' is running in '
 + this.configService.get('APP_ENV') + ' environment.';
```

```
}
```

**in app.controller.ts**

```
@Get('info')
```

```
getInfo(): string {
```

```
 return this.appService.getInfo();
```

```
}
```

# .env: esempio 2 - database e connection reale

**npm i dotenv mysql2**

mysql2 permette di collegarsi a MySQL da Node.js.

**nest g controller database**

```
import { Controller, Get } from '@nestjs/common';
import { DatabaseService } from './database.service';
```

```
@Controller('database')
export class DatabaseController {
 constructor(private readonly databaseService: DatabaseService) {}

 @Get('check-connection')
 async checkConnection() {
 return this.databaseService.testConnection();
 }
}
```

```
{
 "success": true,
 "message": "Connessione al database riuscita"
}
```

**nest g service database**

**src/database/database.service.ts**

```
import { Injectable } from '@nestjs/common';
import { Connection, createConnection } from 'mysql2/promise';
```

```
@Injectable()
export class DatabaseService {
 private conn?: Promise<Connection>;

 private connect(): Promise<Connection> {
 if (!this.conn) {
 this.conn = createConnection({
 host: process.env.DB_HOST,
 port: Number(process.env.DB_PORT),
 user: process.env.DB_USER,
 password: process.env.DB_PASSWORD,
 database: process.env.DB_NAME,
 });
 }
 }
```

```
 return this.conn;
 }
```

```
 private getConnection(): Promise<Connection> {
 return this.connect();
 }
```

```
 public async query(sql: string, params: any[] = []) {
 const connection = await this.getConnection();
 const [rows] = await connection.execute(sql, params);
 return rows;
 }
```

```
 public async execute(sql: string, params: any[] = []) {
 const connection = await this.getConnection();
 const [result] = await connection.execute(sql, params);
 return result;
 }
}
```

# .env: Esempio 4 – Database Service

**nest g service database**

**nest g controller database**

**src/database/database.controller.ts**

```
import { Controller, Get } from '@nestjs/common';
import { DatabaseService } from './database.service';
```

```
@Controller('database')
export class DatabaseController {
 constructor(private readonly databaseService:
 DatabaseService) {}
```

```
 @Get('test')
 async testConnection() {
 return this.databaseService.testConnection();
 }
}
```

Il constructor inietta ConfigService nel service che lo deve usare.

**src/database/database.service.ts**

```
import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { createConnection } from 'mysql2/promise';
@Injectable()
export class DatabaseService {
 constructor(private readonly configService: ConfigService) {}
 async testConnection() {
 try {
 const connection = await createConnection({
 host: this.configService.get<string>('DB_HOST'),
 port: Number(this.configService.get<string>('DB_PORT')),
 user: this.configService.get<string>('DB_USER'),
 password: this.configService.get<string>('DB_PASSWORD'),
 database: this.configService.get<string>('DB_NAME'),
 });
 await connection.ping();
 await connection.end();
 return {
 success: true,
 message: 'Connessione al database riuscita',
 };
 } catch (error) {
 return {
 success: false,
 message: 'Connessione al database fallita',
 };
 }
 }
}
```

# Sistema di Logging: cos'è e perché è importante

Il **sistema di logging** è il meccanismo che registra tutto ciò che accade in un'applicazione:

operazioni eseguite

errori e anomalie

azioni degli utenti

eventi di sistema

👉 In pratica: è il **“diario” del software**

diverse modalità:

log semplice con **console.log**

log semplice con **Logger**

log su file e Filter Errori con **Winston**

è importante in quanto:

✓ **Debug e risoluzione errori**

Permette di capire cosa è successo quando qualcosa non funziona

✓ **Monitoraggio del sistema**

Aiuta a controllare lo stato e le prestazioni dell'applicazione

✓ **Sicurezza**

Traccia accessi, tentativi sospetti e attività anomale

✓ **Manutenzione e miglioramento**

Fornisce dati utili per ottimizzare il software

<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/app-logger-middleware>

# Log semplice con console.log()

```
import { Controller, Get } from '@nestjs/common';
import { AppService } from './app.service';
```

```
@Controller()
export class AppController {
 constructor(private readonly appService:
 AppService) {}
```

```
@Get()
getHello(): string {
 console.log('getHello called');
 return this.appService.getHello();
}
}
```

- console.log() stampa un messaggio nel terminale. È semplice, ma poco strutturato.
- Non indica automaticamente il contesto della classe
- e non distingue bene tra log, warning ed errori.

# il Logger di default in Nest - Controller

Logger è il sistema di log integrato in NestJS.

Aggiunge contesto, timestamp e livello del messaggio.

```
[Nest] 45568 - 28/03/2026, 16:32:37 LOG [AppController] getHello
called
```

log semplice su controller:

utilizza la libreria Logger

**in app.controller**

```
import { Controller, Get, Logger } from '@nestjs/common';
```

```
import { AppService } from './app.service';
```

```
@Controller()
```

```
export class AppController {
```

```
 private readonly logger = new Logger(AppController.name);
```

```
 constructor(private readonly appService: AppService) {}
```

```
 @Get()
```

```
 getHello(): string {
```

```
 this.logger.log('getHello called');
```

```
 return this.appService.getHello();
```

```
 }
```

```
}
```

- il log da terminale

- 

```
[16:32:30] Found 0 errors. Watching for file changes.

[Nest] 45568 - 28/03/2026, 16:32:31 LOG [NestFactory] Starting Nest application...
[Nest] 45568 - 28/03/2026, 16:32:31 LOG [InstanceLoader] AppModule dependencies initialized +6ms
[Nest] 45568 - 28/03/2026, 16:32:31 LOG [RoutesResolver] AppController {}: +2ms
[Nest] 45568 - 28/03/2026, 16:32:31 LOG [RouterExplorer] Mapped {/, GET} route +2ms
[Nest] 45568 - 28/03/2026, 16:32:31 LOG [NestApplication] Nest application successfully started +1ms
[Nest] 45568 - 28/03/2026, 16:32:37 LOG [AppController] getHello called
```

# il Logger di default in Nest - Service

in **app.service**

```
import { Injectable, Logger } from '@nestjs/common'
```

```
@Injectable()
```

```
export class AppService {
```

```
 private readonly logger = new
 Logger(AppService.name);
```

```
 getHello(): string {
```

```
 this.logger.log('getHello called from service');
```

```
 return 'Hello World!';
```

```
 }
```

```
}
```

nella console

```
[Nest] 2144 - 28/03/2026, 16:35:25 LOG [RouterExplorer] Mapped {/, GET} route +2ms
[Nest] 2144 - 28/03/2026, 16:35:25 LOG [NestApplication] Nest application successfully started +2ms
[Nest] 2144 - 28/03/2026, 16:35:30 LOG [AppController] getHello called
[Nest] 2144 - 28/03/2026, 16:35:30 LOG [AppService] getHello called from service
```



# cosa è Winston

Winston è una libreria per **Node.js** che serve a:

- registrare log (info, errori, warning, debug...)
- salvarli su più destinazioni (console, file, database...)
- formattarli (JSON, testo, timestamp...)

Permette di:

- scrivere log su console
- scrivere log su file
- gestire livelli diversi: info, warn, error
- formattare i log
- separare i log normali dagli errori

## Vantaggi principali

- Logging strutturato (JSON → utile per analisi e monitoring)
- Scrittura su file (es. error.log, combined.log)
- Integrazione con sistemi esterni (es. ELK, Datadog)
- Livelli di log configurabili

# log middleware con winston

`npm i winston`

`nest g middleware logger`

**app.module.ts:**

```
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { LoggerMiddleware } from './logger/logger.middleware';
```

```
@Module({
 imports: [],
 controllers: [AppController],
 providers: [AppService],
})
export class AppModule implements NestModule {
 configure(consumer: MiddlewareConsumer) {
 consumer
 .apply(LoggerMiddleware)
 .forRoutes('*');
 }
}
```

**src/logger/logger.middleware.ts**

```
import { Injectable, NestMiddleware } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';
import * as winston from 'winston';
```

```
@Injectable()
export class LoggerMiddleware implements NestMiddleware {
 private logger = winston.createLogger({
 level: 'info',
 format: winston.format.combine(
 winston.format.timestamp(),
 winston.format.printf(({ timestamp, level, message }) => {
 return `${timestamp} [${level}] ${message}`;
 })
),
 transports: [
 new winston.transports.Console(),
 new winston.transports.File({ filename: 'logs/requests.log' }),
 new winston.transports.File({ filename: 'logs/errors.log', level: 'error' }),
],
 });
}
```

```
use(req: Request, res: Response, next: NextFunction) {
 const start = Date.now();

 res.on('finish', () => {
 const duration = Date.now() - start;

 const message = `${req.method} ${req.originalUrl} - ${res.statusCode} - ${duration}ms`;

 if (res.statusCode >= 400) {
 this.logger.error(message);
 } else {
 this.logger.info(message);
 }
 });

 next();
}
```

# Accedere al database con SQL puro

[https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/database\\_mysql\\_api](https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/database_mysql_api)

Collegare un progetto NestJS a un database MySQL  
senza usare TypeOrm

Utilizzo di query SQL scritte manualmente per capire  
bene cosa succede “sotto il cofano”, prima di  
passare successivamente a TypeORM.

## Obiettivi:

installare il driver MySQL

creare una connessione al database

generare una resource posts

creare un CRUD con query SQL

aggiungere categories

introdurre le relazioni tra tabelle

```
nest new database_mysql_api
```

```
npm install mysql2
```

```
nest g resource posts --no-spec
```

```
npm i dotenv
```

```
nest g resource categories --no-spec
```

## main.ts

```
import 'dotenv/config';
```

```
...
```

```
async function bootstrap() {
```

```
posts/
├─ dto/
├─ entities/
├─ posts.controller.ts
├─ posts.module.ts
└─ posts.service.ts
```

# service database

<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/app-db>

nest g service database/mysql

database/mysql/db.service.ts

```
import { Injectable } from '@nestjs/common';
import { Connection, createConnection } from 'mysql2/promise';
```

```
@Injectable()
export class DbService {
 private conn!: Promise<Connection>;
 constructor() {
 this.conn = this.connect();
 }
 private async connect() {
 try {
 if (!this.conn) {
 this.conn = createConnection({
 host: process.env.DB_HOST,
 user: process.env.DB_USER,
 password: process.env.DB_PASSWORD,
 database: process.env.DB_NAME,
 });
 }
 return this.conn;
 } catch (error) {
 console.error('Error connecting to the database:', error);
 throw error;
 }
 }
}
```

```
private async getConnection() {
 if (!this.conn) {
 this.conn = this.connect();
 }
 return this.conn;
}

public async query(sql: string, params?: any[]) {
 const connection = await this.getConnection();
 const [results] = await connection.execute(sql,
 params);
 return results;
}

public async execute(sql: string, params?: any[]) {
 const connection = await this.getConnection();
 const [result] = await connection.execute(sql,
 params);
 return result;
}
```

Use per **SELECT**:

```
const posts = await this.mysqlService.query<Post>(
 'SELECT * FROM posts WHERE active = ?',
 [true],
);
```

Use per **INSERT**:

```
return this.dbService.execute(
 'INSERT INTO leads (name, surname, gender) VALUES (?, ?, ?)',
 [createLeadDto.name, createLeadDto.surname, createLeadDto.gender]
);

return result.insertId;
```

Use per **UPDATE**

```
const fields: string[] = [];
const values: any[] = [];
for (const key in dto) {
 if (dto[key] !== undefined) {
 fields.push(`${key} = ?`);
 values.push(dto[key]);
 }
}
if (fields.length === 0) {
 throw new BadRequestException('No fields to update');
}
values.push(id);
const sql = `UPDATE leads SET ${fields.join(', ')} WHERE id = ?`;
const res = await this.dbService.execute(sql, values);
return await this.findOne(id);
```

/ **DELETE**:

```
return this.dbService.execute('DELETE FROM leads WHERE id = ?', [id]);
```

# CRUD – findOne e findAll

## posts/posts.service.ts

```
import { MysqlService } from
'../database/mysql/mysql.service';
@Injectable()
export class PostsService {
 constructor(private readonly mysqlService:
 MysqlService) {}

 async findAll() {
 return this.mysqlService.query(`
 SELECT
 posts.id,
 posts.title,
 posts.slug,
 posts.content,
 posts.active,
 posts.created_at,
 categories.name AS category_name
 FROM posts
 LEFT JOIN categories
 ON categories.id = posts.category_id
 WHERE posts.active = ?
 `, [true]);
 }
 ... altri metodi
}
```

## pipes

### main.ts

```
import { NestFactory } from
'@nestjs/core';
import { AppModule } from
'./app.module';
import { ValidationPipe } from
'@nestjs/common';
async function bootstrap() {
 const app = await
 NestFactory.create(AppModule);
 app.enableCors({
 origin: '*',
 });
 app.useGlobalPipes(new
 ValidationPipe());
 await app.listen(process.env.PORT ??
 3000);
}
bootstrap();
```

## dto

### create-post.dto.ts

```
import { IsBoolean, IsInt, IsNotEmpty, IsString }
from "class-validator";

export class CreatePostDto {
 @IsNotEmpty()
 @IsString()
 title!: string;

 @IsNotEmpty()
 @IsString()
 slug!: string;

 @IsNotEmpty()
 @IsString()
 content!: string;

 @IsBoolean()
 active?: boolean = true;

 @IsNotEmpty()
 @IsInt()
 category_id!: number;
}
```

# test con swagger

```
npm install @nestjs/swagger
```

## main.ts

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
```

```
async function bootstrap() {
 const app = await NestFactory.create(AppModule);
```

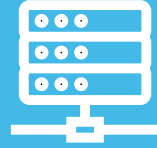
```
 const config = new DocumentBuilder()
 .setTitle('Blog API')
 .setDescription('API NestJS con MySQL e CRUD Posts/Categories')
 .setVersion('1.0')
 .addTag('posts')
 .addTag('categories')
 .build();
```

```
 const document = SwaggerModule.createDocument(app, config);
```

```
 SwaggerModule.setup('api', app, document);
```

```
 await app.listen(process.env.PORT ?? 3000);
}
```

```
bootstrap();
```



# Database & ORM

con Object Relational Mapper **TypeORM**

# Backend e database: ruolo

## BACKEND (server-side)

Il **backend** è il layer applicativo che:  
espone **API** (REST / GraphQL / RPC)  
implementa **business logic**  
gestisce **autenticazione e autorizzazione**  
valida input/output (DTO, schema)  
orchestra accesso a **database, cache, servizi esterni**  
garantisce **sicurezza, performance e scalabilità**

In **Node.js** tipicamente:

Express / Fastify / **NestJS**

**PATTERN: Client → Controller → Service**  
**→ ORM → Database**

Comunicazione asincrona (**event loop**)

## DATABASE

Il **database** è il layer di **persistenza dei dati**:

archivia dati in modo strutturato

garantisce **ACID (Atomicity, Consistency, Isolation, Durability)**

gestisce concorrenza e lock

indicizza per query efficienti

applica vincoli (FK, unique, check)

! Il database **non deve contenere business logic**  
(salvo casi mirati: trigger, view).

Backend	Database
Regole di business	Integrità dei dati
Autorizzazioni	Vincoli strutturali
Validazione semantica	Tipi, chiavi, relazioni
Workflow	Persistenza
Cache, rate limit	Query e indici

<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/app-db-typeorm>



# MySQL vs PostgreSQL (confronto tecnico)

## MySQL (InnoDB)

### Punti di forza

- Semplice da amministrare
- Molto diffuso (hosting shared, WordPress)
- Buone performance read-heavy
- Ecosistema enorme

### Limiti

- Tipizzazione meno rigorosa
- JSON supportato ma limitato
- Query complesse meno eleganti
- Conformità SQL parziale
- Quando usarlo
- CRUD semplici
- CMS
- App con schema stabile
- Team junior

## PostgreSQL

### Punti di forza

- SQL completo e rigoroso
- Tipi avanzati (JSONB, ARRAY, ENUM, RANGE)
- Query complesse e performanti
- Migliore gestione concorrenza
- Ottimo per domain-driven design

### Limiti

- Curva di apprendimento più alta
- Setup leggermente più complesso
- Hosting meno “cheap”
- Quando usarlo
- App business-critical
- Sistemi complessi
- Reporting avanzato
- Dati semi-strutturati
- API moderne

# ORM (Object Relational Mapping)

Parte del sistema che permette di interagire con un database relazionale **usando oggetti del linguaggio di programmazione, invece di scrivere direttamente query SQL.**

È come una specie di "traduttore" tra gli oggetti del codice e le tabelle del database. Permette di lavorare con classi e metodi invece che con query SQL grezze.

ESEMPIO:

```
user.create({ name: "alice" })
```

## 1. MAPPATURA OGGETTI $\leftrightarrow$ TABELLE

- Le **tabelle** del DB diventano **classi/oggetti**
- Le **righe** diventano **istanze**
- Le **colonne** diventano **proprietà**

## 2. GESTIONE AUTOMATICA DELLE QUERY

- Inserimenti (INSERT), aggiornamenti (UPDATE), cancellazioni (DELETE)
- Selezioni (SELECT)
- Filtri, join, relazioni tra tabelle

# Driver SQL vs ORM (Object Relational Mapper)

Scelta tra scrivere codice "vicino al DB" o "vicino al linguaggio di programmazione"

CARATTERISTICA	DRIVER SQL	ORM (OBJECT-RELATIONAL MAPPING)
Caratteristiche principali e vantaggi	• Parte di software che permette a codice di parlare direttamente con DB: traduce chiamate del codice in query SQL comprensibili a DB    • Scrivi <b>SQL puro</b> (es. <code>SELECT * FROM...</code>).    • Mapping manuale del DB → oggetti JS	• Trasforma tabelle del DB in oggetti    • <b>Astrazione</b> tra DB e codice. Scrivi <b>metodi JavaScript/TypeScript</b>.    • Codice più leggibile, più sicuro (SQL injection)    • Migrazioni automatiche    • Relazioni semplificate
Controllo	<b>Massimo</b> : ottimizzi ogni singola virgola	<b>Minimo</b> : l'ORM decide come scrivere la query
Velocità	<b>Molto veloce</b> (nessun "traduttore" in mezzo)	Leggermente più lento ( <b>overhead di traduzione</b> )
Contro	• Più codice boilerplate    • <b>Query manuali, più errori</b>    • <b>Migrazioni e relazioni gestite a mano</b>	• <b>Overhead</b>    • Meno controllo su SQL    • <b>Query complesse più difficili</b>
Esempio	<b>pg, mysql2, sqlite3</b>	<b>TypeORM</b>
	<pre>sql SELECT * FROM users WHERE active = true;</pre>	<pre>repository.find({   where: { active: true }, });</pre>

# ORM Review

ORM	Tipologia	Punti di forza	Contro	Ideale per
Prisma	ORM moderno + <b>query builder</b>	• <b>TS-first</b>     • <b>Type safety</b> eccellente (Typescript)     • Facile da integrare con NestJS                    • <b>Schema DB dichiarativo</b>           • Autocompletamento ottimo                    • Migrazioni integrate	• Meno flessibile su SQL molto complesso     • Runtime leggermente più pesante	• <b>API moderne</b>     • <b>Progetti Typescript</b>     • Team piccoli/medi
Sequelize	ORM storico	• Sul mercato da molto tempo     • Supporta <b>quasi tutti DB SQL</b>      • Ampia documentazione	• Typescript debole                    • API verbose           • Manutenzione percepita come più lenta	• Progetti legacy     • Team con forte background SQL
<b>TypeORM</b>	ORM classico <b>basato su decorator</b>	• Modello <b>entity-oriented, basato su classi</b>      • Relazioni esplicite     • NestJS-friendly	• Migrazioni meno affidabili     • Performance variabili	• Applicazioni enterprise     • Chi viene da mondo Java/Spring o C#/.NET     • Architetture DDD.

# Accesso al Database: TypeORM

<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/app-api-querybuilder-repository>

**TypeORM** è un ORM (Object Relational Mapper) per Node.js e TypeScript.

👉 Permette di lavorare con il database usando **classi e oggetti**, invece di SQL puro.

## installazione

```
npm install @nestjs/typeorm typeorm
```

## ORM = Object Relational Mapper

👉 Traduce tra:

mondo **oggetti (JavaScript)**

mondo **relazionale (MySQL)**

## Architettura TypeORM

🔧 **Componenti principali**

**Entity** → rappresenta tabella

**Repository** → gestisce dati

**Service** → logica applicativa

**Controller** → API + routing system

# TypeORM: Entity

Un **Entity** è una classe TypeScript che rappresenta una tabella del database.

Ogni oggetto dell'Entity corrisponde a una riga della tabella, mentre ogni proprietà della classe corrisponde a una colonna.

TypeORM utilizza i decorator per definire come la classe deve essere mappata sul database.

## Perché usare le Entity?

Permettono di lavorare con oggetti invece che con SQL puro.

Definiscono la struttura dei dati in un unico punto.

Consentono a TypeORM di generare query e gestire le relazioni tra tabelle.

Migliorano la leggibilità e la manutenibilità del codice.

## creare il file category.entity.ts

src/categories/entities/category.entity.ts

```
@Entity('products')
export class Product {
 @PrimaryGeneratedColumn()
 id: number;

 @Column()
 name: string;

 @Column('decimal')
 price: number;

 @Column({ default: true })
 active: boolean;
}
```

# TypeORM: Repository vs QueryBuilder

Quando lavoriamo con TypeORM abbiamo due approcci principali per interrogare il database: **Repository** e **QueryBuilder**.

**Una regola pratica è la seguente:**

Se devi fare un semplice CRUD → usa il **Repository**.

Se devi costruire query articolate con join, report o statistiche → usa il **QueryBuilder**.

Il **Repository** è l'approccio più semplice e immediato.

TypeORM mette a disposizione una serie di metodi già pronti, come `find()`, `findOne()`, `save()` e `remove()`, che permettono di eseguire le operazioni CRUD senza dover scrivere query complesse. È la scelta ideale quando dobbiamo recuperare, inserire, modificare o cancellare dati in modo rapido e leggibile.

Il **QueryBuilder**, invece, offre un controllo molto più dettagliato sulla query SQL generata. Attraverso una sintassi fluente possiamo aggiungere join, filtri dinamici, ordinamenti, raggruppamenti e persino funzioni di aggregazione. Richiede qualche riga di codice in più, ma diventa indispensabile quando la logica di interrogazione è complessa.

In pratica, il Repository è perfetto per la maggior parte delle operazioni quotidiane, mentre il QueryBuilder entra in gioco quando le esigenze diventano più avanzate.

# Repository Pattern

## il Repository Pattern

È un modello architetturale che separa:

**logica applicativa**

**accesso ai dati**

Il repository si occupa di leggere e scrivere nel database.

## Perché è utile

### Senza Repository

Il service contiene SQL e logica insieme.

### Con Repository

Il service resta pulito e leggibile.

## Esempio Iniezione Repository nel service

```
constructor(
 @InjectRepository(User)
 private userRepo:
 Repository<User>
) {}
```

```
await userRepo.find();
await userRepo.findOne({
 where:{ id:1 }
});
await userRepo.save(user);
await userRepo.delete(1);
```

## • Esempio Service

```
• @Injectable()
export class UserService {

 constructor(
 @InjectRepository(User)
 private repo: Repository<User>
) {}

 async allUsers() {
 return this.repo.find();
 }
}
```



# Repository: Esempio

esempio:

```
repo.find()
repo.save()
repo.delete()
```

Il repository è il ponte tra codice e database

```
const users = await userRepository.find({
 select: {},
 where: {},
 relations: {},
 order: {},
 skip: 0,
 take: 10,
});
```

userRepository.find() ritorna una **Promise** che, con await, diventa un **array di entity User complete**.

```
[
 {
 id: 1,
 name: 'Mario',
 email: 'mario@test.it',
 active: true
 },
 {
 id: 2,
 name: 'Luigi',
 email: 'luigi@test.it',
 active: false
 }
]
```

# Query Builder

## QueryBuilder

È uno strumento di TypeORM per creare query SQL avanzate con codice TypeScript.

Serve quando find() non basta

## usarlo

- ✓ JOIN tra tabelle
- ✓ filtri complessi
- ✓ condizioni dinamiche
- ✓ COUNT / SUM / AVG
- ✓ ORDER BY avanzati
- ✓ query professionali

## SELECT

```
const users = await repo
 .createQueryBuilder('user')
 .getMany();
```

## WHERE

```
const users = await repo
 .createQueryBuilder('user')
 .where('user.active = :status', {
 status: true })
 .getMany();
```

## LIMIT / TAKE

```
.take(10)
.skip(20)
```

- **ORDER BY**
- .orderBy('user.createdAt', 'DESC')
- **JOIN Relazioni**
- const users = await repo
- .createQueryBuilder('user')
- .leftJoinAndSelect('user.orders', 'order')
- .getMany();
- **COUNT**
- const total = await repo
- .createQueryBuilder('user')
- .getCount();

# Query Builder

## SELECT PERSONALIZZATO

```
const users = await repo
 .createQueryBuilder('user')
 .select(['user.id', 'user.name'])
 .getMany();
```

```
[
 { id: 1, name: 'Mario' },
 { id: 2, name: 'Luigi' }
]
```

## ESEMPIO

```
return this.customerRepo
 .createQueryBuilder('c')
 .where('c.active = 1')
 .orderBy('c.createdAt', 'DESC')
 .getMany();
```

# SELECT

SQL: SELECT \* FROM users;

Repository

```
const users = await userRepository.find();
```

QueryBuilder

```
const users = await userRepository
 .createQueryBuilder('user')
 .getMany();
```

SQL: SELECT id, name FROM users;

Repository

```
const users = await userRepository.find({
 select: {
 id: true,
 name: true,
 },
});
```

QueryBuilder

```
const users = await userRepository
 .createQueryBuilder('user')
 .select(['user.id', 'user.name'])
 .getMany();
```

# WHERE

- SQL: `SELECT * FROM users WHERE active = 1;`
- Repository
- ```
const users = await userRepository.find({  
  where: {  
    active: true,  
  },  
});
```
- QueryBuilder
- ```
const users = await userRepository
 .createQueryBuilder('user')
 .where('user.active = :active', {
 active: true,
 })
 .getMany();
```

# WHERE AND

- SQL: `SELECT * FROM users WHERE active = 1 AND role = 'admin';`
- Repository
- ```
const users = await userRepository.find({
  where: {
    active: true,
    role: 'admin',
  },
});
```
- QueryBuilder
- ```
const users = await userRepository
 .createQueryBuilder('user')
 .where('user.active = :active', {
 active: true,
 })
 .andWhere('user.role = :role', {
 role: 'admin',
 })
 .getMany();
```

# WHERE OR

- SQL: `SELECT * FROM users WHERE role = 'admin' OR role = 'manager';`
- Repository
- ```
const users = await userRepository.find({
  where: [
    { role: 'admin' },
    { role: 'manager' },
  ],
});
```
- QueryBuilder
- ```
const users = await userRepository
 .createQueryBuilder('user')
 .where('user.role = :r1', {
 r1: 'admin',
 })
 .orWhere('user.role = :r2', {
 r2: 'manager',
 })
 .getMany();
```

# LIKE

- SQL: SELECT \* FROM users WHERE name LIKE '%mar%'
- Repository
- import { Like } from 'typeorm';

```
const users = await userRepository.find({
 where: {
 name: Like('%mar%'),
 },
});
```

- QueryBuilder
- const users = await userRepository  
 .createQueryBuilder('user')  
 .where('user.name LIKE :name', {  
 name: '%mar%',  
 })  
 .getMany();



# IN

SQL: `SELECT * FROM users WHERE id IN (1,2,3);`

## Repository

```
import { In } from 'typeorm';
```

```
const users = await userRepository.find({
 where: {
 id: In([1, 2, 3]),
 },
});
```

## QueryBuilder

```
const users = await userRepository
 .createQueryBuilder('user')
 .where('user.id IN (:...ids)', {
 ids: [1, 2, 3],
 })
 .getMany();
```

## Repository:

```
import { In } from 'typeorm';
```

```
async findByIds(ids: number[]) {
 return this.userRepository.find({
 where: {
 id: In(ids),
 },
 });
}
```

## QueryBuilder:

```
async findByIds(ids: number[]) {
 return this.userRepository
 .createQueryBuilder('user')
 .where('user.id IN (:...ids)', { ids })
 .getMany();
}
```

# ORDER BY

- SQL: `SELECT * FROM users ORDER BY name ASC;`
- Repository
- ```
const users = await userRepository.find({  
  order: {  
    name: 'ASC',  
  },  
});
```
- QueryBuilder
- ```
const users = await userRepository
 .createQueryBuilder('user')
 .orderBy('user.name', 'ASC')
 .getMany();
```

# LIMIT

- SQL: `SELECT * FROM users LIMIT 10;`
- Repository
- `const users = await userRepository.find({  
 take: 10,  
});`
- QueryBuilder
- `const users = await userRepository  
 .createQueryBuilder('user')  
 .take(10)  
 .getMany();`

# COUNT

- SQL: `SELECT COUNT(*) FROM users;`
- Repository
- `const total = await userRepository.count();`
- QueryBuilder
- `const total = await userRepository  
 .createQueryBuilder('user')  
 .getCount();`

# INSERT

```
SQL: INSERT INTO users(name,email)
 VALUES('Mario','mario@test.it');
```

## Repository

```
const user = userRepository.create({
 name: 'Mario',
 email: 'mario@test.it',
});

await userRepository.save(user);
```

## QueryBuilder

```
await userRepository
 .createQueryBuilder()
 .insert()
 .into(User)
 .values({
 name: 'Mario',
 email: 'mario@test.it',
 })
 .execute();
```

# UPDATE

SQL: UPDATE users SET active = 1 WHERE id = 10;

## Repository

```
await userRepository.update(
 10,
 {
 active: true,
 },
);
```

## QueryBuilder

```
await userRepository
 .createQueryBuilder()
 .update(User)
 .set({
 active: true,
 })
 .where('id = :id', {
 id: 10,
 })
 .execute();
```

# DELETE

SQL: DELETE FROM users WHERE id = 10;

Repository

```
await userRepository.delete(10);
```

QueryBuilder

```
await userRepository
 .createQueryBuilder()
 .delete()
 .from(User)
 .where('id = :id', {
 id: 10,
 })
 .execute();
```

# GROUP BY

SQL: SELECT role, COUNT(\*) totale FROM users  
GROUP BY role;

Repository

✗ Non supportato direttamente.

QueryBuilder

```
const data = await userRepository
 .createQueryBuilder('user')
 .select('user.role', 'role')
 .addSelect('COUNT(*)', 'totale')
 .groupBy('user.role')
 .getRawMany();
```



# HAVING

SQL: SELECT role, COUNT(\*) totale FROM users  
GROUP BY role HAVING COUNT(\*) > 5;

QueryBuilder

```
const data = await userRepository
 .createQueryBuilder('user')
 .select('user.role', 'role')
 .addSelect('COUNT(*)', 'totale')
 .groupBy('user.role')
 .having('COUNT(*) > :min', {
 min: 5,
 })
 .getRawMany();
```

# JOIN

Immaginiamo:

users

- id
- name
- role\_id

roles

- id
- name

## Entity User

```
@Entity('users')
export class User {
 @PrimaryGeneratedColumn()
 id: number;

 @Column()
 name: string;

 @Column()
 role_id: number;

 @ManyToOne(() => Role, role => role.users)
 @JoinColumn({ name: 'role_id' })
 role: Role;
}
```

## Entity Role

```
@Entity('roles')
export class Role {
 @PrimaryGeneratedColumn()
 id: number;

 @Column()
 name: string;

 @OneToMany(() => User, user => user.role)
 users: User[];
}
```

# SELECT con JOIN

SQL: `SELECT users.*, roles.* FROM users JOIN roles  
ON roles.id = users.role_id;`

Repository:

```
const users = await this.userRepository.find({
 relations: {
 role: true,
 },
});
```

QueryBuilder:

```
const users = await this.userRepository
 .createQueryBuilder('user')
 .innerJoinAndSelect('user.role', 'role')
 .getMany();
```

Risultato:

```
[
 {
 id: 1,
 name: 'Mario',
 role_id: 1,
 role: {
 id: 1,
 name: 'admin'
 }
 }
]
```

# LEFT JOIN

SQL: `SELECT users.*, roles.* FROM users LEFT JOIN  
roles ON roles.id = users.role_id;`

Repository:

```
const users = await this.userRepository.find({
 relations: {
 role: true,
 },
});
```

QueryBuilder:

```
const users = await this.userRepository
 .createQueryBuilder('user')
 .leftJoinAndSelect('user.role', 'role')
 .getMany();
```

# JOIN con WHERE

SQL: `SELECT users.* FROM users JOIN roles ON roles.id = users.role_id WHERE roles.name = 'admin';`

Repository:

```
const users = await this.userRepository.find({
 where: {
 role: {
 name: 'admin',
 },
 },
 relations: {
 role: true,
 },
});
```

QueryBuilder:

```
const users = await this.userRepository
 .createQueryBuilder('user')
 .innerJoinAndSelect('user.role', 'role')
 .where('role.name = :roleName', {
 roleName: 'admin',
 })
 .getMany();
```

# JOIN selezionando solo alcuni campi

- ```
const users = await this.userRepository
  .createQueryBuilder('user')
  .innerJoin('user.role', 'role')
  .select([
    'user.id',
    'user.name',
    'role.id',
    'role.name',
  ])
  .getMany();
```

RAW Query

Con Repository

```
const users = await userRepository.query(`
  SELECT *
  FROM users
`);
```

Ritorna:

Promise<any[]>

Con await diventa:

any[]

Esempio:

```
[
  { id: 1, name: 'Mario' },
  { id: 2, name: 'Luigi' }
]
```

RAW con parametro

```
const users = await userRepository.query(
  'SELECT * FROM users WHERE id = ?',
  [id],
);
```

RAW con LIKE

```
const users = await userRepository.query(
  'SELECT * FROM users WHERE name LIKE ?',
  [`%${q}%`],
);
```

RAW con IN

```
const ids = [1, 2, 3];
```

```
const placeholders = ids.map(() => '?').join(',');
```

```
const users = await userRepository.query(
  'SELECT * FROM users WHERE id IN (${placeholders})',
  ids,
);
```

RAW con JOIN

```
const users = await userRepository.query(`
  SELECT
    users.id,
    users.name,
    roles.name AS role_name
  FROM users
  INNER JOIN roles ON roles.id = users.role_id
`);
```

Relations - Repository

```
const users = await userRepository.find({
  relations: {
    role: true,
  },
});
```

Equivale concettualmente a una LEFT JOIN:

```
SELECT *
FROM users
LEFT JOIN roles ON roles.id = users.role_id;
```

Risultato:

```
[
  {
    id: 1,
    name: 'Mario',
    role: {
      id: 1,
      name: 'admin'
    }
  }
]
```

Con più relazioni:

```
const users = await userRepository.find({
  relations: {
    role: true,
    posts: true,
  },
});
```

Con relazioni annidate:

```
const users = await userRepository.find({
  relations: {
    posts: {
      category: true,
    },
  },
});
```


QueryBuilder Join

- QueryBuilder equivalente:
- `const users = await userRepository
 .createQueryBuilder('user')
 .leftJoinAndSelect('user.role', 'role')
 .getMany();`
- `.leftJoinAndSelect('user.role', 'role')`
- carica anche role nel risultato.
- `.leftJoin('user.role', 'role')`
- fa la join, ma **non carica automaticamente** role nell'oggetto finale.
- Esempio con filtro sulla relation:
- `const users = await userRepository
 .createQueryBuilder('user')
 .leftJoinAndSelect('user.role', 'role')
 .where('role.name = :name', { name: 'admin' })
 .getMany();`

Colonne e campi in NestJS + TypeORM

Dentro una `@Entity()` ogni proprietà della classe diventa normalmente una **colonna del database** tramite `@Column()`.

La classe rappresenta la tabella.
I campi rappresentano le colonne.

```
@Entity()
export class Post {
  @PrimaryGeneratedColumn()
  id: number;
  @Column()
  title: string;
  @Column({ nullable: true })
  summary: string;
}
```

equivalente:

```
CREATE TABLE post (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  summary VARCHAR(255) NULL
);
```

Opzioni utili:

- nullable
- default
- unique
- length

@Index

Indice **composto su più colonne**

Se vuoi un indice su più colonne, il decorator si mette sulla **classe**:

```
import { Entity, Column, PrimaryGeneratedColumn, Index } from 'typeorm';
```

```
@Index(['lastName', 'firstName'])
```

```
@Entity()
```

Su **una singola colonna**

```
import { Entity, Column, PrimaryGeneratedColumn, Index } from 'typeorm';
```

```
@Entity()
```

```
export class User {
```

```
  @PrimaryGeneratedColumn()
```

```
  id: number;
```

```
@Index()
```

```
  @Column()
```

```
  email: string;
```

migrations

le **migrations** sono uno strumento per **gestire in modo controllato le modifiche alla struttura del database** nel tempo

Le migrations permettono di:

- ✓ Tenere **versionato il database** (come fai con Git per il codice)
- ✓ Sincronizzare il DB tra team e ambienti (dev, staging, produzione)
- ✓ Evitare errori manuali
- ✓ Fare rollback se qualcosa va storto

negli script creati in `src/migrations/`

```
npx ts-node -r tsconfig-paths/register  
./node_modules/typeorm/cli.js migration:generate  
src/migrations/init -d ./data-source.ts
```

- confronta le Entity con il database attuale
- genera automaticamente un file migration

```
npx ts-node -r tsconfig-paths/register  
./node_modules/typeorm/cli.js migration:run -d ./data-source.ts
```

- esegue tutte le migration NON ancora eseguite
- legge la tabella interna: migrations
- applica solo quelle mancanti

```
PS C:\Software\nest.js\corso-repository\hamburgeria> npx ts-node -r tsconfig-paths/register ./node_modules/typeorm/cli.js migration:generate src/migrations/init -d ./data-source.ts  
>>  
◇ injected env (10) from .env // tip: % multiple files { path: ['.env.local', '.env'] }  
Migration C:\Software\nest.js\corso-repository\hamburgeria\src\migrations\1776613707745-init.ts has  
been generated successfully.
```

le Migrations - data-source.ts

a cosa serve data-source.ts? Serve a dire alla CLI di TypeORM:

a quale database collegarsi

dove trovare le entity

dove trovare le migration

come confrontare entity e database

come eseguire gli aggiornamenti dello schema

Differenza tra NestJS e TypeORM CLI

App NestJS avviata normalmente TypeOrmModule.forRoot()

Comandi migration da terminale data-source.ts

dentro la route creare data-source.ts

```
import { DataSource } from 'typeorm';
import * as dotenv from 'dotenv';
dotenv.config();
export const AppDataSource = new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST,
  port: process.env.DB_PORT ? parseInt(process.env.DB_PORT) : 5432,
  username: process.env.DB_USERNAME,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_DATABASE,
  entities: [__dirname + '/*.entity{.ts,.js}'],
  migrations: [__dirname + '/migrations/*{.ts,.js}'],
});
```

generate users

```
npx ts-node -r tsconfig-paths/register ./node_modules/typeorm/cli.js
migration:generate src/migrations/users -d ./data-source.ts
```

generate posts

```
npx ts-node -r tsconfig-paths/register ./node_modules/typeorm/cli.js
migration:generate src/migrations/posts -d ./data-source.ts
```

oppure confronta e genera tutte le migrations (generate)

```
npx ts-node -r tsconfig-paths/register ./node_modules/typeorm/cli.js
migration:generate src/migrations/init -d ./data-source.ts
```

run migrations

```
npx ts-node -r tsconfig-paths/register ./node_modules/typeorm/cli.js migration:run
-d ./data-source.ts
```

NOTA: Quando si usa TypeORM con TypeScript, anche i comandi CLI devono essere eseguiti con ts-node, altrimenti Node non riesce a interpretare i file .ts.

migrations up & down

In NestJS + TypeORM

Ogni migration contiene normalmente due metodi:

```
export class CreateUsersTable1710000000000
  implements MigrationInterface {

  public async up(queryRunner: QueryRunner):
    Promise<void> {
    // crea tabella, colonne, indici...
  }

  public async down(queryRunner: QueryRunner):
    Promise<void> {
    // elimina ciò che hai creato in up()
  }

}
```

up()

Serve per **andare avanti** con la struttura del database.

down()

Serve per **tornare indietro**.

Aggiungere una nuova colonna in NestJS + TypeORM

Aggiungere il campo **sale_price** nella tabella products.

```
@Column({  
  type: 'decimal',  
  precision: 8,  
  scale: 2,  
  default: 0.00  
})  
sale_price!: number;
```

Generare la Migration

Dopo aver modificato la entity, esegui:

```
npx ts-node -r tsconfig-paths/register  
  ./node_modules/typeorm/cli.js migration:generate  
  src/migrations/init -d ./data-source.ts
```

```
npx ts-node -r tsconfig-paths/register  
  ./node_modules/typeorm/cli.js migration:run -d ./data-  
  source.ts
```

TypeORM produrrà un file di migrations con up() e down()

migrations e package.json

- "scripts": {
- "migration:generate": "ts-node -r tsconfig-paths/register ./node_modules/typeorm/cli.js migration:generate src/migrations/init -d ./data-source.ts",
- "migration:run": "ts-node -r tsconfig-paths/register ./node_modules/typeorm/cli.js migration:run -d ./data-source.ts«
- }
- npm run migration:generate
- npm run migration:run

Relazioni Database 1:1

1 : 1

Un utente ha un solo profilo

Ogni record della tabella users è associato a un solo record nella tabella profiles, e viceversa.

In NestJS / TypeORM

si usa @ManyToOne()

spesso si aggiunge @JoinColumn() sul lato proprietario della relazione

Esempio pratico

User → dati di accesso

Profile → nome, cognome, avatar, bio

users

| id | email | password |
|----|------------------|----------|
| 1 | mario@email.it ✉ | 123456 |

profiles

| id | nome | cognome | avatar | userId |
|----|-------|---------|----------|--------|
| 1 | Mario | Rossi | foto.jpg | 1 |

👉 userId viene creato grazie a @JoinColumn()

```
import { Entity, PrimaryGeneratedColumn, Column, OneToOne } from 'typeorm';
import { Profile } from './profile.entity';
```

```
@Entity('users')
export class User {
```

```
  @PrimaryGeneratedColumn()
  id: number;
```

```
  @Column()
  email: string;
```

```
  @Column()
  password: string;
```

```
  @OneToOne(() => Profile, profile =>
    profile.user)
  profile: Profile;
}
```

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  OneToOne,
  JoinColumn
} from 'typeorm';

import { User } from './user.entity';
```

```
@Entity('profiles')
export class Profile {
```

```
  @PrimaryGeneratedColumn()
  id: number;
```

```
  @Column()
  nome: string;
```

```
  @Column()
  cognome: string;
```

```
  @Column({ nullable: true })
  avatar: string;
```

```
  @OneToOne(() => User, user => user.profile)
  @JoinColumn()
  user: User;
}
```

La chiave esterna sta dove c'è @JoinColumn()

Relazioni Database 1:n

Un cliente ha molti ordini

Un singolo cliente può avere più ordini, ma ogni ordine appartiene a un solo cliente.

In NestJS / TypeORM

lato “uno” → @OneToMany()

lato “molti” → @ManyToOne()

Esempio pratico

Customer → anagrafica cliente

Order → data ordine, importo, stato

Nota importante

Nella pratica, la chiave esterna si trova quasi sempre nella tabella del lato **N**, quindi in questo caso dentro orders.

Cosa succede nel Database

customers

| id | nome |
|----|-------------|
| 1 | Mario Rossi |

orders

| id | dataOrdine | importo | stato | customerId |
|----|------------|---------|-----------|------------|
| 1 | 2026-01-10 | 120.00 | Pagato | 1 |
| 2 | 2026-01-15 | 80.00 | In attesa | 1 |

👉 La chiave esterna si trova nella tabella **orders** (customerId)

```
import { Entity, PrimaryGeneratedColumn, Column, OneToMany } from 'typeorm';
import { Order } from './order.entity';
```

```
@Entity('customers')
export class Customer {
```

```
  @PrimaryGeneratedColumn()
  id: number;
```

```
  @Column()
  nome: string;
```

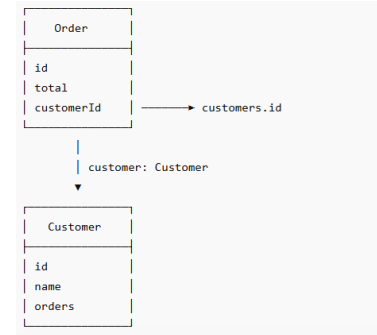
```
  @Column()
  email: string;
```

```
  @OneToMany(() => Order, order =>
    order.customer)
  orders: Order[];
}
```

```
@ManyToOne(() => Customer, customer => customer.orders)
@JoinColumn({ name: 'customer_id' })
customer: Customer;
```

← se voglio chiamarla customer_id

- import {
 - Entity,
 - PrimaryGeneratedColumn,
 - Column,
 - ManyToOne } from 'typeorm';
- import { Customer } from './customer.entity';
- @Entity('orders')
- export class Order {
 - @PrimaryGeneratedColumn()
 - id: number;
 - @Column()
 - dataOrdine: Date;
 - @Column('decimal')
 - importo: number;
 - @Column()
 - stato: string;
 - @ManyToOne(() => Customer, customer => customer.orders)
 - customer: Customer;
 - }



() => Customer # ritorna la CLASSE Customer
customer => customer.orders #dentro la classe Customer, la relazione inversa si chiama orders
customer: Customer; #proprietà customer dentro orders

Relazioni Database n:m

Uno studente segue molti corsi

Uno studente può iscriversi a molti corsi, e ogni corso può avere molti studenti.

In NestJS / TypeORM

si usa `@ManyToMany()`

normalmente serve una **tabella ponte** per collegare le due entità con TypeORM si usa spesso `@JoinTable()`

Esempio pratico

Student

Course

tabella intermedia: student_courses

Come si rappresentano in NestJS

NestJS, insieme a **TypeORM**, permette di definire le relazioni direttamente nelle **entity** usando i decorator:

`@OneToOne()`

`@OneToMany()`

`@ManyToOne()`

`@ManyToMany()`

Cosa succede nel Database

students

| id | nome |
|----|------|
| 1 | Luca |

courses

| id | titolo |
|----|--------------|
| 1 | NestJS Base |
| 2 | SQL Advanced |

| student_courses | |
|-----------------|-----------|
| student_id | course_id |
| 1 | 1 |
| 1 | 2 |

👉 Lo studente Luca segue 2 corsi.

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  ManyToMany,
  JoinTable
} from 'typeorm';
```

```
import { Course } from './course.entity';
```

```
@Entity('students')
export class Student {
```

```
  @PrimaryGeneratedColumn()
  id: number;
```

```
  @Column()
  nome: string;
```

```
  @Column()
  email: string;
```

```
  @ManyToMany(() => Course, course => course.students)
```

```
  @JoinTable({
    name: 'student_courses',
    joinColumn: {
      name: 'student_id',
      referencedColumnName: 'id'
    },
    inverseJoinColumn: {
      name: 'course_id',
      referencedColumnName: 'id'
    }
  })
  courses: Course[];
}
```

- import {
- Entity,
- PrimaryGeneratedColumn,
- Column,
- ManyToMany
- } from 'typeorm';
- import { Student } from './student.entity';
- @Entity('courses')
- export class Course {
- @PrimaryGeneratedColumn()
- id: number;
- @Column()
- titolo: string;
- @Column()
- durata: number;
- @ManyToMany(() => Student, student => student.courses)
- students: Student[];
- }

Relazione ManyToMany senza campi nella PIVOT

```
product.entity.ts
import { Entity, PrimaryGeneratedColumn, Column, ManyToMany,
  JoinTable } from 'typeorm';
import { Category } from '../categories/entities/category.entity';

@Entity()
export class Product {

  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  name: string;

  @ManyToMany(() => Category, (category) => category.products)
  @JoinTable({
    name: 'product_category',
    joinColumn: { name: 'product_id', referencedColumnName: 'id' },
    inverseJoinColumn: { name: 'category_id', referencedColumnName:
      'id' },
  })
  categories!: Category[];
}
```

importare ManyToMany e la tabella relazionata

definire la relazione

Il "lato owner" è quello che "possiede" la relazione, ovvero chi ha @JoinTable.
L'altro lato è il lato "inverso" e ha solo @ManyToMany con il riferimento all'altro.

| Product (owner) | DietaryPreference (inverso) |
|----------------------|-----------------------------|
| @ManyToMany(...) | @ManyToMany(...) |
| @JoinTable(...) | // niente @JoinTable |
| dietaryPreferences[] | products[] |

La tabella pivot product_dietary_preference viene letta da entrambi ma è definita una volta sola su Product.

Come scegliere il lato owner? Convenzionalmente è l'entità più "principale" della relazione — nel tuo caso Product è il centro del dominio, quindi è naturale che sia lui a dichiarare la pivot. Funzionerebbe anche al contrario, ma è meno intuitivo.

Relazione ManyToMany con campi nella PIVOT (entity esterna)

Avendo un campo nella pivot **non puoi usare @ManyToMany** — serve una entity esplicita con @ManyToOne su entrambi i lati.

creare entity

src\products\entities\product-ingredient.entity.ts

product-ingredient.entity.ts

```
@Entity("product_ingredient")
```

```
export class ProductIngredient {
```

```
  @PrimaryColumn({ type: "bigint" })
```

```
  product_id!: number;
```

```
  @PrimaryColumn({ type: "bigint" })
```

```
  ingredient_id!: number;
```

```
  @Column({ type: "tinyint", nullable: true })
```

```
  is_optional?: boolean;
```

```
  @ManyToOne(() => Product, (product) => product.productIngredients)
```

```
  @JoinColumn({ name: "product_id" })
```

```
  product!: Product;
```

```
  @ManyToOne(() => Ingredient, (ingredient) =>
```

```
    ingredient.productIngredients)
```

```
  @JoinColumn({ name: "ingredient_id" })
```

```
  ingredient!: Ingredient;
```

```
}
```

gestire le 2 relazioni @oneToMany

product.entity.ts

```
@OneToOne(() => ProductIngredient, (pi) => pi.product) productIngredients!:  
ProductIngredient[];
```

ingredient.entity.ts

```
@OneToOne(() => ProductIngredient, (pi) => pi.ingredient) productIngredients!:  
ProductIngredient[];
```

progetto database_mysql_typeORM

- **Roadmap del progetto**
- Creazione progetto NestJS
- Installazione MySQL + TypeORM
- Configurazione .env
- Configurazione ConfigModule
- Configurazione TypeOrmModule
- Generazione resource
- Creazione Entity
- Relazioni OneToMany / ManyToOne
- Configurazione migrations
- Generazione migrations
- Esecuzione migrations
- Creazione DTO
- Repository Pattern
- CRUD Categories
- CRUD Posts
- Relazioni tra tabelle
- Swagger API Documentation
- Test API REST

progetto database_mysql_typeORM

```
nest new database_mysql_typeorm
cd database_mysql_typeorm
npm install @nestjs/typeorm typeorm
mysql2
npm install @nestjs/config
npm install @nestjs/swagger swagger-
  ui-express
npm i dotenv
npm install typeorm-ts-node-commonjs
  --save-dev
npm install --save-dev ts-node tsconfig-
  paths
```

```
nest g resources posts
nest g resources categories
```

```
https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/database\_mysql\_typeORM
```

main.ts

```
import dotenv from
  'dotenv';
dotenv.config();
```

o in alternativa

```
ConfigModule.forRoot({
  envFilePath: '.env',
}),
```

**importo ConfigModule e TypeOrmModule
app.module.ts**

```
...
import { ConfigModule } from '@nestjs/config';
import { TypeOrmModule } from
  '@nestjs/typeorm/dist/typeorm.module';
```

```
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
      envFilePath: '.env',
    }),
    TypeOrmModule.forRootAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        type: 'mysql',
        host: config.get<string>('DB_HOST'),
        port: config.get<number>('DB_PORT'),
        username: config.get<string>('DB_USER'),
        password: config.get<string>('DB_PASSWORD'),
        database: config.get<string>('DB_NAME'),
        autoLoadEntities: true,
        synchronize: false,
      }),
    ),
  ],
})
```

migrations in scripts package.json

- "scripts": {
- "build": "nest build",
- ...
- ...
- "typeorm": "node -r ts-node/register -r tsconfig-paths/register ./node_modules/typeorm/cli.js",
- "migration:generate": "npm run typeorm -- migration:generate ./src/migrations/Init -d ./src/data-source.ts",
- "migration:run": "npm run typeorm -- migration:run -d ./src/data-source.ts",
- "migration:revert": "npm run typeorm -- migration:revert -d ./src/data-source.ts"

creiamo le entities (le tabelle di typeORM)

categories/category.entity.ts

```
import { Post } from '../posts/entities/post.entity';
import { Column, Entity, OneToMany, PrimaryGeneratedColumn } from 'typeorm';
```

```
@Entity('categories')
export class Category {
  @PrimaryGeneratedColumn()
  id!: number;
```

```
  @Column({ length: 255 })
  name!: string;
```

```
  @Column({ type: 'text', nullable: true })
  description?: string;
```

```
  @Column({ default: true })
  active: boolean = true;
```

```
  @OneToMany(() => Post, post => post.category)
  posts!: Post[];
}
```

posts/post.entity.ts

```
import { Category } from '../categories/entities/category.entity';
import {
  Column,
  Entity,
  JoinColumn,
  ManyToOne,
  PrimaryGeneratedColumn,
} from 'typeorm';
```

```
@Entity('posts')
export class Post {
  @PrimaryGeneratedColumn()
  id!: number;
```

```
  @Column({ length: 255 })
  title!: string;
```

```
  @Column({ length: 255, unique: true })
  slug!: string;
```

```
  @Column({ type: 'text', nullable: true })
  content?: string;
```

```
  @Column({ default: true })
  active: boolean = true;
```

```
  @ManyToOne(() => Category, category => category.posts, {
    nullable: true,
    onDelete: 'SET NULL',
  })
  @JoinColumn({ name: 'category_id' })
  category!: Category;
}
```

registriamo le entities nei moduli

categories.module.ts

```
import { Module } from '@nestjs/common';
import { CategoriesService } from './categories.service';
import { CategoriesController } from './categories.controller';
import { Category } from './entities/category.entity';
import { TypeOrmModule } from '@nestjs/typeorm/dist/typeorm.module';
```

```
@Module({
  imports: [TypeOrmModule.forFeature([Category])],
  controllers: [CategoriesController],
  providers: [CategoriesService],
})
export class CategoriesModule {}
```

posts.module.ts

```
import { Module, Post } from '@nestjs/common';
import { PostsService } from './posts.service';
import { PostsController } from './posts.controller';
import { TypeOrmModule } from '@nestjs/typeorm/dist/typeorm.module';
```

```
@Module({
  imports: [
    TypeOrmModule.forFeature([Post])
  ],
  controllers: [PostsController],
  providers: [PostsService],
})
export class PostsModule {}
```

migrations

obiettivo: creare le tabelle posts e categories create da TypeORM.

```
npm run migration:generate
```

```
npm run migration:run
```

```
PS C:\Corsi\ITS Turismo Marche\2025-2027 Fullstack
Senigallia\docenza_backend\esempi\database_mysql_type-orm> npm run
migration:generate
```

```
> database_mysql_type-orm@0.0.1 migration:generate
> npm run typeorm -- migration:generate ./src/migrations/Init -d ./src/data-source.ts
```

```
> database_mysql_type-orm@0.0.1 typeorm
> node -r ts-node/register -r tsconfig-paths/register ./node_modules/typeorm/cli.js
migration:generate ./src/migrations/Init -d ./src/data-source.ts
```

```
Migration C:\Corsi\ITS Turismo Marche\2025-2027 Fullstack
Senigallia\docenza_backend\esempi\database_mysql_type-orm\src\migrations\1779630249263-Init.ts has been generated successfully.
```

```
PS C:\Corsi\ITS Turismo Marche\2025-2027 Fullstack
Senigallia\docenza_backend\esempi\database_mysql_type-orm>
```

Nota verificare di avere inserito in scripts

```
"scripts": {
```

```
  "build": "nest build",
```

```
...
```

```
...
```

```
  "typeorm": "node -r ts-node/register -r tsconfig-paths/register ./node_modules/typeorm/cli.js",
```

```
  "migration:generate": "npm run typeorm -- migration:generate ./src/migrations/Init -d ./src/data-source.ts",
```

```
  "migration:run": "npm run typeorm -- migration:run -d ./src/data-source.ts",
```

```
  "migration:revert": "npm run typeorm -- migration:revert -d ./src/data-source.ts"
```

```
0 migrations are already loaded in the database.
1 migrations were found in the source code.
1 migrations are new migrations must be executed.
query: START TRANSACTION
query: CREATE TABLE `categories` (`id` int NOT NULL AUTO_INCREMENT, `name` varchar(255) NOT NULL, `description` text NULL, `active` tinyint NOT NULL DEFAULT 1, PRIMARY KEY (`id`)) ENGINE=InnoDB
query: CREATE TABLE `posts` (`id` int NOT NULL AUTO_INCREMENT, `title` varchar(255) NOT NULL, `slug` varchar(255) NOT NULL, `content` text NULL, `active` tinyint NOT NULL DEFAULT 1, `category_id` int NULL, UNIQUE INDEX `IDX_54ddf9075260407dcfdd724857` (`slug`), PRIMARY KEY (`id`)) ENGINE=InnoDB
query: ALTER TABLE `posts` ADD CONSTRAINT `FK_852f266adc5d67c40405c887b49` FOREIGN KEY (`category_id`) REFERENCES `categories` (`id`) ON DELETE SET NULL ON UPDATE NO ACTION
query: INSERT INTO `corso_blogdb_typeorm`.`migrations` (`timestamp`, `name`) VALUES (?, ?) -- PARAMETERS: [1779630249263,"Init1779630249263"]
Migration Init1779630249263 has been executed successfully.
query: COMMIT
```

CRUD categories

```
// categories.service.ts
constructor(
  @InjectRepository(Category)
  private readonly
  categoryRepository:
  Repository<Category>,
) {}
```

```
import { Injectable, NotFoundException } from
 '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Category } from './entities/category.entity';
import { Repository } from 'typeorm';
import { CreateCategoryDto } from './dto/create-
category.dto';
import { UpdateCategoryDto } from './dto/update-
category.dto';
```

```
@Injectable()
export class CategoriesService {
  constructor(
    @InjectRepository(Category)
    private readonly categoryRepository:
    Repository<Category>,
  ) {}
```

```
  findAll() {
    return this.categoryRepository.find({
      where: {
        active: true,
      },
    });
  }
```

```
  async findOne(id: number) {
    const category = await
    this.categoryRepository.findOne({
      where: { id },
      relations: {
```

```
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Category } from './entities/category.entity';
import { Repository } from 'typeorm';
import { CreateCategoryDto } from './dto/create-category.dto';
import { UpdateCategoryDto } from './dto/update-category.dto';
```

```
@Injectable()
export class CategoriesService {
  constructor(
    @InjectRepository(Category)
    private readonly categoryRepository: Repository<Category>,
  ) {}
```

```
  findAll() {
    return this.categoryRepository.find({
      where: {
        active: true,
      },
    });
  }
```

```
  async findOne(id: number) {
    const category = await this.categoryRepository.findOne({
      where: { id },
      relations: {
        posts: true,
      },
    });
```

```
    if (!category) {
      throw new NotFoundException(`Category con id ${id} non
      trovata`);
    }
```

```
    return category;
  }
```

```
  create(createCategoryDto: CreateCategoryDto) {
```

CRUD posts

- **// posts.service.ts**
constructor(
 @InjectRepository(Post)
 private readonly postRepository: Repository<Post>,

 @InjectRepository(Category)
 private readonly categoryRepository: Repository<Category>,
) {}

swagger per il test delle api

main.ts

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import dotenv from 'dotenv';
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';

dotenv.config();

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  const config = new DocumentBuilder()
    .setTitle('Database MySQL API')
    .setDescription('API for managing posts with MySQL database')
    .setVersion('1.0')
    .addTag('posts')
    .build();
  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);

  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

src/data-source.ts (per eseguire le migrations)

```
import 'dotenv/config';
import { DataSource } from 'typeorm';
import { Post } from './posts/entities/post.entity';
import { Category } from './categories/entities/category.entity';

export default new DataSource({
  type: 'mysql',
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT),
  username: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  entities: [Post, Category],
  migrations: ['src/migrations/*.ts'],
  synchronize: false,
});
```

package.json

```
{
  "scripts": {
    "typeorm": "typeorm-ts-node-commonjs",
    "migration:generate": "npm run typeorm -- migration:generate src/migrations/Init -d src/data-source.ts",
    "migration:run": "npm run typeorm -- migration:run -d src/data-source.ts",
    "migration:revert": "npm run typeorm -- migration:revert -d src/data-source.ts"
  }
}
```

TypeORM carica le migrations dalla proprietà `migrations` della `DataSource`

approfondimento: decorator personalizzato es IsSlug

È una funzione che definisce una **regola di validazione riutilizzabile** (es. validare uno *slug*)

👉 Trasforma una regola tecnica (regex) in qualcosa di leggibile

❌ Senza decorator custom

@Matches(/^[-a-z0-9-]+\$/)

slug: string;

👎 Poco leggibile

📄 Ripetuto in più punti

🔧 Difficile da modificare

✅ Con decorator custom

@IsSlug()

slug: string;

✓ Più chiaro

✓ Riutilizzabile

✓ Manutenibile

🎯 Vantaggi principali

✅ Leggibilità

Il codice è più comprensibile (anche per altri sviluppatori)

✅ Riutilizzo

Scrivi una volta, usi ovunque

✅ Manutenzione

Modifichi una sola volta la regola

✅ Logica di business

"Slug" diventa un concetto esplicito nel codice

nest g d common/decorators/is-slug

```
import { registerDecorator, ValidationOptions } from 'class-validator';
```

```
export function IsSlug(validationOptions?: ValidationOptions) {  
  return function (object: object, propertyName: string) {  
    registerDecorator({  
      name: 'isSlug',  
      target: object.constructor,  
      propertyName,  
      options: validationOptions,  
      validator: {  
        validate(value: unknown) {  
          return typeof value === 'string' && /^[a-z0-9]+(?:-[a-z0-9]+)*$/i.test(value);  
        },  
        defaultMessage() {  
          return `${propertyName} deve essere uno slug valido (es. "mio-slug-123")`;  
        },  
      },  
    });  
  };  
}
```

Code Quality & ESLint

CODICE LEGGIBILE, COERENTE, AFFIDABILE NEL TEMPO

Cos'è un linter

Strumento di **analisi statica del codice** (analizza il codice senza eseguirlo) che individua:

Errori potenziali

Bad practice

Incoerenze stilistiche

Lavora **prima dell'esecuzione del codice**, quindi:

Intercetta **problemi prima che diventino bug**

Aiuta a scrivere codice più chiaro e manutenibile

Non giudica se il codice funziona, ma se è scritto bene ed in modo sicuro.

```
// Esempio di codice senza Linter (Static Analysis Error)
function calculate(total) {
  if (total = 100) { // Errore: assegnazione invece di confronto
    console.log("Sconto applicato");
  }
  return total * 1.22
}
```

```
46:10 error 'calculate' is defined but never used @typescript-eslint/no-unused-vars
47:7 error Expected a conditional expression and instead saw an assignment no-cond-assign
```

ESLint cosa corregge

- **Formattazione**
 - punto e virgola mancante o in eccesso
 - virgolette singole vs doppie
 - indentazione sbagliata
 - spazi attorno agli operatori
- **Import**
 - ordinamento degli import
 - rimozione di righe vuote tra import
- **Stile codice**
 - trailing comma mancante
 - parentesi inutili
 - var → let o const

ESLint in NestJS

NestJS utilizza ESLint già configurato nel progetto creato

`npm run lint`

Cosa controlla?

Esempi:

`const test = 'ciao'`

✗ manca ;

`if(value == 1)`

✗ uso di == invece di ===

`unusedVariable`

✗ variabile inutilizzata

Prettier in NestJS

Prettier è un formatter automatico del codice.

Non controlla errori logici:

si occupa solo della formattazione.

In NestJS

NestJS integra Prettier automaticamente.

```
npm run format
```

Esempio

Codice scritto male:

```
const name="Mario«
```

Dopo Prettier:

```
const name = 'Mario';
```

Cosa formatta?

- indentazione
- spazi
- virgolette
- punti e virgola
- ritorni a capo
- lunghezza righe

Static Analysis

La **static analysis** analizza il codice:
senza eseguirlo

leggendo struttura, tipi, flussi

Vantaggi:

intercetta bug logici

migliora sicurezza

riduce errori runtime

ESLint è uno strumento di **static analysis**,
non di testing.

COME FUNZIONA

1. **Lettura** del codice .ts
2. **Creazione** dell'**AST**: codice convertito in una struttura ad albero (Abstract Syntax Tree) per capirne la logica.
3. **Verifica** delle regole: AST confrontato con set di regole (es: "non usare any", "gestisci sempre le Promise")

| PROBLEMA | ESEMPIO | PERCHÉ È UN RISCHIO NEL BACKEND |
|--------------------|--|--|
| Bug Potenziali | Variabili non definite o assegnazioni errate (es. <code>if (x = 10)</code>). | Causano logiche di business errate o crash improvvisi . |
| Pattern Pericolosi | Promise "appese" (floating promises) senza await . | Il server potrebbe non rispondere o lasciare operazioni DB a metà . |
| Debito Tecnico | Variabili o import dichiarati e mai utilizzati . | Appesantisce il codice e rende difficile la manutenzione per il team. |
| Violazioni di Tipo | Tentativo di usare un metodo stringa su un numero (grazie a TypeScript). | È la causa principale degli errori Runtime nel backend. |

prima della consegna

Flusso consigliato prima di consegnare:

`npm run format`

`npm run lint`

`npm run test`

`npm run build`

esempio pratico

npm run lint # controlla tutti i file npm run lint -- --fix #
controlla e corregge automaticamente dove può

npm run lint

npm run lint --fix

nessun output significa nessun problema — ESLint ha trovato
tutto a posto

void bootstrap();

```
import {  
  Column,  
  Entity,  
  // OneToMany,  
  JoinColumn,  
  PrimaryGeneratedColumn,  
  ManyToOne,  
} from 'typeorm';
```

```
> nest-postgresql-onetomany@0.0.1 lint  
> eslint "{src,apps,libs,test}/**/*.ts" --fix
```

```
C:\Software\nest.js\corso-repository\esercizi\7-nest-postgresql-  
onetomany\src\main.ts
```

```
16:1 warning Promises must be awaited, end with a call to .catch, end  
with a call to .then with a rejection handler or be explicitly marked as  
ignored with the `void` operator @typescript-eslint/no-floating-  
promises
```

```
C:\Software\nest.js\corso-repository\esercizi\7-nest-postgresql-  
onetomany\src\posts\entities\post.entity.ts
```

```
4:3 error 'OneToMany' is defined but never used @typescript-  
eslint/no-unused-vars
```

✖ 2 problems (1 error, 1 warning)

```
PS C:\Software\nest.js\corso-repository\esercizi\7-nest-postgresql-  
onetomany>
```

ESEMPI SEGNALAZIONI ESLINT

/Funzione di connessione al DB con connection pooling

```
const connectDB = async () => {
```

```
  try{
```

Unexpected console statement. Only these console methods are allowed: warn, error. eslint([no-console](#))

```
    // S  
    var console: Console
```

```
    if (
```

View Problem (Alt+F8) Quick Fix... (Ctrl+.)

```
      console.log("MongoDB già connesso");
```

```
      return;
```

```
    }
```

```
    await mongoose.connect(process.env.MONGO_URI!,
```

```
  )
```

```
  }
```

```
}
```

'main' is declared but its value is never read. ts(6133)

```
async function main() {
```

'main' is defined but never used. eslint([@typescript-eslint/no-unused-vars](#))

```
  return
```

```
}
```

function main(): void

View Problem (Alt+F8) Quick Fix... (Ctrl+.)

```
function main() {
```

```
  // 3. Errore segnalato: @typescript-eslint/no-floating-promises
```

```
  // Dimenticare 'await' nel backend è pericoloso: l'operazione potrebbe fallire silenziosamente.
```

```
  saveToDatabase();
```

```
}
```


ESLint vs Prettier

Entrambi lavorano su codice **ma ESLint si occupa della Qualità** (Logica), **Prettier si occupa della Forma** (Estetica).

Non sono alternativi uno all'altro ma **complementari**: ESLint assicura che il backend non crashi per errori banali; Prettier assicura che, se più programmatori lavorano allo stesso progetto, questo sembri scritto da una sola persona.

| CARATTERISTICA | ESLINT | PRETTIER |
|------------------------|---|--|
| Scopo Principale | Trovare errori di logica e stile. | Formattazione automatica e consistente. |
| Cosa controlla | Bug logici : variabili non definite, codice irraggiungibile;
Best practice : evitare uso di eval(), obbligare uso di const invece di var
Regole Typescript : verificare gestione Promise | Spaziatura : indentazione;
Lunghezza riga : va a capo automaticamente se una riga supera 100 caratteri;
Stile dei caratteri : uso dei punti e virgola, apici singoli o doppi. |
| Capacità di correzione | Può suggerire correzioni | Corregge il 100% dei problemi di stile riscrivendo il codice per renderlo conforme a regole stabilite. |
| Conoscenza del Codice | Capisce come il codice "gira" (AST). | Vede il codice come testo da impaginare. |
| Esempio di Regola | no-unused-vars (variabile inutilizzata). | semi: true (aggiungi sempre il ;). |

Autenticazione & JWT

Cosa è JWT

JWT significa:

JSON Web Token

È uno standard utilizzato per rappresentare informazioni in modo sicuro e compatto.

Viene spesso utilizzato per:

Autenticazione

Autorizzazione

Scambio di informazioni tra sistemi

HTTP è stateless → il server non sa chi sei

JWT (**JSON Web Token**) è un modo per **identificare un utente senza salvare sessioni sul server**.

👉 In pratica è un “**badge digitale**” che il server ti dà dopo il login.

🧠 **Spiegazione semplice STATELESS**

L'utente fa **login**

Il server **verifica username/password**

Il server crea un **token (JWT)**

Il **client lo salva**

Il client **lo manda ad ogni richiesta**

👉 Il server **NON** deve ricordarsi nulla

Il token è la tua identità

La guard la verifica

Perché usare JWT?

- Dopo il login il server deve riconoscere l'utente.
- Senza JWT:
 - Login
 - ↓
 - Server salva sessione
 - ↓
 - Ad ogni richiesta cerca la sessione
- Con JWT:
 - Login
 - ↓
 - Server genera JWT
 - ↓
 - Client invia JWT ad ogni richiesta
 - ↓
 - Server verifica il JWT
- Il server non deve mantenere una sessione.

Struttura di un JWT

HEADER.PAYLOAD.SIGNATURE

Header: contiene info tecniche:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload (parte importante)

```
{  
  "username": "admin",  
  "role": "editor"  
}
```

NON è criptato ❌ ma è solo codificato (base64)

Signature (firma)

Serve per garantire che il token **non sia stato modificato**

👉 creata con:
payload + secret key

il server firma il token

il client NON può modificarlo

il server verifica la firma

👉 Se cambi anche 1 carattere → token invalido

Cosa contiene di solito un JWT

id utente

username

ruolo

scadenza (exp)

struttura JWT

JWT NON è cifrato

Chi riceve il token può leggerlo.

👉 quindi:

- ✗ NON mettere password
- ✗ NON mettere dati sensibili

JWT vs Sessioni (importantissimo)

JWT

Stateless

Non salva dati server

Scalabile

Usato per API

Sessione

Stateful

Salva dati server

Meno scalabile

Usato per web classico

stateless vs stateful

Differenza fondamentale

👉 stateless JWT:

“Mi fido del token”

👉 stateful:

“Controllo nel database”

JWT e Passport

****Passport.js**** è una libreria per gestire l'autenticazione.

👉 NestJS la usa tramite:

@nestjs/passport

A cosa serve

- ✓ Gestire login
- ✓ Validare utenti
- ✓ Supportare strategie (JWT, Google, Facebook...)

In NestJS

Passport è il “motore”

JWT è il “metodo di autenticazione”

JWT + Passport – Perché usarli

🚀 **Vantaggi**

- ✓ Sicurezza API
 - ✓ Stateless (scalabile)
 - ✓ Standard moderno
 - ✓ Facile integrazione frontend

JWT in generale

JWT è un token firmato che il client invia ad ogni richiesta.

Flusso:

Login → ricevo token

Richiesta API → Authorization: Bearer <token>

NestJS verifica il token

Se valido → request.user

Autenticazione vs Autorizzazione

Autenticazione = chi sei?

Autorizzazione = cosa puoi fare?

Esempio:

Token valido → sei autenticato

Role.Admin → puoi fare DELETE

Role.User → non puoi fare DELETE

NestJS separa questi concetti usando i **Guard**

Autenticazione – React + NestJS HttpOnly vs Local Storage

- Ci sono 2 metodi:
- Metodo più semplice:
JWT in Local Storage: JWT nel JSON + Authorization Bearer

Metodo più sicuro per web app:
JWT in cookie HttpOnly

- Con JWT HttpOnly, React non vede mai il token: lo riceve e lo reinvia automaticamente il browser.
- Nota:
 - Il cookie HttpOnly è utile quando c'è un **browser**
 - Se chiamata da server a server (esempio da command a backend non entra in gioco browser e non centrano i CORS)

Differenza con JWT nel localStorage

Cookie HttpOnly (più sicuro)

React
↓
POST login
↓
Cookie salvato dal browser
↓
Cookie inviato automaticamente

Il JavaScript del frontend NON può leggere il token.
`localStorage.getItem(...)`
non funziona.

LocalStorage (meno sicuro)

POST login
↓
Backend restituisce JWT
↓
React salva in localStorage
↓
React lo aggiunge manualmente
`localStorage.setItem('token', token);`

JWT in Local Storage

<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/app-jwt-backend-frontend-local-storage>

JWT restituito nel JSON

NestJS risponde così:

```
{  
  "access_token": "eyJhbGciOiJIUzI1..."  
}
```

React salva il token, ad esempio in memoria o localStorage, e poi lo invia:

```
fetch('http://localhost:3000/profile', {  
  headers: {  
    Authorization: `Bearer ${token}`,  
  },  
});
```

Schema:

```
Login  
↓  
NestJS ritorna JWT nel JSON  
↓  
React salva JWT  
↓  
React lo manda con Authorization Bearer
```

Vantaggi:

Semplice

Molto usato nelle SPA

Comodo anche per mobile app

Svantaggio:

Se salvato in localStorage è più esposto a XSS

JWT in cookie HttpOnly

<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/app-jwt-backend-frontend-httponly>

NestJS non ritorna il JWT nel JSON, ma lo mette in un cookie:

Set-Cookie: access_token=JWT; HttpOnly; SameSite=Lax

React non può leggere il cookie.

React fa solo:

```
fetch('http://localhost:3000/profile', {  
  credentials: 'include',  
});
```

Attenzioni:

Serve configurare CORS con credentials: true

Serve gestire SameSite / Secure

Serve protezione CSRF se necessario

Schema:

Login



NestJS crea JWT



NestJS lo salva in cookie HttpOnly



Browser salva il cookie



React fa chiamate con credentials: include



Browser invia automaticamente il cookie

Vantaggi:

Il token non è leggibile da JavaScript

Migliore protezione contro furto token via XSS

Schema Completo HttpOnly

React
| invia email + password
▼
NestJS
| verifica utente
▼
Database
| genera JWT
▼
NestJS risponde con header:
Set-Cookie: access_token=JWT; HttpOnly; SameSite=Lax
▼
Browser salva il cookie
▼
React NON legge il token
▼
Nelle richieste successive React usa:
credentials: 'include'
▼
Il browser invia automaticamente:
Cookie: access_token=JWT
▼
NestJS legge il cookie
▼
Passport valida JWT
▼
req.user = utente autenticato

Per applicazioni React + NestJS moderne, oggi si vedono principalmente due approcci:
JWT in Authorization Bearer (molto comune per SPA e mobile)
JWT in cookie HttpOnly (considerato più sicuro contro attacchi XSS)

```
@Post('login')
async login(
  @Body() dto: LoginDto,
  @Res({ passthrough: true }) res: Response,
) {
  const result = await this.authService.login(dto);

  res.cookie('access_token', result.access_token, {
    httpOnly: true,
    secure: false, // true in HTTPS/produzione
    sameSite: 'lax',
    maxAge: 1000 * 60 * 60,
  });

  return {
    message: 'Login effettuato',
  };
}
```

React **non legge il cookie HttpOnly**.

Fa solo la chiamata con:

credentials: 'include'

Esempio fetch

```
fetch('http://localhost:3000/profile', {
  method: 'GET',
  credentials: 'include',
});
```

Cosa significa credentials: true

Quando parliamo di **credenziali** nei CORS intendiamo dati “sensibili” che il browser può inviare insieme alla richiesta, ad esempio:

- Cookie
- Header Authorization
- Certificati client

Nel caso più comune: **cookie di sessione o cookie JWT**.

Esempio senza credenziali

React chiama NestJS:

```
fetch('http://localhost:3000/profile');
```

Il browser fa la richiesta, ma **non invia automaticamente i cookie** cross-origin.

Quindi NestJS potrebbe rispondere:

Non sei autenticato

Per dire al browser di inviare anche i cookie:

```
– fetch('http://localhost:3000/profile', {  
  credentials: 'include',  
});
```

oppure con axios

```
– axios.get('http://localhost:3000/profile', {  
  withCredentials: true,  
});
```

Anche il backend deve autorizzare le credenziali:

```
– app.enableCors({  
  origin: 'http://localhost:5173',  
  credentials: true,  
});
```

Importante:

```
– origin: '*'
```

non va bene insieme a:

```
– credentials: true
```

Devi indicare l’origine precisa.

Credenziali e Cookie

Se si utilizzano:

Cookie

Sessioni

Autenticazione

bisogna abilitare:

```
app.enableCors({  
  origin: 'http://localhost:4200',  
  credentials: true,  
});
```

Il browser aggiungerà:

Access-Control-Allow-Credentials: true

Esempio login con cookie

Login

React invia username e password:

```
fetch('http://localhost:3000/auth/login', {  
  method: 'POST',  
  credentials: 'include',  
  headers: {  
    'Content-Type': 'application/json',  
  },  
  body: JSON.stringify({  
    email: 'mario@test.it',  
    password: '123456',  
  }),  
});
```

Richiesta HTTP:

POST /auth/login

```
{  
  "email": "mario@test.it",  
  "password": "123456"  
}
```


Backend - Verifica

NestJS verifica le credenziali

Database



Utente trovato?



Password corretta?



Genero JWT

restituisce il cookie

```
@Post('login')
async login(
  @Body() dto: LoginDto,
  @Res({ passthrough: true }) res: Response,
) {

  const token = this.authService.generateJwt(dto);

  res.cookie('access_token', token, {
    httpOnly: true,
    secure: false,
    sameSite: 'lax',
    maxAge: 86400000,
  });

  return {
    message: 'Login effettuato',
  };
}
```

Risposta HTTP:

Set-Cookie:

access_token=eyJhbGciOiJIUzI1NiIs...

Browser salva il cookie + chiamata successiva

In Chrome:

F12

Application

Cookies

http://localhost:3000

Vedrai:

| Name | Value |
|--------------|-------------------------------|
| access_token | eyJhbGciOiJIUzI1NiIsInR5cGU6I |

Chiamata successiva

```
fetch('http://localhost:3000/profile', {  
  credentials: 'include',  
});
```

Tu NON aggiungi manualmente il token.
Il browser aggiunge automaticamente:
GET /profile

Cookie:

access_token=eyJhbGciOiJIUzI1NiIsInR5cGU6I

Backend **NestJS** legge il cookie

Ad esempio:

```
@Get('profile')  
profile(@Req() req: Request) {  
  
  const token =  
    req.cookies.access_token;  
  
  return this.authService.verify(token);  
}
```

JWT con Refresh Token

versione token in «local storage»

JWT – oggetti coinvolti

<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/app-jwt-refresh-token-localstorage>

Backend NestJS: 9 file
Frontend HTML/JS: 1 file
Configurazione: .env

Oggetto creato

AppModule

.env

AuthModule

AuthService

AuthController

JwtStrategy

JwtAuthGuard

PostsModule

PostsService

PostsController

@UseGuards (JwtAuthGuard)

Punto saliente

Carica ConfigModule globalmente e collega AuthModule e PostsModule.

Contiene configurazioni esterne, come JWT_SECRET e durata del token.

Configura JwtModule usando ConfigService.

Verifica username/password e genera il JWT + token e refresh token.

Espone la rotta POST /auth/login.

Decide dove leggere il token e come validarlo.

Blocca le rotte se il JWT manca o non è valido.

Raggruppa controller e service dei post.

Gestisce i post in un array statico.

Espone le rotte CRUD dei post.

Rende protette POST, PUT e DELETE.

```
app-config-service-jwt/  
├── .env  
├── src/  
│   ├── app.module.ts  
│   ├── auth/  
│   │   ├── auth.module.ts  
│   │   ├── auth.controller.ts  
│   │   ├── auth.service.ts  
│   │   ├── guards/  
│   │   │   └── jwt-auth.guard.ts  
│   │   └── strategies/  
│   │       └── jwt.strategy.ts  
│   └── posts/  
│       ├── posts.module.ts  
│       ├── posts.controller.ts  
│       └── posts.service.ts  
└── frontend/  
    └── index.html
```

Installazione

- **Obiettivo:** creare una mini API NestJS con autenticazione JWT e refresh token
- nest new app-posts-jwt
- cd app-posts-jwt
- npm install @nestjs/config @nestjs/jwt @nestjs/passport passport passport-jwt
- npm install -D @types/passport-jwt

.env

Le configurazioni sensibili non vanno scritte direttamente nel codice

access_token serve per accedere alle API

refresh_token serve solo per ottenere un nuovo access_token

refresh_token deve durare di più

in un progetto reale andrebbe salvato/validato anche lato server

quando il token va in 401, viene fatta nuova chiamata con refresh token e salvato il token generato (nuovo)

```
JWT_SECRET=super_secret_key
```

```
JWT_EXPIRES_IN=15m
```

```
JWT_REFRESH_SECRET=super_refresh_secret_key
```

```
JWT_REFRESH_EXPIRES_IN=7d
```

app.module.ts

app.module.ts è il **modulo principale** dell'app NestJS.
È il punto in cui Nest collega i vari pezzi dell'applicazione.

in questo caso configura JWT leggendo i valori dal file .env.
con ConfigService

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { PostsModule } from './posts/posts.module';
import { AuthModule } from './auth/auth.module';
import { ConfigModule } from '@nestjs/config';
```

```
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true
    }),
    PostsModule,
    AuthModule
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

auth.module.ts

AuthModule è il modulo che raggruppa tutto ciò che riguarda l'autenticazione.

Serve a collegare:

- AuthController
- AuthService
- JwtModule
- PassportModule
- JwtStrategy

all'interno il JwtModule principale firma l'**access token**.

```
import { StringValue } from 'ms';

@Module({
  imports: [
    PassportModule,
    JwtModule.registerAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        secret: config.get<string>('JWT_SECRET'),
        signOptions: {
          expiresIn: config.get<StringValue>('JWT_EXPIRES_IN') ?? '1h',
        },
      }),
    ],
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy],
})
```


auth.service.ts

AuthService contiene la logica di autenticazione.
È il componente che verifica le credenziali e genera i token JWT.

Responsabilità

si occupa di:

- ✓ verificare username e password
- ✓ generare access token
- ✓ generare refresh token
- ✓ rigenerare i token tramite refresh token

```
login(username: string, password: string) {
  const user = this.users.find(
    (u) => u.username === username &&
    u.password === password,
  );
  if (!user) {
    throw new
    UnauthorizedException('Credenziali non
    valide');
  }
  return this.generateTokens(user.id,
    user.username);
}
```

```
private generateTokens(userId: number, username: string) {
  const payload = {
    sub: userId,
    username,
  };

  const accessToken = this.jwtService.sign(payload, {
    secret: this.configService.get<StringValue>('JWT_SECRET'),
    expiresIn: this.configService.get<StringValue>('JWT_EXPIRES_IN'),
  });

  const refreshToken = this.jwtService.sign(payload, {
    secret: this.configService.get<StringValue>('JWT_REFRESH_SECRET'),
    expiresIn: this.configService.get<StringValue>('JWT_REFRESH_EXPIRES_IN'),
  });

  return {
    access_token: accessToken,
    refresh_token: refreshToken,
  };
}
```

```
refresh(refreshToken: string) {
  try {
    const payload = this.jwtService.verify(refreshToken, {
      secret: this.configService.get<string>('JWT_REFRESH_SECRET'),
    });

    return this.generateTokens(payload.sub, payload.username);
  } catch {
    throw new UnauthorizedException('Refresh token non valido');
  }
}
```

auth.controller.ts

AuthController espone le rotte HTTP per l'autenticazione.

A cosa serve?

Nel nostro esempio gestisce:

POST /auth/login

POST /auth/refresh

```
@Controller('auth')
export class AuthController {
  constructor(private authService: AuthService) {}

  @Post('login')
  login(@Body() body: LoginDto) {
    return this.authService.login(body.username,
    body.password);
  }

  @Post('refresh')
  refresh(@Body() body: RefreshTokenDto) {
    return this.authService.refresh(body.refresh_token);
  }
}
```

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlbnVzZXJuYW1lIjoieWVhbnVzZXJuYW1lIiwiaWF0IjE5ODM4NnN0.CRAQUYSy-
  hqSI5FzJTOikHXuG_gcakQAOKj_zVDOcvw",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlbnVzZXJuYW1lIjoieWVhbnVzZXJuYW1lIiwiaWF0IjE5ODM4NnN0.EhbkTw64P1P-
  DtBPNOkRtkbZEq3f7zX4K3SnpGPhA6s"
}
```

jwt.strategy.ts

JwtStrategy definisce **come NestJS deve leggere e validare un JWT.**

A cosa serve?

Serve a dire a NestJS:

dove trovare il token
con quale chiave verificarlo
cosa salvare in req.user

Questa riga legge il token dall'header:

`ExtractJwt.fromAuthHeaderAsBearerToken()`

cioè da:

Authorization: Bearer TOKEN

```
constructor(configService: ConfigService) {  
  super({  
    jwtFromRequest:  
      ExtractJwt.fromAuthHeaderAsBearerToken(),  
    secretOrKey: configService.get<string>('JWT_SECRET'),  
  });  
}  
  
validate(payload: { sub: number; username: string }) {  
  return {  
    userId: payload.sub,  
    username: payload.username,  
  };  
}
```

jwt-auth.guard.ts

jwt-auth.guard.ts

è il **guardiano** delle rotte protette.

A cosa serve?

Serve a bloccare una richiesta se:

- manca il token

- il token è scaduto

- il token non è valido

```
import { Injectable } from '@nestjs/common';  
import { AuthGuard } from '@nestjs/passport';  
  
@Injectable()  
export class JwtAuthGuard extends AuthGuard('jwt') {}
```

posts.module.ts

è il modulo che raggruppa tutto ciò che riguarda i post.

Collega:

PostsController
PostsService

```
import { Module } from '@nestjs/common';
import { PostsController } from '../posts.controller';
import { PostsService } from '../posts.service';

@Module({
  controllers: [PostsController],
  providers: [PostsService]
})
export class PostsModule {}
```

posts.service.ts

PostsService contiene la logica dei post.

A cosa serve?

Gestisce i dati in un array statico:

```
findOne(id: number) {
  const post = this.posts.find((p) => p.id === id);

  if (!post) {
    throw new NotFoundException('Post non trovato');
  }
  return post;
}

create(data: Omit<Post, 'id'>) {
  const newPost: Post = {
    id: Date.now(),
    ...data,
  };
  this.posts.push(newPost);
  return newPost;
}

update(id: number, data: Partial<Omit<Post, 'id'>>) {
  const post = this.findOne(id);
```

```
import { Injectable, NotFoundException } from '@nestjs/common';

export interface Post {
  id: number;
  title: string;
  content: string;
}

@Injectable()
export class PostsService {
  private posts: Post[] = [
    { id: 1, title: 'Primo post', content: 'Contenuto del primo post' },
    { id: 2, title: 'Secondo post', content: 'Contenuto del secondo post' },
  ];

  findAll() {
    return this.posts;
  }
}
```

posts.controller.ts

Cos'è?

PostsController è il controller che espone le API dei post.

È il punto in cui definiamo gli endpoint HTTP:

GET

POST

PUT

DELETE

Riceve le richieste dal client e chiama PostsService.

```
@Get('/:id')
findOne(@Param('id', ParseIntPipe) id: number) {
  return this.postsService.findOne(id);
}

@UseGuards(JwtAuthGuard)
@Post()
create(@Body() body: { title: string; content: string }) {
  return this.postsService.create(body);
}

@UseGuards(JwtAuthGuard)
@Put('/:id')
update(
  @Param('id', ParseIntPipe) id: number,
  @Body() body: { title?: string; content?: string },
) {
  return this.postsService.update(id, body);
}
```

Rotte finali

- Libere:
 - GET /posts
 - GET /posts/:id
 - POST /auth/login
- Protette:
 - POST /posts
 - PUT /posts/:id
 - DELETE /posts/:id
- per le rotte protette serve:
- Authorization: Bearer TOKEN

Richiamo delle rotte index-localstorage.html (salva in local storage)

```
$> npx serve -l 5500
```

login:

```
const response = await fetch(`${API_URL}/auth/login`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    username,
    password
  })
});
```

refresh:

```
const response = await fetch(`${API_URL}/auth/refresh`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    refresh_token: refreshToken
  })
});
```

url protette e refresh access token

```
async function apiFetch(url, options = {}) {
  let response = await fetch(url, {
    ...options,
    headers: {
      ...(options.headers || {}),
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${accessToken}`
    }
  });

  if (response.status === 401) {
    await refreshAccessToken();

    response = await fetch(url, {
      ...options,
      headers: {
        ...(options.headers || {}),
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${accessToken}`
      }
    });
  }
}
```

```
localStorage.setItem('accessToken', accessToken);
localStorage.setItem('refreshToken', refreshToken);
```

JWT con Refresh Token

versione token «HttpOnly»

- Con **HttpOnly** il refresh_token non lo salvi in localStorage: lo salva il browser come cookie, ma JavaScript **non può leggerlo**.

– Flusso:

– login

↓

backend restituisce access_token nel JSON

backend salva refresh_token in cookie HttpOnly

↓

quando access token scade

↓

frontend chiama /auth/refresh con credentials: 'include'

↓

backend legge il cookie e genera nuovo access_token

```
src/  
├── auth/  
│   ├──  
│   ├── auth.controller.ts  
│   │   ├── auth.module.ts  
│   │   ├── auth.service.ts  
│   │   ├── jwt.strategy.ts  
│   │   └── jwt-  
│   └── auth.guard.ts  
│       ├── posts/  
│       │   ├──  
│       │   ├── posts.controller.ts  
│       │   │   └── posts.module.ts  
│       ├── app.module.ts  
│       ├── app.controller.ts  
│       ├── app.service.ts  
│       └── main.ts
```

JWT versione HttpOnly

<https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/app-jwt-refresh-token-httponly>

- installazione pacchetti aggiuntivi:
 - npm install cookie-parser
 - npm install -D @types/cookie-parser

```
app-config-service-jwt/  
├── .env  
├── src/  
│   ├── app.module.ts  
│   ├── auth/  
│   │   ├── auth.module.ts  
│   │   ├── auth.controller.ts  
│   │   ├── auth.service.ts  
│   │   ├── guards/  
│   │   │   └── jwt-auth.guard.ts  
│   │   └── strategies/  
│   │       └── jwt.strategy.ts  
│   └── posts/  
│       ├── posts.module.ts  
│       ├── posts.controller.ts  
│       └── posts.service.ts  
└── frontend/  
    └── index.html
```

main.ts

```
import * as cookieParser from 'cookie-parser';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.use(cookieParser());

  app.enableCors({
    origin: 'http://localhost:5500',
    credentials: true,
  });

  await app.listen(3000);
}

bootstrap();
```

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import cookieParser from 'cookie-parser';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.use(cookieParser());
  app.enableCors(
    {
      origin: 'http://localhost:5500',
      credentials: true,
    }
  );
  await app.listen(process.env.PORT ?? 3000);
}

bootstrap();
```

auth.controller.ts

Cosa cambia: access_token non esce più dal body — va in cookie HttpOnly.

Il refresh cookie ora ha path: '/auth/refresh' così il browser lo manda solo a quell'endpoint.

sameSite: 'lax' → 'strict' per ridurre superficie CSRF.

lax (il tuo originale) manda il cookie sulle navigazioni GET cross-site (link, redirect) ma lo blocca sulle richieste POST/PUT/DELETE cross-site — che sono esattamente quelle pericolose per CSRF. È il default dei browser moderni proprio perché è il bilanciamento sensato tra usabilità e sicurezza.

```
@Post('login')
@HttpCode(200)
login(
  @Body() body: LoginDto,
  @Res({ passthrough: true }) res: Response,
) {
  const tokens = this.authService.login(body.username,
body.password);
  this.setAccessTokenCookie(res, tokens.access_token); // ← aggiunto
  this.setRefreshTokenCookie(res, tokens.refresh_token);
  return { message: 'Login effettuato' }; // ← niente access_token
nel body
}
```

```
private setAccessTokenCookie(res: Response, token: string) {
  res.cookie(ACCESS_COOKIE, token, { // ← nuovo metodo
    httpOnly: true,
    sameSite: 'strict',
    secure: process.env.NODE_ENV === 'production',
    maxAge: 15 * 60 * 1000,
    path: '/',
  });
}
```

jwt.strategy.ts

Modifica chiave: jwtFromRequest ora legge il cookie invece dell'header Authorization.

passReqToCallback: true è necessario per avere accesso a req anche in validate().

```
constructor(configService: ConfigService) {  
  super({  
    jwtFromRequest: (req: Request) => {           // ← era  
      fromAuthHeaderAsBearerToken()  
      return req?.cookies?.['access_token'] ?? null;  
    },  
  
    secretOrKey: configService.get<string>('JWT_SECRET'),  
    ignoreExpiration: false,  
    passReqToCallback: true, // ← serve per leggere  
    req.cookies  
  
  });  
}
```

index.html - httponly

eliminate:

```
accessToken = data.access_token;  
localStorage.setItem('accessToken', accessToken);  
writeLog('Access token ottenuto e salvato nel Local  
Storage');
```

```
'Authorization': `Bearer ${accessToken}`
```

logout

prima:

```
accessToken = null;  
localStorage.removeItem('accessToken');
```

dopo:

```
await fetch(`${API}/auth/logout`, { method: 'POST',  
  credentials: 'include' });
```

logout

prima:

```
accessToken = null;  
localStorage.removeItem('accessToken');
```

dopo:

```
await fetch(`${API}/auth/logout`, { method: 'POST',  
  credentials: 'include' });
```

perché un cookie HttpOnly non può essere rimosso da JavaScript — solo il server può farlo con clearCookie.
Il resto — getPosts, insertPost, la logica di refresh — è strutturalmente identico.

Prima: refresh token in localStorage

let refreshToken =

```
localStorage.getItem('refreshToken');
```

Il JavaScript poteva leggere il refresh token.

- JS vede access_token
- JS vede refresh_token

Ora: refresh token in cookie HttpOnly

let accessToken =

```
localStorage.getItem('accessToken');
```

Nel frontend resta solo accessToken.

Il refreshToken non esiste più nel JS.

- JS vede access_token
- JS NON vede refresh_token

browser invia cookie automaticamente

Differenza nel refresh

Prima (local storage):

```
body: JSON.stringify({  
  refresh_token: refreshToken  
})
```

Ora (HttpOnly).

credentials: 'include'

browser, manda anche i cookie nella richiesta.

Quindi /auth/refresh riceve il refresh token dal cookie
HttpOnly.

LocalStorage: JavaScript legge il token.

HttpOnly cookie: JavaScript non legge il refresh token, lo gestisce il browser.

JWT + Passport con NestJS e TypeORM — Guida completa

Struttura finale dei file

```
src/
├── auth/
│   ├── guards/
│   │   ├── local-auth.guard.ts [attiva LocalStrategy sulla rotta login]
│   │   └── roles.guard.ts [get Role from Controller]
│   ├── strategies/
│   │   ├── local.strategy.ts [validate user+pwd]
│   │   └── jwt.strategy.ts [validate JWT payload]
│   ├── auth.controller.ts
│   ├── auth.module.ts
│   ├── auth.service.ts [valida utente e genera JWT]
│   ├── jwt-auth.guard.ts [verifica token]
│   ├── role.enum.ts [definisce ruoli]
│   └── roles.decorator.ts [definisce decorator]
├── users/
│   ├── entities/user.entity.ts
│   ├── users.module.ts
│   └── users.service.ts
├── migrations/
│   └── 1779638364804-Init.ts
├── test/
│   ├── app.e2e-spec.ts
│   ├── auth.e2e-spec.ts
│   └── jest-e2e.json
├── app.module.ts
└── main.ts
```

https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/jwt_database_typeorm

Gruppo 1 — Login Serve solo a generare il token.

- AuthController
- AuthService
- UsersService
- User entity

Gruppo 2 — JWT questo utente è autenticato

- JwtModule
- JwtStrategy
- AuthGuard('jwt')

Gruppo 3 — Ruoli questo utente può fare questa azione?

- Role enum
- @Roles()
- RolesGuard

Gruppo 4 — Controller protetti Qui applichi le regole.

```
@Get()
findAll() {
  return this.postsService.findAll();
}
```

```
username/password
↓
cerco utente nel DB
↓
bcrypt.compare()
↓
genero JWT
```

```
Authorization: Bearer token
↓
Passport legge il token
↓
JwtStrategy lo verifica
↓
salva i dati in request.user
```

```
@Roles(Role.Admin)
↓
RolesGuard legge request.user.role
↓
se ruolo corretto → passa
↓
altrimenti → 403
```

per eseguire test: `npx jest --config test/jest-e2e.json --forceExit`

Flusso completo

per eseguire test: `npx jest --config test/jest-e2e.json --forceExit`

LOGIN

POST /auth/login { username, password }
→ LocalAuthGuard → LocalStrategy.validate()
→ authService.validateUser() →
bcrypt.compare()
→ req.user = { id, username, role }
→ authService.login(req.user)
→ { access_token: "eyJ..." }

ROTTA PROTETTA

GET /posts Authorization: Bearer eyJ...
→ JwtAuthGuard → JwtStrategy.validate()
→ verifica firma JWT con JWT_SECRET
→ req.user = { id, username, role }
→ esegue il controller

JWT + Passport con NestJS e TypeORM — Guida completa

1. Pacchetti da installare

```
npm install @nestjs/passport passport passport-local passport-jwt
npm install -D @types/passport-local @types/passport-jwt
Già presenti: @nestjs/jwt, bcrypt, @types/bcrypt
```

2. .env

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=root
DB_NAME=corso_blogdb_typeorm
```

```
JWT_SECRET=supersecret2026
PORT=3000
```

3. Database — Tabella users

src/migrations/1779638364804-Init.ts

```
import { MigrationInterface, QueryRunner } from "typeorm";

export class Init1779638364804 implements MigrationInterface {
  name = 'Init1779638364804'

  public async up(queryRunner: QueryRunner): Promise<void> {
    await queryRunner.query(`
      CREATE TABLE \`${users}\` (
        \`${id}\` int NOT NULL AUTO_INCREMENT,
        \`${username}\` varchar(100) NOT NULL,
        \`${password}\` varchar(255) NOT NULL,
        \`${role}\` enum ('user','admin','superadmin') NOT NULL DEFAULT 'user',
        \`${active}\` tinyint NOT NULL DEFAULT 1,
        UNIQUE INDEX \`${IDX_fe0bb3f6520ee0469504521e71}\` (\`${username}\`),
        PRIMARY KEY (\`${id}\`)
      ) ENGINE=InnoDB
    `);
  }

  public async down(queryRunner: QueryRunner): Promise<void> {
    await queryRunner.query(`DROP INDEX \`${IDX_fe0bb3f6520ee0469504521e71}\` ON \`${users}\``);
    await queryRunner.query(`DROP TABLE \`${users}\``);
  }
}
```

Users — Entity, Service, Module

src/users/entities/user.entity.ts

```
import { Column, Entity, PrimaryGeneratedColumn } from
  'typeorm';
import { Role } from '../auth/role.enum';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn()
  id!: number;

  @Column({ length: 100, unique: true })
  username!: string;

  @Column({ length: 255 })
  password!: string;    // hash bcrypt, mai plaintext

  @Column({ type: 'enum', enum: Role, default: Role.User })
  role!: Role;

  @Column({ default: true })
  active!: boolean;
}
```

src/users/users.service.ts

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { User } from '../entities/user.entity';

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private readonly userRepository: Repository<User>,
  ) {}

  findByUsername(username: string) {
    return this.userRepository.findOne({
      where: { username, active: true },
    });
  }
}
```

Users — Entity, Service, Module

src/users/users.module.ts

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from
  '@nestjs/typeorm';
import { UsersService } from './users.service';
import { User } from './entities/user.entity';

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  exports: [UsersService], // <-- esportato: serve
    ad AuthModule
})
export class UsersModule {}
```

Auth — Enum e Decorator

src/auth/role.enum.ts

```
export enum Role {  
  User    = 'user',  
  Admin    = 'admin',  
  SuperAdmin = 'superadmin',  
}
```

src/auth/roles.decorator.ts

```
import { SetMetadata } from '@nestjs/common';  
import { Role } from '../role.enum';  
  
export const ROLES_KEY = 'roles';  
export const Roles = (...roles: Role[]) =>  
  SetMetadata(ROLES_KEY, roles);
```

Auth — Strategie Passport

src/auth/strategies/local.strategy.ts

```
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { Strategy } from 'passport-local';
import { AuthService } from '../auth.service';

@Injectable()
export class LocalStrategy extends PassportStrategy(Strategy) {
  constructor(private readonly authService: AuthService) {
    super(); // default: legge body.username e body.password
  }

  async validate(username: string, password: string) {
    const user = await this.authService.validateUser(username, password);
    if (!user) throw new UnauthorizedException('Credenziali non valide');
    return user; // Passport mette questo in req.user
  }
}
```

src/auth/strategies/jwt.strategy.ts

```
import { Injectable } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { ConfigService } from '@nestjs/config';

interface JwtPayload {
  sub: number;
  username: string;
  role: string;
}

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(configService: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: configService.get<string>('JWT_SECRET') ?? 'default_secret',
    });
  }

  validate(payload: JwtPayload) {
    // Questo oggetto finisce in req.user su ogni rotta protetta
    return { id: payload.sub, username: payload.username, role: payload.role };
  }
}
```


Auth — Guard

src/auth/guards/local-auth.guard.ts

```
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

@Injectable()
export class LocalAuthGuard extends AuthGuard('local') {}
// attiva LocalStrategy → usato solo su POST /auth/login
```

src/auth/jwt-auth.guard.ts

```
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}
// attiva JwtStrategy → usato su ogni rotta protetta
```

src/auth/strategies/jwt.strategy.ts

```
import { Injectable } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { ConfigService } from '@nestjs/config';

interface JwtPayload {
  sub: number;
  username: string;
  role: string;
}

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(configService: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: configService.get<string>('JWT_SECRET') ?? 'default_secret',
    });
  }

  validate(payload: JwtPayload) {
    // Questo oggetto finisce in req.user su ogni rotta protetta
    return { id: payload.sub, username: payload.username, role: payload.role };
  }
}
```

Auth — Service, Controller, Module

src/auth/auth.service.ts

```
import { Injectable } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import * as bcrypt from 'bcrypt';
import { UsersService } from '../users/users.service';
import { User } from '../users/entities/user.entity';

@Injectable()
export class AuthService {
  constructor(
    private readonly jwtService: JwtService,
    private readonly usersService: UsersService,
  ) {}

  // Chiamato da LocalStrategy: verifica credenziali
  async validateUser(username: string, password: string): Promise<Omit<User, 'password'> | null> {
    const user = await this.usersService.findByUsername(username);
    if (user && (await bcrypt.compare(password, user.password))) {
      const { password: _, ...result } = user;
      return result; // password esclusa prima di mettere in req.user
    }
    return null;
  }

  // Chiamato dal controller dopo che Passport ha già validato
  login(user: Omit<User, 'password'>) {
    const payload = { sub: user.id, username: user.username, role: user.role };
    return { access_token: this.jwtService.sign(payload) };
  }
}
```

src/auth/auth.controller.ts

```
import { Controller, Post, Request, UseGuards } from '@nestjs/common';
import { ApiBody, ApiOperation, ApiTags } from '@nestjs/swagger';
import { AuthService } from './auth.service';
import { LocalAuthGuard } from './guards/local-auth.guard';

@ApiTags('auth')
@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @ApiOperation({ summary: 'Login con username e password' })
  @ApiBody({
    schema: {
      type: 'object',
      properties: {
        username: { type: 'string' },
        password: { type: 'string' },
      },
    },
  })
  @UseGuards(LocalAuthGuard) // Passport esegue LocalStrategy, poi req.user è pronto
  @Post('login')
  login(@Request() req: any) {
    return this.authService.login(req.user);
  }
}
```

auth.modules

src/auth/auth.module.ts

```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { AuthController } from './auth.controller';
import { AuthService } from './auth.service';
import { UsersModule } from '../users/users.module';
import { LocalStrategy } from './strategies/local.strategy';
import { JwtStrategy } from './strategies/jwt.strategy';
```

```
@Module({
  imports: [
    PassportModule,
    JwtModule.register({
      secret: process.env.JWT_SECRET || 'default_secret',
      signOptions: { expiresIn: '1h' },
    }),
    UsersModule, // porta UsersService in questo modulo
  ],
  controllers: [AuthController],
  providers: [AuthService, LocalStrategy, JwtStrategy],
})
export class AuthModule {}
```

App Module

src/app.module.ts

```
import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { PostsModule } from './posts/posts.module';
import { CategoriesModule } from './categories/categories.module';
import { AuthModule } from './auth/auth.module';
import { UsersModule } from './users/users.module';
import { AppController } from './app.controller';
import { AppService } from './app.service';
```

```
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true, envFilePath: '.env' }),
    TypeOrmModule.forRootAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        type: 'mysql',
        host: config.get<string>('DB_HOST'),
        port: config.get<number>('DB_PORT'),
        username: config.get<string>('DB_USER'),
        password: config.get<string>('DB_PASSWORD'),
        database: config.get<string>('DB_NAME'),
        autoLoadEntities: true,
        synchronize: false,
      }),
    ],
    PostsModule,
    CategoriesModule,
    AuthModule,
    UsersModule,
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

src/main.ts

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import dotenv from 'dotenv';
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
```

```
dotenv.config();
```

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
```

```
  const config = new DocumentBuilder()
    .setTitle('Database MySQL API')
    .setDescription('API for managing posts with MySQL database')
    .setVersion('1.0')
    .addTag('posts')
    .addTag('categories')
    .addTag('auth')
    .addBearerAuth() // aggiunge il bottone "Authorize" in Swagger
    .build();
```

```
  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);
```

```
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

Proteggere una rotta — esempio su PostsController

post.controller.ts

```
import { Controller, Get, UseGuards, Request } from
  '@nestjs/common';
import { ApiBearerAuth } from '@nestjs/swagger';
import { JwtAuthGuard } from '../auth/jwt-auth.guard';
import { Roles } from '../auth/roles.decorator';
import { Role } from '../auth/role.enum';

@ApiBearerAuth()      // mostra il lucchetto in Swagger
@UseGuards(JwtAuthGuard) // protegge tutte le rotte del
  controller
@Controller('posts')
export class PostsController {

  @Roles(Role.Admin)    // decoratore informativo (serve
    un RolesGuard se vuoi applicarlo)
  @Get()
  findAll(@Request() req: any) {
    console.log(req.user); // { id, username, role } messo da
    JwtStrategy
    // ...
  }
}
```

JWT: Esempio 2

Obiettivo

Proteggere API REST NestJS con:

Anonimo → solo GET

User → GET, POST, PATCH/PUT

Admin → tutto, incluso DELETE

Punto di partenza: abbiamo già:

- PostsController

- CategoriesController

- Swagger funzionante

- CRUD testato

partiamo da:

https://github.com/docentemaurocasadei/backend-nestjs-esempi/tree/main/database_mysql_type-orm

JWT con Passport - installazione

installazione

npm install @nestjs/passport passport passport-jwt

npm install -D @types/passport-jwt

New-Item src/auth/strategies -ItemType Directory

New-Item src/auth/guards -ItemType Directory

New-Item src/auth/strategies/jwt.strategy.ts

New-Item src/auth/guards/jwt.guard.ts

src/

└─ auth/

│ └─ auth.module.ts

│ └─ auth.service.ts

│ └─ auth.controller.ts

│ └─ strategies/

│ └─ jwt.strategy.ts

└─ guards/

└─ jwt.guard.ts

- ✓ JwtStrategy → valida il token e popola req.user
- ✓ JwtAuthGuard → protegge le route (401 se no token)
- ✓ RolesGuard → controlla i ruoli (403 se ruolo sbagliato)
- ✓ Roles decorator → definisce i ruoli richiesti per route/controller

jwt.strategies.js

controlla il token e se è valido ti dice chi sei req.user

Request: GET /categories

Header: Authorization: Bearer eyJhbGc...



[JwtStrategy] Estrae il token dall'header



[JwtStrategy] Verifica la firma con JWT_SECRET



[JwtStrategy] validate(payload) → { userId, username, role }



req.user = { userId, username, role }



Controller esegue la logica

```
import { Injectable } from '@nestjs/common'; import { PassportStrategy } from '@nestjs/passport';  
import { ExtractJwt, Strategy } from 'passport-jwt'; import { ConfigService } from '@nestjs/config';  
@Injectable()
```

```
export class JwtStrategy extends PassportStrategy(Strategy) {  
  constructor(private readonly configService: ConfigService) {  
    const secret = configService.get<string>('JWT_SECRET');  
    if (!secret) throw new Error('JWT_SECRET non definito nel .env');
```

```
  }  
  super({  
    jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),  
    secretOrKey: secret,  
  });  
  }  
  async validate(payload: any) {  
    // Quello che ritorni qui finisce in req.user  
    return {  
      userId: payload.sub,  
      username: payload.username,  
      role: payload.role,  
    };  
  }  
}
```


jwt-auth.guard.ts

tutta la logica è già in JwtStrategy. Il guard fa solo da collegamento tra il decoratore @UseGuards() e Passport

```
@UseGuards(JwtAuthGuard) → AuthGuard('jwt')  
→ JwtStrategy.validate()
```

jwt-auth.guard.ts

```
import { Injectable } from '@nestjs/common';  
import { AuthGuard } from '@nestjs/passport';
```

```
@Injectable()
```

```
export class JwtAuthGuard extends AuthGuard('jwt') {}
```

per proteggere route:

```
@Get('profile')
```

```
@UseGuards(JwtGuard) // ← una riga sola
```

```
getProfile(@Request() req) {
```

```
  return req.user;
```

```
}
```

auth.module.ts

PassportModule.register({ defaultStrategy: 'jwt' })

Inizializza Passport nell'applicazione e imposta jwt come strategia di default. Questo significa che se usi AuthGuard() senza argomenti, usa automaticamente JWT.

JwtStrategy nei providers

Registrando JwtStrategy come provider, NestJS la istanzia e Passport la riconosce automaticamente con il nome 'jwt'. È il collegamento tra il tuo codice e il sistema di Passport.

JwtGuard nei providers

Lo rende disponibile per l'injection nel modulo. Senza questa riga non potresti iniettarlo nei controller con @UseGuards(JwtGuard).

exports: [JwtGuard]

Rende JwtGuard disponibile agli altri moduli. Per esempio CategoriesModule potrà proteggere le sue route con @UseGuards(JwtGuard) senza dover reimportare nulla, grazie a questo export.

```
import { Module } from '@nestjs/common';
import { PassportModule } from '@nestjs/passport';
import { AuthService } from './auth.service';
import { AuthController } from './auth.controller';
import { JwtStrategy } from './strategies/jwt.strategy';
import { JwtAuthGuard } from './guards/jwt.guard';
```

```
@Module({
  imports: [
    PassportModule.register({ defaultStrategy: 'jwt' }),
  ],
  controllers: [AuthController],
  providers: [
    AuthService,
    JwtStrategy, // ← Passport la registra automaticamente
    JwtAuthGuard,
  ],
  exports: [JwtAuthGuard], // ← altri moduli possono usare il guard
})
export class AuthModule {}
```

auth.service.ts

validateUser() controlla le credenziali e ritorna l'utente se sono corrette, null altrimenti. Per ora è hardcoded, lo collegheremo al DB in seguito.

login() prende l'utente validato, costruisce il payload e genera il token JWT firmato con il secret del .env. Il campo sub è la convenzione standard JWT per l'ID utente.

```
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
```

```
@Injectable()
```

```
export class AuthService {
  constructor(private readonly jwtService: JwtService) {}
```

```
  validateUser(username: string, password: string) {
    // TODO: sostituire con query al DB quando avremo UsersModule
    if (username === 'test' && password === 'password') {
      return { userId: 1, username: 'test', role: 'admin' };
    }
    return null;
  }
```

```
  login(user: { userId: number; username: string; role: string }) {
    const payload = {
      sub: user.userId, // ← standard JWT, identifica l'utente
      username: user.username,
      role: user.role,
    };
    return {
      access_token: this.jwtService.sign(payload),
    };
  }
}
```

auth.controller.ts

@Controller('auth') — tutte le route di questo controller partono da /auth.

@Post('login') — espone l'endpoint POST /auth/login.

@Body() — estrae username e password dal body della richiesta JSON.

```
import { Controller, Post, Body, UnauthorizedException } from '@nestjs/common';
```

```
import { AuthService } from './auth.service';
```

```
@Controller('auth')
```

```
export class AuthController {
```

```
  constructor(private readonly authService: AuthService) {}
```

```
  @Post('login')
```

```
  login(@Body() body: { username: string; password: string }) {
```

```
    const user = this.authService.validateUser(body.username, body.password);
```

```
    if (!user) throw new UnauthorizedException('Credenziali non valide');
```

```
    return this.authService.login(user);
```

```
  }
```

```
}
```

test jwt con passport

corretta:

```
curl -X POST http://localhost:3000/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username": "test", "password":  
  "password"}'
```

credenziali errate

```
curl -X POST http://localhost:3000/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username": "test", "password":  
  "sbagliata"}'
```

// Protezione Solo una route

```
@Get('profile')  
@UseGuards(JwtGuard)  
getProfile() {}
```

// Protezione Tutto il controller protetto

```
@UseGuards(JwtGuard)  
@Controller('categories')  
export class CategoriesController {}
```

Ruoli con JwtRoles

I **ruoli (roles)** servono per controllare **chi può fare cosa** nell'applicazione.

👉 Esempio:

ADMIN → accesso completo

USER → accesso limitato

New-Item src/auth/decorators -ItemType Directory

New-Item src/auth/decorators/roles.decorator.ts

New-Item src/auth/guards/roles.guard.ts

src/

└─ auth/

└─ guards/

│ └─ jwt-auth.guard.ts

│ └─ roles.guard.ts

└─ decorators/

└─ roles.decorator.ts

roles.guard.ts**

✓ Controlla il ruolo dell'utente

✓ Decide se può accedere alla rotta

🏷️ decorators/

👉 Servono per dichiarare regole in modo semplice

* **roles.decorator.ts**

✓ Definisce i ruoli richiesti

✓ Es: `@Roles('admin')`

roles.decorator.ts

Role è un enum che definisce i ruoli disponibili nell'app. Usare un enum invece di stringhe libere evita errori di battitura.

Roles() è un decorator custom che attacca i ruoli richiesti come metadato sulla route. Per esempio `@Roles(Role.ADMIN)` dice al RolesGuard che solo gli admin possono accedere.

```
import { SetMetadata } from '@nestjs/common';
```

```
export enum Role {  
  ADMIN = 'admin',  
  USER = 'user',  
}
```

```
export const Roles = (...roles: Role[]) => SetMetadata('roles',  
  roles);
```

roles.guard.ts

Reflector legge i metadati attaccati dal decorator `@Roles()` sulla route.

requiredRoles sono i ruoli richiesti per accedere alla route.
Se non ce ne sono, la route è pubblica e passa direttamente.

user.role viene da `req.user` che `JwtStrategy.validate()` ha popolato dopo aver verificato il token.

`@Roles(Role.ADMIN)`



RolesGuard legge 'admin' dai metadati



controlla `req.user.role`



'admin' === 'admin' →  passa

'user' === 'admin' →  403 ForbiddenException

```
import { Injectable, CanActivate, ExecutionContext, ForbiddenException }  
  from '@nestjs/common';  
import { Reflector } from '@nestjs/core';  
import { Role } from '../decorators/roles.decorator';
```

```
@Injectable()
```

```
export class RolesGuard implements CanActivate {  
  constructor(private readonly reflector: Reflector) {}
```

```
  canActivate(context: ExecutionContext): boolean {  
    const requiredRoles = this.reflector.get<Role[]>('roles',  
      context.getHandler());  
    if (!requiredRoles) return true; // ← nessun ruolo richiesto, passa
```

```
    const { user } = context.switchToHttp().getRequest();  
    if (!requiredRoles.includes(user.role)) {  
      throw new ForbiddenException('Accesso negato');  
    }
```

```
    return true;
```

```
  }  
}
```


protezione rotte

abilitare solo lettura per user e modifica, inserimento, cancellazione per admin

categories.controller.ts

```
@Controller('categories')
```

```
@UseGuards(JwtAuthGuard, RolesGuard) // ← protegge  
tutto il controller
```

```
export class CategoriesController {
```

```
  @Get()
```

```
    @Roles(Role.ADMIN, Role.USER) // ← tutti gli utenti  
    loggati
```

```
    findAll() {  
      return 'lista categorie';  
    }  
  }
```

```
  @Get('admin')
```

```
    @Roles(Role.ADMIN) // ← solo admin
```

```
    adminOnly() {  
      return 'solo per admin';  
    }  
  }  
}
```

in auth.module.ts

```
@Module({
```

```
  imports: [
```

```
    PassportModule.register({ defaultStrategy: 'jwt' }),  
  ],
```

```
  controllers: [AuthController],
```

```
  providers: [
```

```
    AuthService,  
    JwtStrategy,  
    JwtAuthGuard,
```

```
    RolesGuard, // ← aggiunto
```

```
  ],
```

```
  exports: [
```

```
    JwtAuthGuard,  
    RolesGuard, // ← aggiunto
```

```
  ],
```

```
})
```

Upload di file

Multer & NestJS

Upload di file con Multer in NestJS

Per gestire l'upload di file in NestJS si utilizza **Multer**, un middleware pensato per leggere richieste HTTP di tipo: multipart/form-data

Questo formato viene usato quando un client invia uno o più file al server.

Cosa va fatto

Nel controller bisogna:

- creare una rotta POST;
- attivare Multer con FileInterceptor;
- indicare il nome del campo file;
- configurare dove salvare il file;
- recuperare il file con @UploadedFile().

Perché si usa l'interceptor

In NestJS un **interceptor** intercetta la richiesta prima che arrivi al metodo del controller.

Nel caso dell'upload:

@UseInterceptors(FileInterceptor('file'))

serve perché il file **non arriva come JSON normale**.

Arriva dentro una richiesta multipart/form-data, quindi **NestJS da solo non lo legge come un normale @Body()**.

Il FileInterceptor fa tre cose importanti:

- legge il file dalla richiesta;
- lo passa a Multer;
- rende il file disponibile nel controller tramite @UploadedFile().

```
@Controller('upload')
export class UploadController {
  constructor(
    private readonly configService:
    ConfigService
  ) {
  }

  @Post()
  @ApiConsumes('multipart/form-
data')
  @ApiBody({
    schema: {
      type: 'object',
      properties: {
        file: {
          type: 'string',
          format: 'binary',
        },
      },
    },
  })
}
```

```
@UseInterceptors(
  FileInterceptor('file', {
    storage: diskStorage({
      destination: process.env.PATH_UPLOADS ||
'./uploads',
      filename: (req, file, cb) => {
        const name = Date.now() + '-' +
file.originalname;
        cb(null, name);
      },
    }),
  ),
)
upload(@UploadedFile() file:
Express.Multer.File) {
  return {
    message: 'File caricato correttamente',
    filename: file.filename,
    path: file.path,
  };
}
```

Installazione in Produzione

CI/CD automatico per app NestJS con GitHub Actions

Obiettivo

Automatizzare il deploy della web app **Hamburgeria** sviluppata in **NestJS** su VPS Linux.

Flusso desiderato:

git push su GitHub



GitHub Actions parte automaticamente



Connessione SSH al VPS



Esecuzione update.sh



Docker rebuild + restart



Applicazione online aggiornata

Tecnologie utilizzate

GitHub Repository codice sorgente

GitHub Actions per CI/CD

VPS Linux Ubuntu

SSH per accesso remoto

Docker + Docker Compose

NestJS backend

Variabili ambiente .env.production

Preparazione server VPS

- **Accesso server**
- `ssh root@IP_SERVER` (oppure chiave SSH)
- **Cartella applicazioni**
- `cd /opt/apps`
- **Clonazione progetto**
- `git clone https://github.com/docentemaurocasadei/hamburgeria-PW hamburgeria`
- Struttura finale:
- `/opt/apps/hamburgeria`

Configurazione ambiente produzione

- Creare file:
- back-end/.env.production
- Esempio:
- `NODE_ENV=production`
`PORT=3000`
`DB_HOST=mysql`
`DB_PORT=3306`
`DB_USER=root`
`DB_PASS=password`
`DB_NAME=hamburgeria`
`JWT_SECRET=secretkey`
- Questo file contiene configurazioni sensibili del server.

Avvio Docker in produzione

- Comando:
- `docker compose \`
`--env-file ./back-end/.env.production \`
`-f docker-compose.yml \`
`-f docker-compose.prod.yml \`
`up -d --build`
- **Significato**
- usa env produzione
- merge file docker compose
- rebuild immagini
- avvio container in background

Deploy Key GitHub

- Il server deve poter eseguire:
- git pull
- senza password.
- **Procedura**
- Sul VPS:
- `ssh-keygen -t ed25519`
`cat ~/.ssh/id_ed25519.pub`
- Copiare la chiave pubblica.
- Su GitHub:
- Repo → Settings → Deploy keys
- Aggiungere chiave.

Secrets GitHub Actions

- Nel repository GitHub creare:
- VPS_HOST
VPS_USER
VPS_PORT
VPS_KEY
- **Dove**
- Settings → Secrets and variables → Actions

File workflow deploy.yml

Percorso:

.github/workflows/deploy.yml

È il file che dice a GitHub **quando** e **come** eseguire il deploy automatico.

name: Deploy Hamburgeria

on:

push:

branches:

- main

jobs:

deploy:

runs-on: ubuntu-latest

steps:

- name: Deploy via SSH

uses: appleboy/ssh-action@v1.0.3

with:

host: \${ secrets.VPS_HOST }

username: \${ secrets.VPS_USER }

key: \${ secrets.VPS_KEY }

port: \${ secrets.VPS_PORT }

script: |

bash /opt/apps/hamburgeria/update.sh

Script update.sh sul server

Percorso:

/opt/apps/hamburgeria/update.sh

È lo script reale di deploy.

Contenuto:

```
#!/bin/bash
set -e

cd /opt/apps/hamburgeria

git pull origin main

docker compose \
--env-file ./back-end/.env.production \
-f docker-compose.yml \
-f docker-compose.prod.yml \
up -d --build
```

come funziona

Come funziona il deploy

Lo sviluppatore esegue:

`git add .`

`git commit -m "update"`

`git push`

GitHub Actions rileva il push e avvia:

Deploy Hamburgeria

Aggiornamento automatico sul VPS.

Vantaggi del CI/CD

- ✓ deploy rapido
 - ✓ niente accessi manuali al server
 - ✓ meno errori umani
 - ✓ rollback più semplice
 - ✓ standard professionale DevOps
 - ✓ ideale per team software house

DOCUMENTAZIONE E SWAGGER

Swagger è uno strumento che permette di documentare e testare le API REST.

In pratica genera una pagina web dove possiamo vedere:

GET /users
POST /users
GET /users/:id
PUT /users/:id
DELETE /users/:id

Per ogni endpoint mostra:

- metodo HTTP
- URL
- parametri
- body della richiesta
- risposte possibili
- codici di stato

Frase semplice per gli studenti:

Swagger è il manuale interattivo delle nostre API.

Senza Swagger, chi usa la nostra API deve leggere il codice o chiedere allo sviluppatore.

Con Swagger, invece, abbiamo una pagina dove si può:

- vedere tutti gli endpoint disponibili
- capire quali dati inviare
- testare direttamente le chiamate
- documentare automaticamente il backend

Esempio:

POST /users

richiede magari un body come:

```
{  
  "name": "Mario Rossi",  
  "email": "mario@example.com"  
}
```

Swagger lo mostra in modo chiaro

Installazione SWAGGER in NestJS

```
npm install @nestjs/swagger
```

main.ts

```
import { NestFactory } from '@nestjs/core';  
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';  
import { AppModule } from './app.module';
```

```
async function bootstrap() {  
  const app = await NestFactory.create(AppModule);
```

```
  const config = new DocumentBuilder()  
    .setTitle('API Gestionale')  
    .setDescription('Documentazione delle API del progetto')  
    .setVersion('1.0')  
    .addTag('users')  
    .build();
```

```
  const document = SwaggerModule.createDocument(app, config);
```

```
  SwaggerModule.setup('api', app, document);
```

```
  await app.listen(3000);  
}
```

```
bootstrap();
```

<http://localhost:3000/api>



chiamare una API con fetch

Scenario: pagina statica → dati dinamici

Il browser non deve conoscere il database: chiede dati a una API.



- HTML prepara la struttura
- JavaScript ascolta un evento
- fetch() invia la richiesta HTTP
- la risposta JSON viene mostrata nella pagina

1. Prepariamo l'HTML

Servono elementi identificabili, così JavaScript sa dove leggere e dove scrivere.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Products</title>
  <link rel="stylesheet" href="./style.css">
</head>
<body>
  <button id="loadBtnFetch">Carica prodotti fetch</button>
  <button id="loadBtnAsyncAwait">Carica prodotti async-await</button>

  <div id="productsList" class="products-grid"></div>

  <p id="message"></p>

  <template id="productCardTemplate">
    <div class="product-card">
      <h3 class="product-name"></h3>
      <p class="product-description"></p>
      <p class="product-price"></p>
    </div>
  </template>

  <script src="./products.js"></script>

  <script src="./products.js"></script>
</body>
</html>
```

#loadBtn
intercetta il click

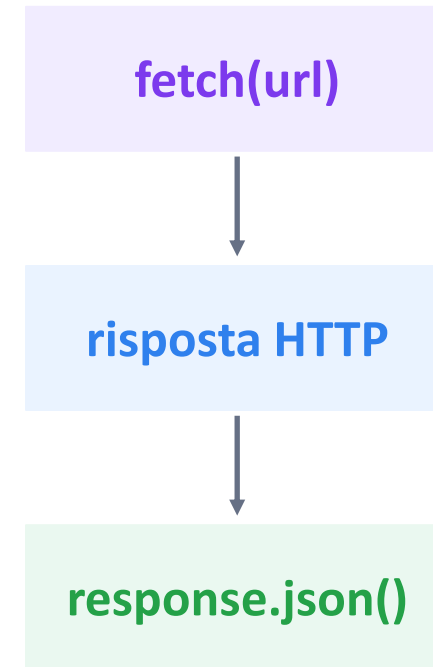
#usersList
mostra i dati

#message
mostra stato/errore

2. Prima chiamata con fetch()

fetch() restituisce una Promise: la richiesta non blocca la pagina.

```
document.getElementById('loadBtnFetch').addEventListener('click', function() {
  fetch('http://localhost:3000/products')
    .then(response => response.json())
    .then(data => {
      loadProductsWithError(data);
    })
    .catch(error => {
      document.getElementById('message').textContent = 'Errore nel caricamento dei prodotti';
      console.error('Errore:', error);
    });
});
```



Idea chiave: prima arriva la risposta, poi la convertiamo in JSON.

Mostriamo i dati nel DOM

Dai dati JSON creiamo elementi HTML dinamici.

```
function loadProductsWithError(data) {  
  const message = document.getElementById('message');  
  const productList = document.getElementById('productsList');  
  const template = document.getElementById('productCardTemplate');  
  message.textContent = 'Caricamento prodotti...';  
  productList.innerHTML = '';  
  try {  
    data.forEach(product => {  
      const card = template.content.cloneNode(true);  
      card.querySelector('.product-name').textContent = product.name;  
      card.querySelector('.product-description').textContent =  
        product.description ?? 'Nessuna descrizione disponibile';  
      card.querySelector('.product-price').textContent =  
        product.price ? '€ ' + product.price : 'Prezzo non disponibile';  
      productList.appendChild(card);  
    });  
    message.textContent = 'Prodotti caricati: ' + data.length;  
  } catch (error) {  
    message.textContent = 'Errore nel caricamento dei prodotti';  
    console.error('Errore:', error);  
  }  
}
```

JSON users[]



forEach()



2a. lo stile css

formattazione della pagina

```
.products-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 16px;
  margin-top: 20px;
}

.product-card {
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 16px;
  background: white;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

.product-name {
  margin: 0 0 8px;
  font-size: 20px;
}

.product-description {
  color: #555;
}

.product-price {
  font-weight: bold;
  color: #0a7;
}
```

3. Versione moderna: async / await

Stesso comportamento, codice più leggibile.

```
const button = document.getElementById('loadBtnAsyncAwait');

button.addEventListener('click', loadProducts);

async function loadProducts() {
  const message = document.getElementById('message');
  const productList = document.getElementById('productsList');
  const template = document.getElementById('productCardTemplate');

  message.textContent = 'Caricamento prodotti...';
  productList.innerHTML = '';

  try {
    const response = await fetch('http://localhost:3000/products');

    if (!response.ok) {
      throw new Error('Errore HTTP ' + response.status);
    }

    const products = await response.json();

    loadProductsWithError(products);

    message.textContent = 'Prodotti caricati: ' + products.length;
  } catch (error) {
    message.textContent = 'Errore nel caricamento dei prodotti';
    console.error('Errore:', error);
  }
}
```

async

la funzione può usare
await

await

aspetta il risultato della
Promise

json()

converte il corpo della
risposta

eseguiamo su server

- Per eseguire una pagina HTML come vero sito locale possiamo usare:
- `npx serve .`
- **`npx serve`** avvia (grazie a node.js) un piccolo **web server locale** nella cartella corrente.
- potrai aprirlo dal browser tramite un indirizzo tipo:
- `http://localhost:3000`
- oppure:
- `http://localhost:5000`
- per personalizzare la porta
- `npx serve . --listen 5500`

```
PS C:\Corsi\ITS Turismo Marche\2025-2027 Fullstack Senigallia\docenza_backend\esempi\fetch_frontend_html_js> npx serve .

Serving!

- Local:   http://localhost:49789
- Network: http://192.168.1.68:49789

This port was picked because 3000 is in use.

Copied local address to clipboard!
```


6. Gestire errori e caricamento

Una buona pagina informa l'utente: caricamento, successo, errore.

```
try {  
  const response = await fetch('http://localhost:3000/products');  
  if (!response.ok) {  
    throw new Error('Errore HTTP ' + response.status);  
  }  
  ....  
  ....  
  ....  
  message.textContent = 'Prodotti caricati: ' + products.length;  
} catch (error) {  
  message.textContent = 'Errore nel caricamento dei prodotti';  
  console.error('Errore:', error);  
}
```

Caricamento...

response.ok?

successo

catch(error)

7. Inviare dati: fetch con POST

Non solo leggere: possiamo anche mandare un body JSON al server.

```
async function createProduct() {  
  const response = await fetch('/api/product', {  
    method: 'POST',  
    headers: {  
      'Content-Type': 'application/json'  
    },  
    body: JSON.stringify({  
      name: 'Prodotto1',  
      sku: 'PDT1X20'  
    })  
  });  
  
  const result = await response.json();  
  console.log(result);  
}
```

method
POST

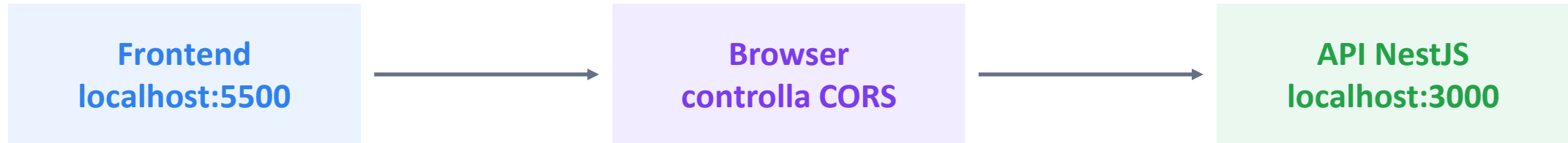
headers
JSON

body
stringify

API
crea record

8. Attenzione a CORS

Quando frontend e backend stanno su domini/porte diverse, **il browser applica regole di sicurezza**.



Se CORS non è abilitato, la chiamata può essere bloccata dal browser.

```
// main.ts in NestJS
app.enableCors({
  origin: 'http://localhost:5500',
});
```

CORS serve a impedire che **una pagina web caricata nel browser dell'utente** faccia chiamate cross-origin non autorizzate usando il browser dell'utente.

CORS: la differenza è l'origine del sito

Il browser fa sempre la richiesta, ma controlla **da quale sito è partita la pagina** che esegue il codice JavaScript.

Caso 1 — Same-origin

Pagina aperta:

<https://banca.it>

JavaScript chiama:

<https://banca.it/api/conto>

Il browser consente la richiesta perché la pagina e l'API hanno la stessa origine

Non conta solo chi riceve la richiesta.

Conta soprattutto l'origine della pagina che la genera.

Caso 2 — Cross-origin

Pagina aperta:

<https://sito-malevolo.it>

JavaScript chiama:

<https://banca.it/api/conto>

Origine diversa:

sito-malevolo.it → banca.it

Il browser chiede:

banca.it autorizza richieste provenienti da sito-malevolo.it?

Se la risposta è no, il browser blocca l'accesso alla risposta.

versione con file ts e non js

installare il compilatore typescript

```
npm init -y
```

```
npm install typescript --save-dev
```

viene creato il package.json

creare tsconfig.json

```
npx tsc --init
```

creiamo il product.ts e poi compiliamolo

```
npx tsc
```

```
npx serve . --listen 5500
```

tsconfig.json

```
{  
  "compilerOptions": {  
    "target": "ES2020",  
    "module": "ES6",  
    "rootDir": "./src",  
    "outDir": "./dist",  
    "strict": true  
  }  
}
```

```
progetto/  
├─ index.html  
├─ src/  
│   └─ products.ts  
├─ dist/  
│   └─ products.js  
├─ style.css  
└─ tsconfig.json
```

migliorare il codice

npm i prettier

npx prettier --write "src/**/*.ts"



tsconfig.json



style.css



index.html



products.ts

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ES6",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true
  }
}
```

```
progetto/
├─ index.html
├─ src/
│   └─ products.ts
├─ dist/
│   └─ products.js
├─ style.css
└─ tsconfig.json
```

versione con react e vite

Creazione del progetto

```
npm create vite@latest fetch-react -- --template react-ts
```

Crea un nuovo progetto React con TypeScript utilizzando Vite.

fetch-react → nome della cartella progetto

react-ts → template React + TypeScript

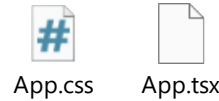
```
cd fetch-react
```

```
npm install
```

nel backend ricordarsi di aggiungere CORS

```
app.enableCors({  
  origin: ['http://localhost:5500',  
    'http://localhost:5173'],  
});
```

creare il App.tsx e App.css



```
npm run dev
```



Vantaggi di Vite

- Avvio molto veloce
- Refresh automatico
- Supporto TypeScript immediato
- Build ottimizzata
- Configurazione semplice

Unit Test

unit test

Uno **unit test** testa **una singola unità di codice in isolamento**, dove per "unità" si intende tipicamente una funzione o una classe.

La parola chiave è **isolamento** — tutte le dipendenze esterne vengono sostituite con dei finti (mock/stub), in modo che il test verifichi solo il comportamento di quella specifica unità, senza che il risultato dipenda da database, rete, file system o altre classi.

i file utilizzati per il test sono *.spec.ts

per eseguire gli unit test

```
npm run test (tutti)
```

```
npm run test posts
```

oppure

```
npm run test posts.controller
```

```
npm run test posts.service
```

jest e sqlite

Jest è un framework per i test in JavaScript/TypeScript, sviluppato da Meta. Si occupa di:

eseguire i file *.spec.ts

fornire le funzioni describe, it, expect, beforeAll, ecc.

generare il report finale con pass/fail

gestire mock, spy, coverage

In pratica è il "motore" che fa girare i tuoi test

SQLite è un database relazionale, come PostgreSQL o MySQL, ma con una differenza fondamentale: **non ha un server**. Il database è un singolo file .db sul disco (oppure :memory: solo in RAM, come nel tuo caso).

È ideale per i test perché:

non richiede installazione né configurazione

si crea e distrugge in millisecondi

si comporta come un vero database SQL, quindi TypeORM ci lavora normalmente

con database: ':memory:' SQLite esiste **solo in RAM** durante l'esecuzione dei test e viene distrutto automaticamente alla fine

i test con spec.ts usando sqllite per ambiente di dev

installare jest - Jest è un **framework per fare test in JavaScript/TypeScript**

```
npm install --save-dev @types/jest
```

aggiungere in tsconfig.json

```
"compilerOptions": {  
.....  
  "types": ["jest", "node"]  
}
```

metti in package.json in jest

```
"jest": {  
.....  
  "moduleNameMapper": {  
    "^src/(.*)$": "<rootDir>/$1"  
  }  
}
```

sqllite

```
npm install --save-dev sqllite3
```

alla fine

```
npm run test
```

controller.spec.ts

```
posts.controller.spec.ts
describe('PostsController', () => {
  let controller: PostsController;

  beforeEach(async () => {
    const module: TestingModule = await Test.createTestingModule({
      controllers: [PostsController],
      providers: [
        PostsService,
        { provide: getRepositoryToken(Post), useValue: {} }, // aggiunto
      ],
    }).compile();

    controller = module.get<PostsController>(PostsController);
  });

  it('should be defined', () => {
    expect(controller).toBeDefined();
  });
});
```

service.spec.ts

```
import { Test, TestingModule } from '@nestjs/testing';
import { TypeOrmModule } from '@nestjs/typeorm';
import { PostsService } from './posts.service';
import { Post } from './entities/post.entity';
import { User } from './users/entities/user.entity';
import { Category } from './categories/entities/category.entity';

describe('PostsService (SQLite)', () => {
  let service: PostsService;
  let module: TestingModule;
  let createdPostId: number; // ← id condiviso tra i test
  beforeAll(async () => {
    try {
      module = await Test.createTestingModule({
        imports: [
          TypeOrmModule.forRoot({
            type: 'sqlite',
            database: ':memory:',
            entities: [Post, User, Category],
            synchronize: true,
          }),
          TypeOrmModule.forFeature([Post]),
        ],
        providers: [PostsService],
      }).compile();
```

```
      service = module.get<PostsService>(PostsService);
    } catch (e) {
      console.error('✖ beforeAll failed:', e); // ← mostra l'errore vero
    }
  }, 15000);
  afterAll(async () => {
    if (module) await module.close();
  });
  it('should create and fetch posts', async () => {
    const post = await service.create({ title: 'Test', content: 'Contenuto' });
    createdPostId = post.id; // ← salva l'id
    const posts = await service.findAll();
    expect(posts.length).toBe(1);
  });
  it('should update a post', async () => {
    await service.update(createdPostId, { title: 'Titolo aggiornato' });
    const posts = await service.findAll();
    expect(posts[0].title).toBe('Titolo aggiornato');
  });
  it('should delete a post', async () => {
    await service.remove(createdPostId);
    const posts = await service.findAll();
    expect(posts.length).toBe(0);
  });
});
```

Async

```
@Get()
getUsers() {
  return this.users;
}
```

👉 risposta immediata ✅

Ma se fosse lento?

```
@Get()
getUsers() {
  const users = database.getUsers(); // lento
  return users;
}
```

👉 problema:
il database ci mette tempo
il server deve **aspettare**

```
@Get('attendi-async')
async attendiAsync() {
  await new Promise(resolve => setTimeout(resolve, 3000));
  return 'Async function risolta dopo 3 secondi';
}

@Get('attendi-promise')
attendi() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve('Promise risolta dopo 3 secondi');
    }, 3000);
  });
}

@Get('attendi-no-promise')
attendi_no_promise() {
  setTimeout(() => {
    return 'ciao';
  }, 3000);
}
```

Promise e async fanno la stessa cosa
async è solo più facile da leggere

la 3 opzione attendi-no-promise NON aspetta e ritorna 1 (id del timer)